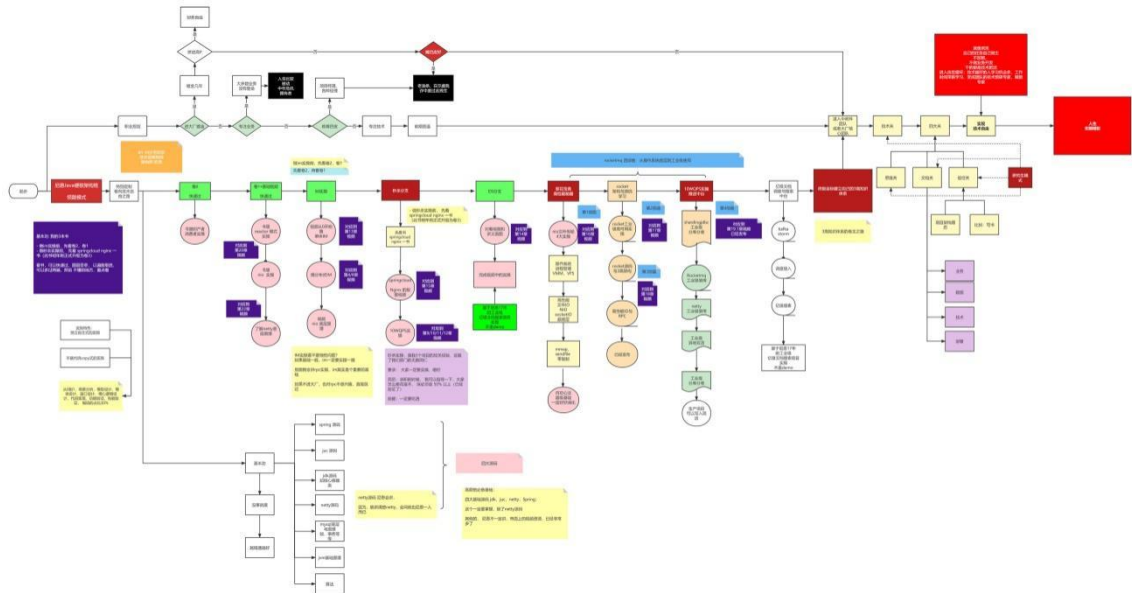


牛逼的职业发展之路

40 岁老架构尼恩用一张图揭秘：Java 工程师的高端职业发展路径，走向食物链顶端的之路

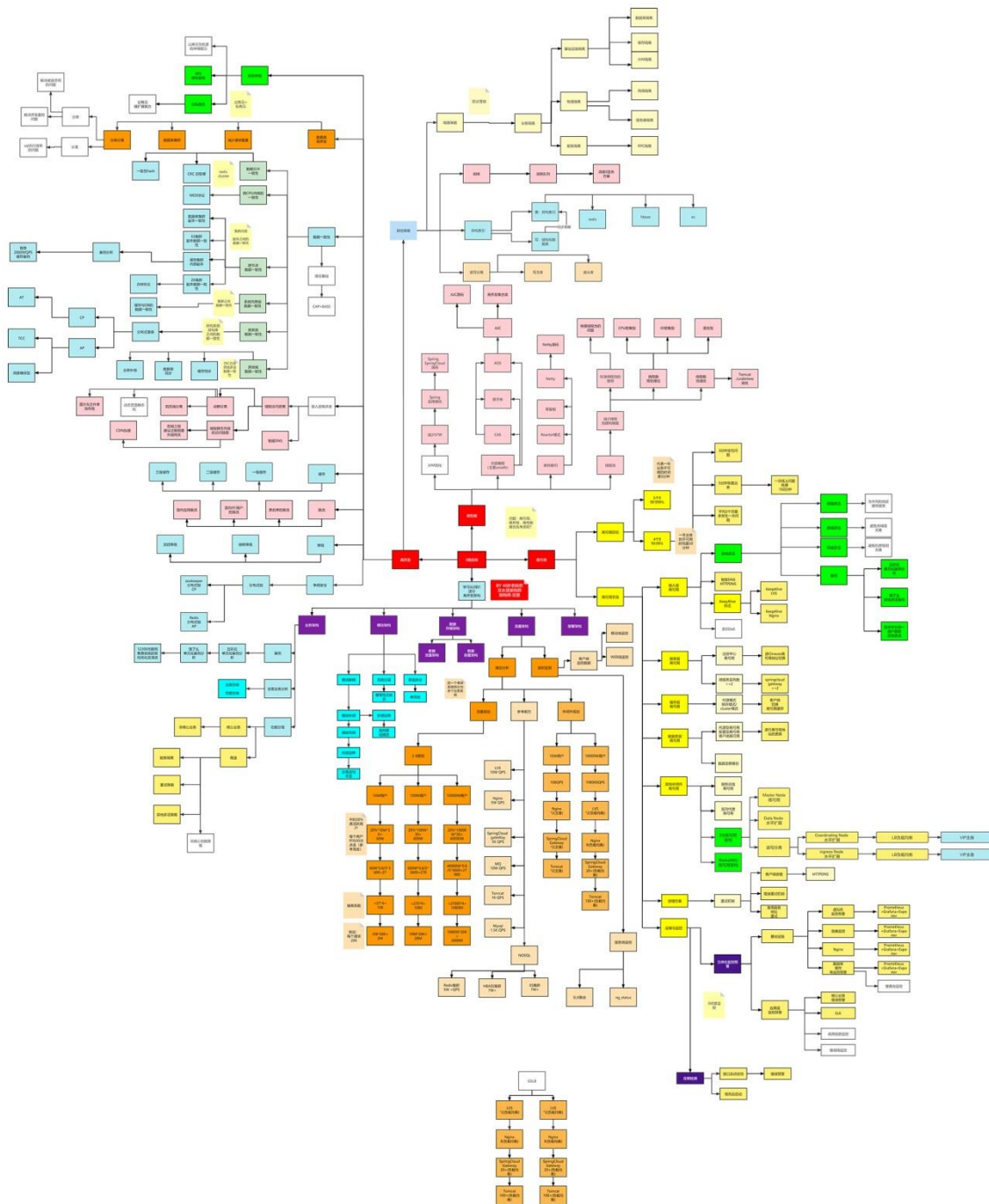
链接：<https://www.processon.com/view/link/618a2b62e0b34d73f7eb3cd7>



史上最全：价值10W的架构师知识图谱

此图梳理于尼恩的多个 3 高生产项目：多个亿级人民币的大型 SAAS 平台和智慧城市项目

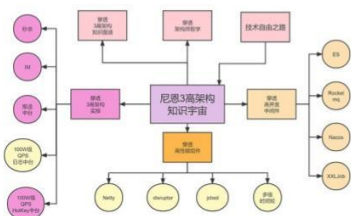
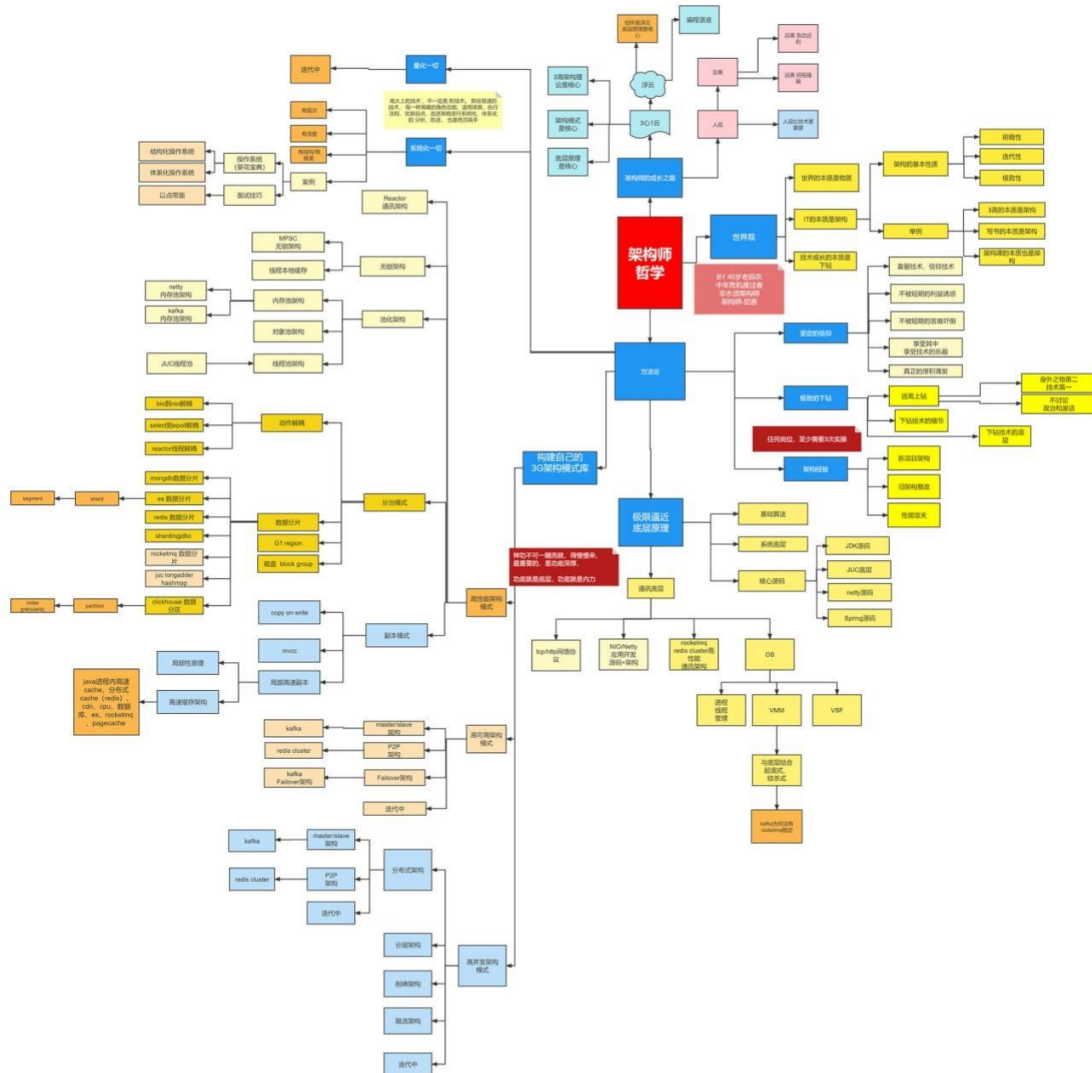
链接：<https://www.processon.com/view/link/60fb9421637689719d246739>



牛逼的架构师哲学

40 岁老架构师尼恩对自己的 20 年的开发、架构经验总结

链接: <https://www.processon.com/view/link/616f801963768961e9d9aec8>



牛逼的3高架构知识宇宙

尼恩 3 高架构知识宇宙，帮助大家穿透 3 高架构，走向技术自由，远离中年危机

链接: <https://www.processon.com/view/link/635097d2e0b34d40be778ab4>



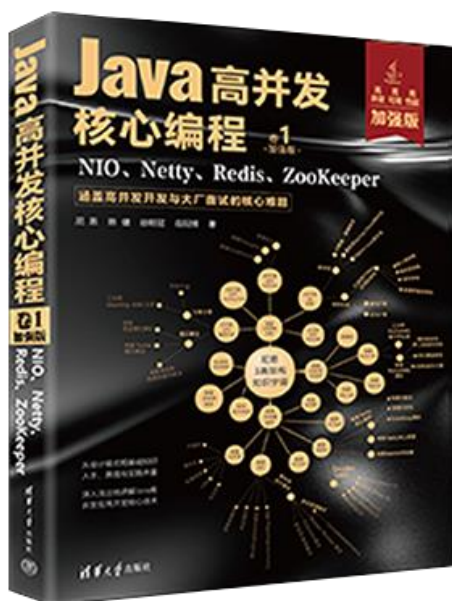
尼恩Java高并发三部曲（卷1加强版）

老版本：《Java 高并发核心编程 卷1：NIO、Netty、Redis、ZooKeeper》（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷1 **加强版**：NIO、Netty、Redis、ZooKeeper》

- 由浅入深地剖析了高并发 IO 的底层原理。
- 图文并茂地介绍了 TCP、HTTP、WebSocket 协议的核心原理。
- 细致深入地揭秘了 Reactor 高性能模式。
- 全面介绍了 Netty 框架，并完成单体 IM、分布式 IM 的实战设计。
- 详尽地介绍了 ZooKeeper、Redis 的使用，以帮助提升高并发、可扩展能力

详情：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



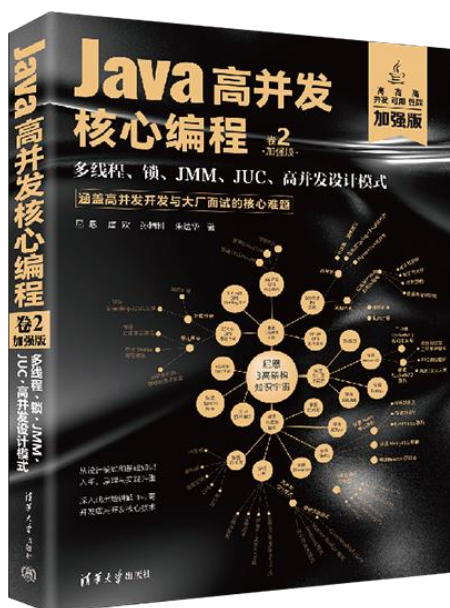
尼恩Java高并发三部曲（卷2加强版）

老版本：《Java 高并发核心编程 卷2：多线程、锁、JMM、JUC、高并发设计模式》
（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷2 **加强版**：多线程、锁、JMM、JUC、高并发设计模式》

- 由浅入深地剖析了 Java 多线程、线程池的底层原理。
- 总结了 IO 密集型、CPU 密集型线程池的线程数预估算法。
- 图文并茂地介绍了 Java 内置锁、JUC 显式锁的核心原理。
- 细致深入地揭秘了 JMM 内存模型。
- 全面介绍了 JUC 框架的设计模式与核心原理，并完成其高核心组件的实战介绍。
- 详尽地介绍了高并发设计模式的使用，以帮助提升高并发、可扩展能力

详情参阅：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



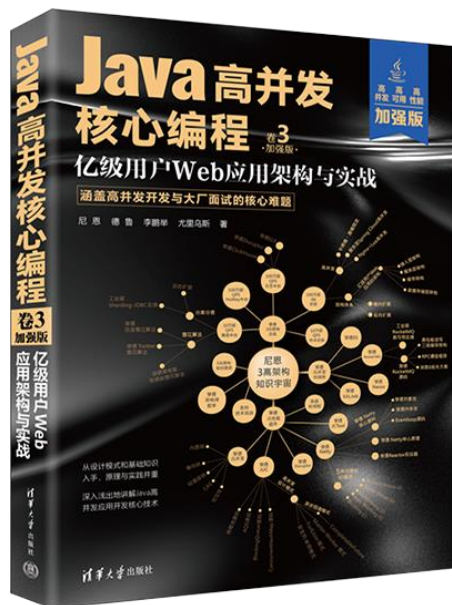
尼恩Java高并发三部曲（卷3加强版）

老版本：《SpringCloud Nginx 高并发核心编程》（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷3 **加强版**：亿级用户 Web 应用架构与实战》

- 在当今的面试场景中，3 高知识是大家面试必备的核心知识，本书基于亿级用户 3 高 Web 应用的架构分析理论，为大家对 3 高架构系统做一个系统化和清晰化的介绍。
- 从 Java 静态代理、动态代理模式入手，抽丝剥茧地解读了 SpringCloud 全家桶中 RPC 核心原理和执行过程，这是高级 Java 工程师面试必备的基础知识。
- 从Reactor 反应器模式入手，抽丝剥茧地解读了Nginx 核心思想和各配置项的底层知识和原理，这是高级 Java 工程师、架构师面试必备的基础知识。
- 从观察者模式入手，抽丝剥茧地解读了 RxJava、Hystrix 的核心思想和使用方法，这也是高级 Java 工程师、架构师面试必备的基础知识。

详情：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



尼恩Java面试宝典

35 个专题（卷王专供+ 史上最全 + 2023 面试必备）

详情：<https://www.cnblogs.com/crazymakercircle/p/13917138.html>



名称	
专题01: JVM面试题 (卷王专供 + 史上最全 + 2023面试必备) -V10-from-尼恩Java面试宝典-release.pdf	
专题02: Java算法面试题 (卷王专供 + 史上最全 + 2023面试必备) -V2 -from-Java面试红宝书-release.pdf	
专题03: Java基础面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书 -release.pdf	
专题04: 架构设计面试题 (卷王专供+ 史上最全 + 2023面试必备) -V10-from-Java面试红宝书-release.pdf	
专题05: Spring面试题_专题06: SpringMVC_专题07: Tomcat面试题 (卷王专供+ 史上最全 + 2023面试必备) -V3-from-尼恩面试宝典-release.pdf	
专题08: SpringBoot面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题09: 网络协议面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题10: TCP/IP协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题11: JUC并发包与容器类 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题12: 设计模式面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题13: 死锁面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题14: Redis 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V5-from-Java面试红宝书-release.pdf	
专题15: 分布式锁 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题16: Zookeeper 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题17: 分布式事务面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题18: 一致性协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题19: Zab协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题20: Paxos 协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题21: raft 协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题22: Linux面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题23: Mysql 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V11-from-Java面试红宝书-release.pdf	
专题24: SpringCloud 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V12-from-Java面试红宝书-release.pdf	
专题25: Netty 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题26: 消息队列面试题: RabbitMQ、Kafka、RocketMQ (卷王专供+ 史上最全 + 2023面试必备) -V10-from-Java面试红宝书-release.pdf	
专题27: 内存泄漏 内存溢出 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题28: JVM 内存溢出 实战 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题29: 多线程面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题30: HR面试题: 过五关斩六将后, 小心阴沟翻船! (史上最全、避坑宝典) -V2-from-Java面试红宝书-release.pdf	
专题31: Hash链表面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题32: 大厂面试的基本流程和面试准备 (阿里、腾讯、网易、京东、头条.....) -V2-from-Java面试红宝书-release.pdf	
专题33: BST、AVL、RBT红黑树、三大核心数据结构 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题34: Elasticsearch面试题 (卷王专供+ 史上最全 + 2023面试必备) -V3-from-Java面试红宝书-release.pdf	
专题35: Mybatis面试题 (卷王专供+ 史上最全 + 2023面试必备) -V3-from-尼恩Java面试宝典-release.pdf	

高性能核心组件穿透的意义

要想成为高手，就要穿透式、起底式、绞杀式的掌握 顶级组件、王者组件。

唯有如此，才能成为技术王者。

从中吸取思想和精华，为大家自己的业务CRUD所有。

接下来，和尼恩一起，开始穿透之旅吧：

- 高性能核心组件之1：

穿透“**IO之王 Netty**” 架构和源码

- 穿透高性能核心组件之2：

“**队列之王 Disruptor**” 架构和源码（Disruptor 红宝书）

- 穿透高性能核心组件之3：

穿透“**缓存之王 Caffeine**” 架构和源码（Caffeine 红宝书）

- 穿透高性能核心组件之4：

穿透“**链路之王 Skywalking**” 架构和源码

此文，带大家 穿透高性能核心组件之2：

队列之王 Disruptor” 架构和源码（Disruptor 红宝书）

所以，此文又名：

Disruptor 红宝书

文章会不断升级，最新版本，请关注尼恩的朋友圈。

disruptor 红宝书目的：

作为Java领域**最高性能的 队列**，没有之一，大家不光要懂，而是 **需要深入骨髓的搞懂**。

所以，给大家奉上了本书，并且配备了视频进行 详细介绍，目的：

帮助，大家穿透 一个 绝对核心高性能 Java高性能的 队列 的架构和原理，让 **面试题 五体投地、顶礼膜拜**。

本书目录

[高性能核心组件穿透的意义](#)

disruptor 红宝书目的:

本书目录

队列之王 Disruptor 简介

Java内置队列的问题

Disruptor 的使用场景

1 前置知识：伪共享原理与实操

CPU的结构

缓存行 cache line

什么是 伪共享 (False Sharing) 问题?

伪共享问题 (False Sharing) 的本质

伪共享问题 的解决方案

一个缓冲行填充的例子

伪共享False Sharing在java 8中解决方案

伪共享性能比对实操：结论，差6倍

存在伪共享场景下的 耗时计算

消除伪共享场景下的 耗时计算

使用unsafe访问变量的耗时计算

性能总结

JDK源码中如何解决 伪共享问题

LongAdder以及 Striped64如何解决伪共享问题

Cell元素如何消除伪共享

对于伪共享，我们在实际开发中该怎么做?

3 Disruptor框架是如何解决伪共享问题的?

Sequence的结构和源码

2: Disruptor 的使用实战

定义一个Event和工厂

定义事件处理器 (消费者)

定义事件源(生产者)

组装起来

事件转换器

通过Java 8 Lambda使用Disruptor

构造Disruptor对象的几个要点

Disruptor如何实现高性能?

Disruptor和BlockingQueue比较:

原理: Disruptor 的内部Ring Buffer环形队列

RingBuffer是什么

Disruptor使用环形队列的优势:

关闭Disruptor

3: Disruptor 的使用场景分析

Disruptor 使用细分场景

单生产者多消费者并行场景

ProducerType 的类型

单生产者多消费者并行场景的参考代码

多生产者单消费者场景

ProducerType 的类型

多生产者场景的要点

多生产者场景的参考代码

单生产者多消费者竞争场景

disruptor如何设置多个竞争消费者?

演示代码如下:

多个消费者串行消费场景

多个消费者串行消费场景案例

执行结果

菱形方式执行场景

场景特点

菱形方式执行场景案例

链式并行执行场景

场景特点

- 场景案例
 - 多组消费者相互隔离场景
 - 场景特点
 - 场景案例
- 多组消费者航道执行模式
 - 场景特点
 - 场景案例
- 六边形执行顺序
 - 场景特点
 - 参考代码
 - 执行结果

4: 架构师视角，深入Disruptor源码分析

核心概念

- Ring Buffer
- Sequence
- Sequencer
- Sequence Barrier
- Wait Strategy
- Event
- EventProcessor
- EventHandler
- Producer
- RingBuffer
- Disruptor

Disruptor的无锁架构

- 如何管理消费者和生产者之间的依赖关系呢？**
- 如何管理消费者与消费者之间的依赖关系呢？
- 依赖关系管理的例子
- 如何避免未消费事件的写入覆盖呢？**

Disruptor的总体模块架构

核心类Sequence

核心类Sequencer

- Sequencer 的实现类
- SingleProducerSequencer
- MultiProducerSequencer

核心类消费者仓库 和消费者信息

- 消费者仓库 ConsumerRepository
- 消费者的信息 ConsumerInfo
- 消费者的信息 实现类
- WorkerPoolInfo多线程消费者信息

消费者处理器

- BatchEventProcessor单线程批处理事件
- 事件消费者组EventHandlerGroup

SequenceBarrier协调屏障

- disruptor管理两种依赖关系
- 消除锁和CAS操作
- SequenceBarrier的几个方法
- SequenceBarrier进行依赖消费
- ProcessingSequenceBarrier

RingBuffer预分配内存

Disruptor的等待策略

- uml图
- SleepingWaitStrategy
 - 源码
- YieldingWaitStrategy
 - 源码
- BusySpinWaitStrategy

Disruptor 的缓存行填充

队列之王 Disruptor 简介

Disruptor是英国外汇交易公司LMAX开发的一个高性能队列，研发的初衷是解决内存队列的延迟问题（在性能测试中发现竟然与I/O操作处于同样的数量级）。

基于Disruptor开发的系统单线程能支撑每秒600万订单，2010年在QCon演讲后，获得了业界关注。

2011年，企业应用软件专家Martin Fowler专门撰写长文介绍Disruptor。

2011年，Disruptor还获得了Oracle官方的**Duke大奖**。

目前，包括Apache Storm、Camel、Log4j 2在内的很多知名项目都应用了Disruptor以获取高性能。

要深入了解 disruptor ，咱们从 Java的 内置队列开始介绍起。

Java内置队列的问题

介绍Disruptor之前，我们先来看一看常用的线程安全的内置队列有什么问题。

Java的内置队列如下表所示。

队列	有界性	锁	数据结构
ArrayBlockingQueue	bounded	加锁	arraylist
LinkedBlockingQueue	optionally-bounded	加锁	linkedList
ConcurrentLinkedQueue	unbounded	无锁	linkedList
LinkedTransferQueue	unbounded	无锁	linkedList
PriorityBlockingQueue	unbounded	加锁	heap
DelayQueue	unbounded	加锁	heap

队列的底层一般分成三种：数组、链表和堆。

其中，堆一般情况下是为了实现带有优先级特性的队列，

暂时不做介绍，后面讲netty 定时任务的时候，再介绍。

从数组和链表两种数据结构来看，两类结构如下：

- 基于数组线程安全的队列，比较典型的是ArrayBlockingQueue，它主要通过加锁的方式来保证线程安全；
- 基于链表的线程安全队列分成LinkedBlockingQueue和ConcurrentLinkedQueue两大类，前者也通过锁的方式来实现线程安全，而后者通过原子变量compare and swap（以下简称“CAS”）这种**无锁方式**来实现的。

和ConcurrentLinkedQueue一样，上面表格中的LinkedTransferQueue都是通过原子变量compare and swap（以下简称“CAS”）这种不加锁的方式来实现的

但是，对 volatile类型的变量进行 CAS 操作，存在伪共享问题，

Disruptor 的使用场景

Disruptor 它可以用来作为高性能的有界内存队列， 适用于两大场景：

- 生产者消费者场景
- 发布订阅 场景

生产者消费者场景。Disruptor的最常用的场景就是“生产者-消费者”场景，对场景的就是“一个生产者、多个消费者”的场景，并且要求顺序处理。

备注，这里和JCTool 的 MPSC 队列，刚好相反， MPSC 使用于多生产者，单消费者场景

发布订阅 场景：Disruptor也可以认为是观察者模式的一种实现， 实现发布订阅模式。

当前业界开源组件使用Disruptor的包括Log4j2、Apache Storm等，

1 前置知识：伪共享原理与实操

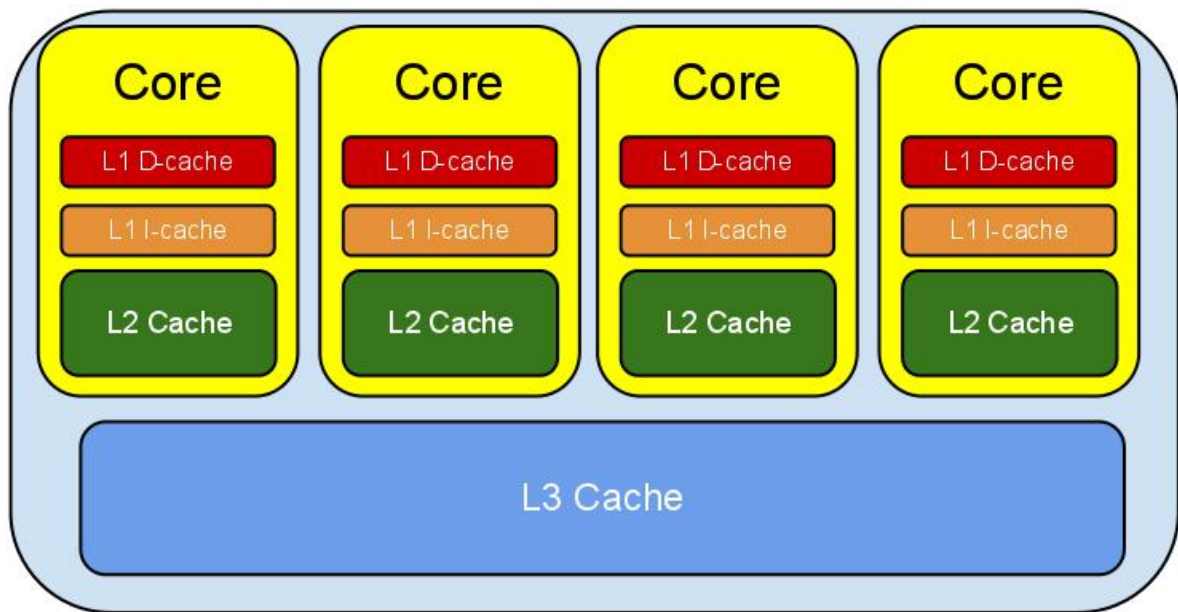
在介绍 无锁框架 disruptor 之前，作为前置的知识，首先给大家介绍 伪共享 原理&性能对比实战。

CPU的结构

下图是计算的基本结构。

L1、L2、L3分别表示一级缓存、二级缓存、三级缓存，越靠近CPU的缓存，速度越快，容量也越小。

- L1缓存很小但很快，并且紧靠着在使用它的CPU内核；
- L2大一些，也慢一些，并且仍然只能被一个单独的CPU核使用；
- L3更大、更慢，并且被单个插槽上的所有CPU核共享；
- 最后是主存，由全部插槽上的所有CPU核共享。



级别越小的缓存，越接近CPU，意味着速度越快且容量越少。

L1是最接近CPU的，它容量最小（比如256个字节），速度最快，

每个核上都有一个L1 Cache(准确地说每个核上有两个L1 Cache，一个存数据 L1d Cache，一个存指令 L1i Cache)；

L2 Cache 更大一些（比如256K个字节），速度要慢一些，一般情况下每个核上都有一个独立的L2 Cache；

二级缓存就是一级缓存的存储器：

一级缓存制造成本很高因此它的容量有限，二级缓存的作用就是存储那些CPU处理时需要用到、一级缓存又无法存储的数据。

L3 Cache是三级缓存中最大的一级，例如（比如12MB个字节），同时也是最慢的一级，在同一个CPU插槽之间的核共享一个L3 Cache。

三级缓存和内存可以看作是二级缓存的存储器，它们的容量递增，但单位制造成本却递减。

L3 Cache和L1，L2 Cache有着本质的区别。

L1和L2 Cache都是每个CPU core独立拥有一个，而L3 Cache是几个Cores共享的，可以认为是一个更小但是更快的内存。

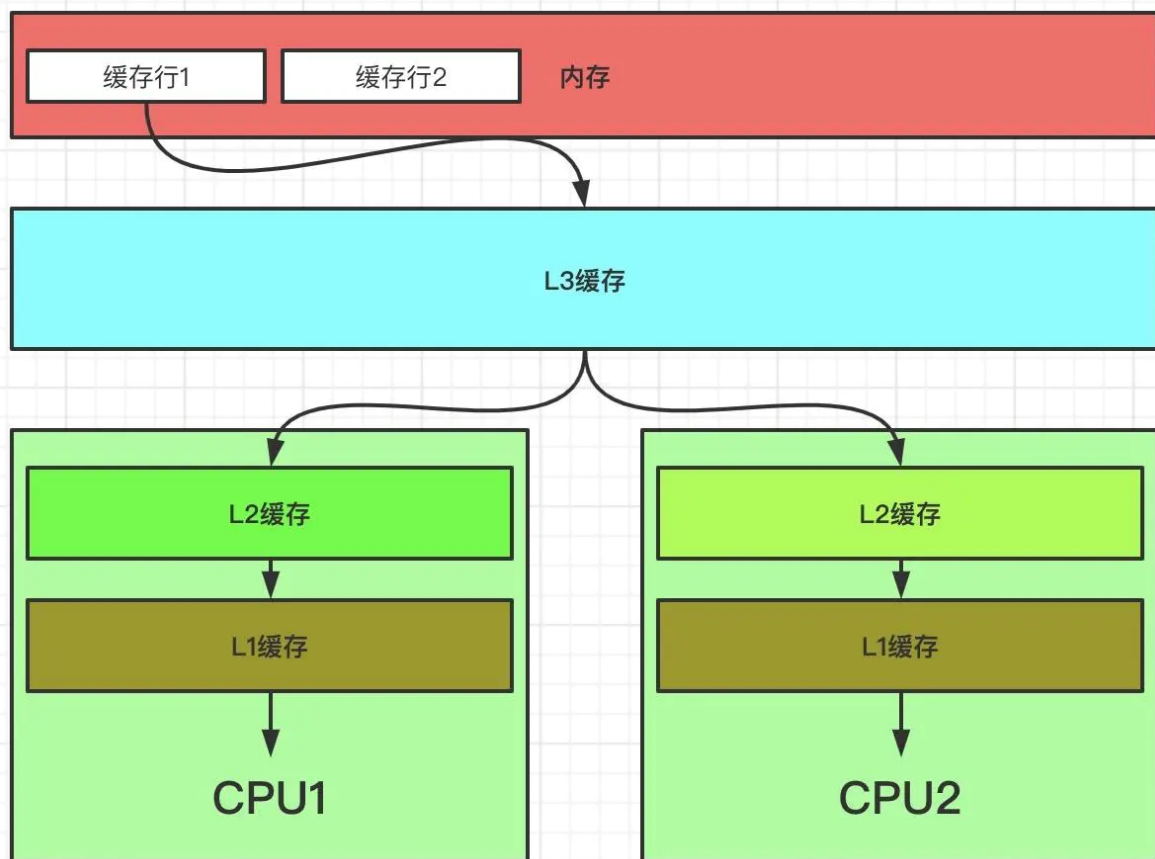
缓存行 cache line

为了提高IO效率，CPU每次从内存读取数据，并不是只读取我们需要计算的数据，而是一批一批去读取的，这一批数据，也叫Cache Line（缓存行）。

也可以理解为**批量读取，提升性能**。为啥要一批、一批的读取呢？这也满足**空间的局部性原理**（具体请参见葵花宝典）。

从读取的角度来说，缓存，是由缓存行Cache Line组成的。

所以使用缓存时，并不是一个一个字节使用，而是一行缓存行、一行缓存行这样使用；



换句话说，CPU存取缓存都是按照一行，为最小单位操作的。并不是按照字节为单位，进行操作的。

一般而言，读取一行数据时，是将我们需要的数据周围的连续数据一次性全部读取到缓存中。这段连续的数据就称为一个**缓存行**。

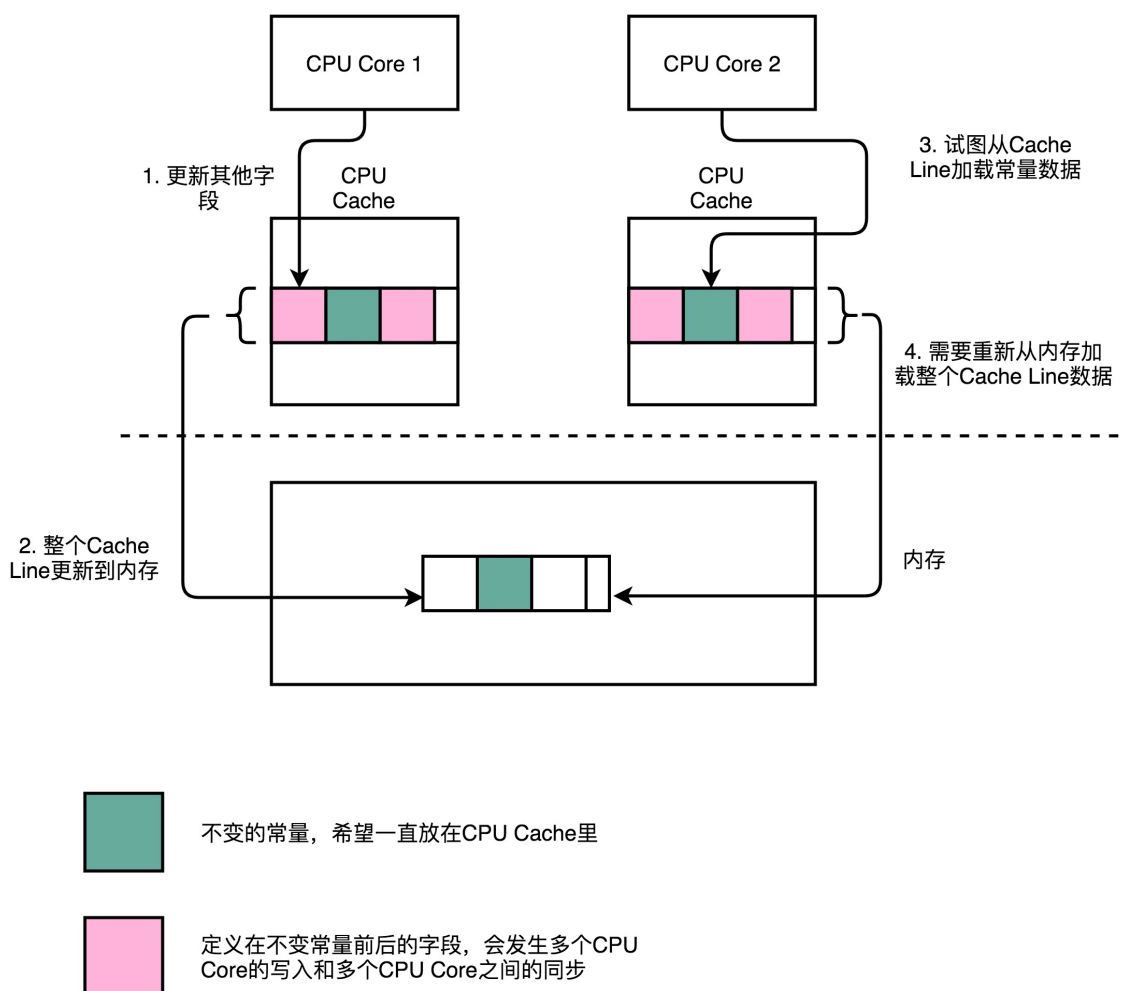
一般一行缓存行有64字节。intel处理器的缓存行是64字节。目前主流的CPU Cache的Cache Line大小都是64Bytes。

假设我们有一个512 Bytes 的一级缓存，那么按照64 Bytes 的缓存单位大小来算，这个一级缓存所能存放的缓存个数就是 $512/64 = 8$ 个。

所以，Cache Line可以简单的理解为CPU Cache中的最小缓存单位。

这些CPU Cache的写回和加载，都不是以一个变量作为单位。这些都是以整个Cache Line作为单位。

如果一个常量和变量放在一行，那么变量的更新，也会影响常量的使用：



CPU在加载数据时，整个缓存行过期了，加载常量的时候，自然也会把这个数据从内存加载到高速缓存。

什么是 伪共享（False Sharing）问题？

提前说明：翻译有瑕疵，伪共享（False Sharing），应该翻译为“错共享”，才更准确

CPU的缓存系统是以缓存行(cache line)为单位存储的，一般的大小为64bytes。

在多线程程序的执行过程中，存在着一种情况，多个需要频繁修改的变量存在同一个缓存行当中。

假设：有两个线程分别访问并修改X和Y这两个变量，X和Y恰好在同一个缓存行上，这两个线程分别在不同的CPU上执行。

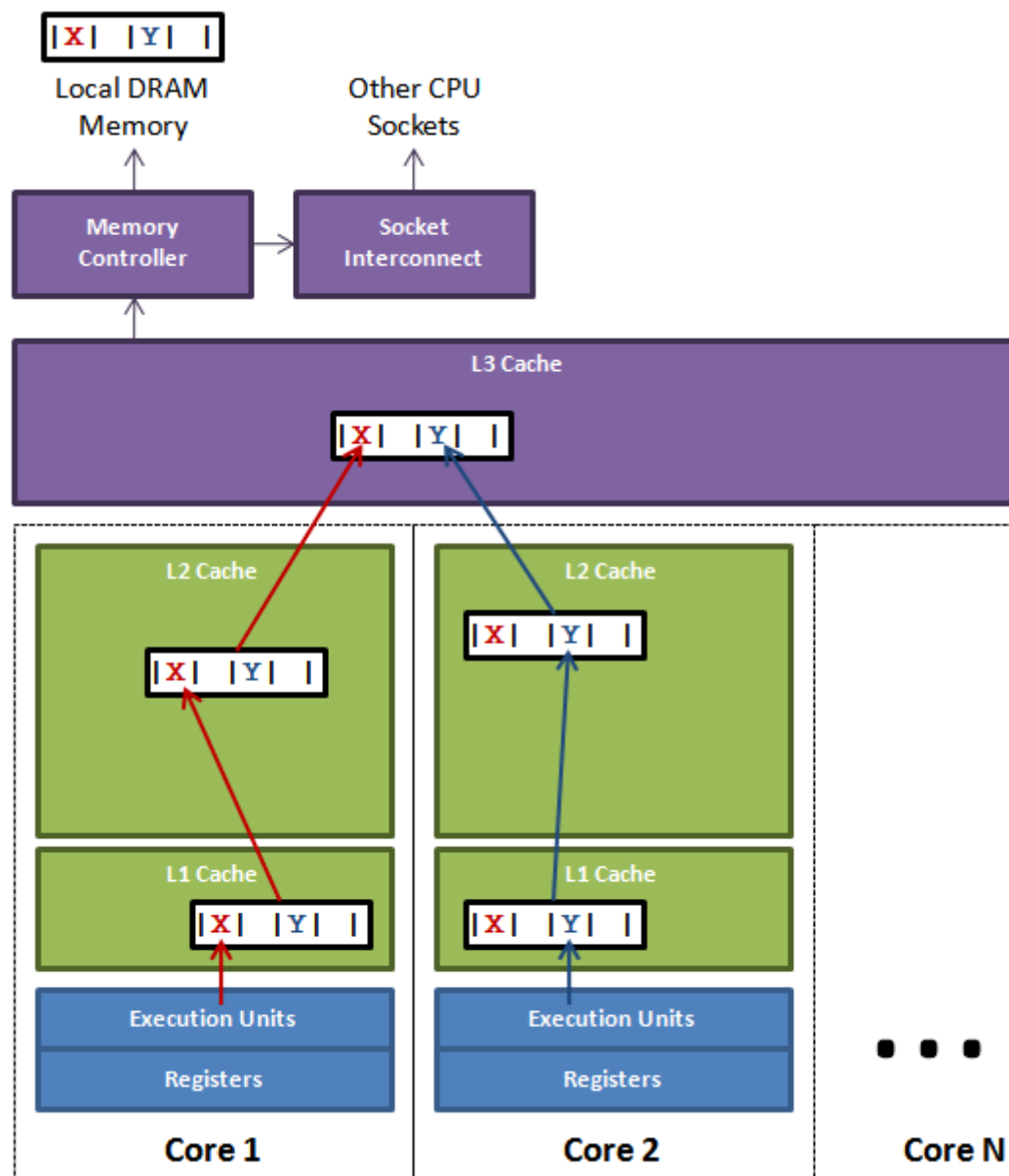
那么每个CPU分别更新好X和Y时将缓存行刷入内存时，发现有别的修改了各自缓存行内的数据，这时缓存行会失效，从L3中重新获取。

这样的话，程序执行效率明显下降。

为了减少这种情况的发生，其实就是避免X和Y在同一个缓存行中，

如何操作呢？可以主动添加一些无关变量将缓存行填满，

比如在X对象中添加一些变量，让它有64 Byte那么大，正好占满一个缓存行。



两个线程（Thread1 和 Thread2）同时修改一个同一个缓存行上的数据 X Y:

如果线程1打算更改a的值，而线程2准备更改b的值：

```
Thread1: x=3;
```

```
Thread2: y=2;
```

由x值被更新了，所以x值需要在线程1和线程2之间传递（从线程1到线程2），

x、y的变更，都会引起 cache line 整块 64 bytes 被刷新，因为cpu核之间以cache line的形式交换数据(cache lines的大小一般为64bytes)。

在并发执行的场景下，每个线程在不同的核中被处理。

假设 x,y是两个频繁修改的变量，x,y，还位于同一个缓存行。

如果，CPU1修改了变量x时，L3中的缓存行数据就失效了，也就是CPU2中的缓存行数据也失效了，CPU2需要的y需要重新从内存加载。

如果，CPU2修改了变量y时，L3中的缓存行数据就失效了，也就是CPU1中的缓存行数据也失效了，CPU1需要的x需要重新从内存加载。

x,y在两个cpu上进行修改，本来应该是互不影响的，但是由于缓存行在一起，导致了相互受到了影响。

伪共享问题（False Sharing）的本质

出现伪共享问题（False Sharing）的原因：

- 一个缓存行可以存储多个变量（存满当前缓存行的字节数）；64个字节可以放8个long，16个int
- 而CPU对缓存的修改又是以缓存行为最小单位的；不是以long、byte这样的数据类型为单位的
- 在多线程情况下，如果需要修改“共享同一个缓存行的其中一个变量”，该行中其他变量的状态就会失效，甚至进行一致性保护

所以，伪共享问题（False Sharing）的本质是：

对缓存行中的单个变量进行修改了，导致整个缓存行其他不相关的数据也就失效了，需要从主存重新加载

如果其中有volatile修饰的变量，需要保证线程可见性的变量，还需要进入缓存与数据一致性的保障流程，如mesi协议的数据一致性保障用了其他变量的Core的缓存一致性。

缓存一致性是根据缓存行为单元来进行同步的，假如y是volatile类型的，假如a修改了x，而其他的线程用到y，虽然用到的不是同一个数据，但是他们（数据X和数据Y）在同一个缓存行中，其他的线程的缓存需要保障数据一致性而进行数据同步，当然，同步也需要时间。

一个CPU核心在加载一个缓存行时要执行上百条指令。如果一个核心要等待另外一个核心来重新加载缓存行，那么他就必须等在那里，称之为stall(停止运转)。

伪共享问题的解决方案

减少伪共享也就意味着减少了stall的发生，其中一个手段就是通过填充(Padding)数据的形式，来保证本应有可能位于同一个缓存行的两个变量，在被多线程访问时必定位于不同的缓存行。

简单的说，就是以**空间换时间**：使用占位字节，将变量的所在的缓冲行塞满。

disruptor 无锁框架就是这么干的。

一个缓冲行填充的例子

下面是一个填充了的缓存行的，尝试p1, p2, p3, p4, p5, p6为AtomicLong的value的缓存行占位，将AtomicLong的value变量的所在的缓冲行塞满，

代码如下：

```
package com.crazymakercircle.demo.cas;

import java.util.concurrent.atomic.AtomicLong;
```

```

public class PaddedAtomicLong extends AtomicLong {
    private static final long serialVersionUID = -3415778863941386253L;

    /**
     * Padded 6 long (48 bytes)
     */
    public volatile long p1, p2, p3, p4, p5, p6 = 7L;

    /**
     * Constructors from {@link AtomicLong}
     */
    public PaddedAtomicLong() {
        super();
    }

    public PaddedAtomicLong(long initialValue) {
        super(initialValue);
    }

    /**
     * To prevent GC optimizations for cleaning unused padded references
     */
    public long sumPaddingToPreventOptimization() {
        return p1 + p2 + p3 + p4 + p5 + p6;
    }
}

```

例子的部分结果如下:

```

printable = com.crazymakercircle.basic.demo.cas.busi.PaddedAtomicLong object
internals:

```

OFFSET	SIZE	TYPE	DESCRIPTION	VALUE
0	4		(object header)	01 00 00 00
(00000001 00000000 00000000 00000000) (1)				
4	4		(object header)	00 00 00 00
(00000000 00000000 00000000 00000000) (0)				
8	4		(object header)	50 08 01 f8
(01010000 00001000 00000001 11111000) (-134150064)				
12	4		(alignment/padding gap)	
16	8	long	AtomicLong.value	0
24	8	long	PaddedAtomicLong.p1	0
32	8	long	PaddedAtomicLong.p2	0
40	8	long	PaddedAtomicLong.p3	0
48	8	long	PaddedAtomicLong.p4	0
56	8	long	PaddedAtomicLong.p5	0
64	8	long	PaddedAtomicLong.p6	7

Instance size: 72 bytes

Space losses: 4 bytes internal + 0 bytes external = 4 bytes total

伪共享False Sharing在java 8中解决方案

JAVA 8中添加了一个@Contended的注解, 添加这个的注解, 将会在自动进行缓存行填充。

```
Striped64
g.java x AtomicLong.java x FalseShareTest.java x LongAdder.java x Striped64.java x ContendedDemo.java
116
117 * JVM intrinsics note: It would be possible to use a release-only
118 * form of CAS here, if it were provided.
119 */
120 @sun.misc.Contended static final class Cell {
121     volatile long value;
122     Cell(long x) { value = x; }
123     final boolean cas(long cmp, long val) { return UNSAFE.compareAndSwapLong(this, valueOffset, cmp, val); }
124
125     // Unsafe mechanics
126     private static final sun.misc.Unsafe UNSAFE;
127     private static final long valueOffset;
128     static {
129         try {
130             UNSAFE = sun.misc.Unsafe.getUnsafe();
131             Class<?> ak = Cell.class;
132             valueOffset = UNSAFE.objectFieldOffset(ak.getDeclaredField("value"));
133         } catch (Exception e) {
134             throw new Error(e);
135         }
136     }
137 }
138
139
140
141
142 /** Number of CPUs, to place bound on table size */
```

CSDN @架构师-尼恩

下面有一个@Contended的例子：

```
package com.crazymakercircle.basic.demo.cas.busi;
import sun.misc.Contended;
public class ContendedDemo
{
    //有填充的演示成员
    @Contended
    public volatile long padVar;

    //没有填充的演示成员
    public volatile long notPadVar;
}
```

以上代码使得padVar和notPadVar都在不同的cache line中。@Contended 使得notPadVar字段远离了对象头部分。

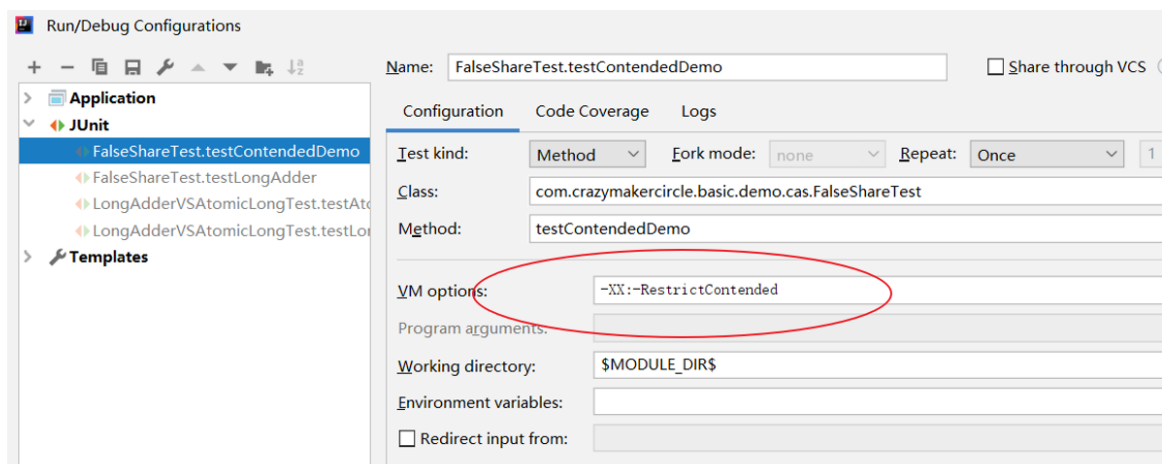
```

printable = com.crazymakercircle.basic.demo.cas.busi.ContendedDemo object
internals:
  OFFSET  SIZE  TYPE DESCRIPTION                               VALUE
      0    4      (object header)                01 00 00 00 (00000001 00000000
00000000 00000000) (1)
      4    4      (object header)                00 00 00 00 (00000000 00000000
00000000 00000000) (0)
      8    4      (object header)                50 08 01 f8 (01010000 00001000
00000001 11111000) (-134150064)
     12    4      (alignment/padding gap)
     16    8    long ContendedDemo.padVar          0
     24    8    long ContendedDemo.notPadVar        0
Instance size: 32 bytes
Space losses: 4 bytes internal + 0 bytes external = 4 bytes total

```

执行时，必须加上虚拟机参数-XX:-RestrictContended，@Contended注释才会生效。

很多文章把这个漏掉了，那样的话实际上就没有起作用。



新的结果；

```

printable = com.crazymakercircle.basic.demo.cas.busi.ContendedDemo object
internals:
  OFFSET  SIZE  TYPE DESCRIPTION                               VALUE
      0    4      (object header)                01 00 00 00 (00000001 00000000
00000000 00000000) (1)
      4    4      (object header)                00 00 00 00 (00000000 00000000
00000000 00000000) (0)
      8    4      (object header)                50 08 01 f8 (01010000 00001000
00000001 11111000) (-134150064)
     12    4      (alignment/padding gap)
     16    8    long ContendedDemo.notPadVar          0
     24   128      (alignment/padding gap)
    152    8    long ContendedDemo.padVar          0
    160   128      (loss due to the next object alignment)
Instance size: 288 bytes
Space losses: 132 bytes internal + 128 bytes external = 260 bytes total

```

在Java 8中，使用@Contended注解的对象或字段的前后各增加128字节大小的padding，使用2倍于大多数硬件缓存行的大小来避免相邻扇区预取导致的伪共享冲突。我们目前的缓存行大小一般为64Byte，这里Contended注解为我们前后加上了128字节绰绰有余。

注意：如果想要@Contended注解起作用，需要在启动时添加JVM参数-XX:-RestrictContended参数后 @sun.misc.Contended 注解才有。

可见至少在JDK1.8以上环境下，只有@Contended注解才能解决伪共享问题，但是消耗也很大，占用了宝贵的缓存，用的时候要谨慎。

另外：

@Contended 注释还可以添加在类上，每一个成员，都会加上。

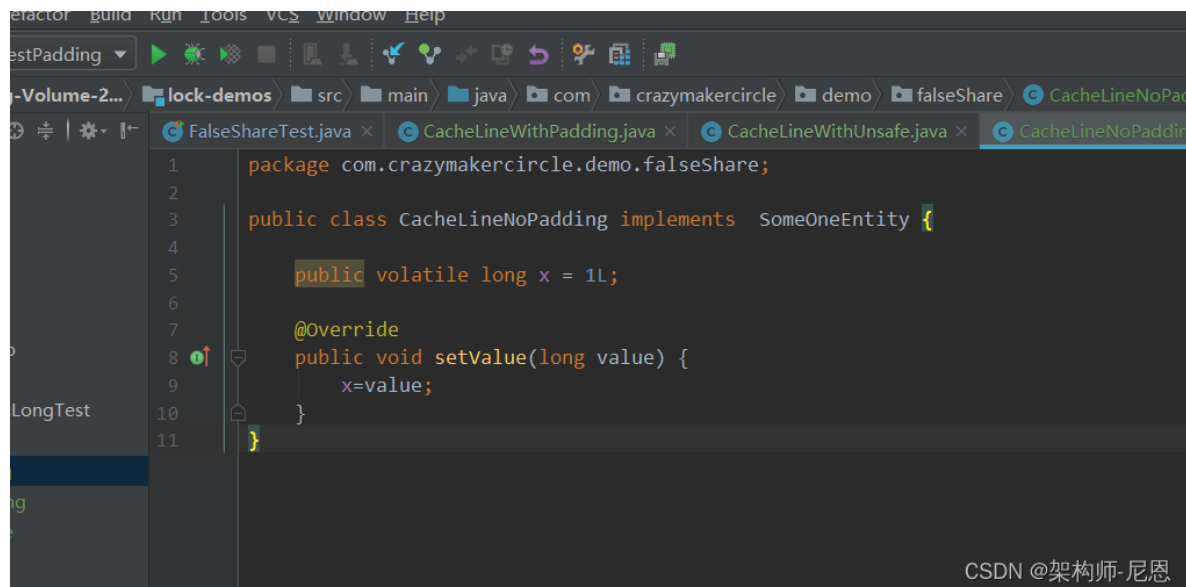
伪共享性能比对实操：结论，差6倍

三个实操：

- 首先存在伪共享场景下的 耗时计算
- 其次是消除伪共享场景下的 耗时计算
- 再次是使用unsafe访问变量时的耗时计算

存在伪共享场景下的 耗时计算

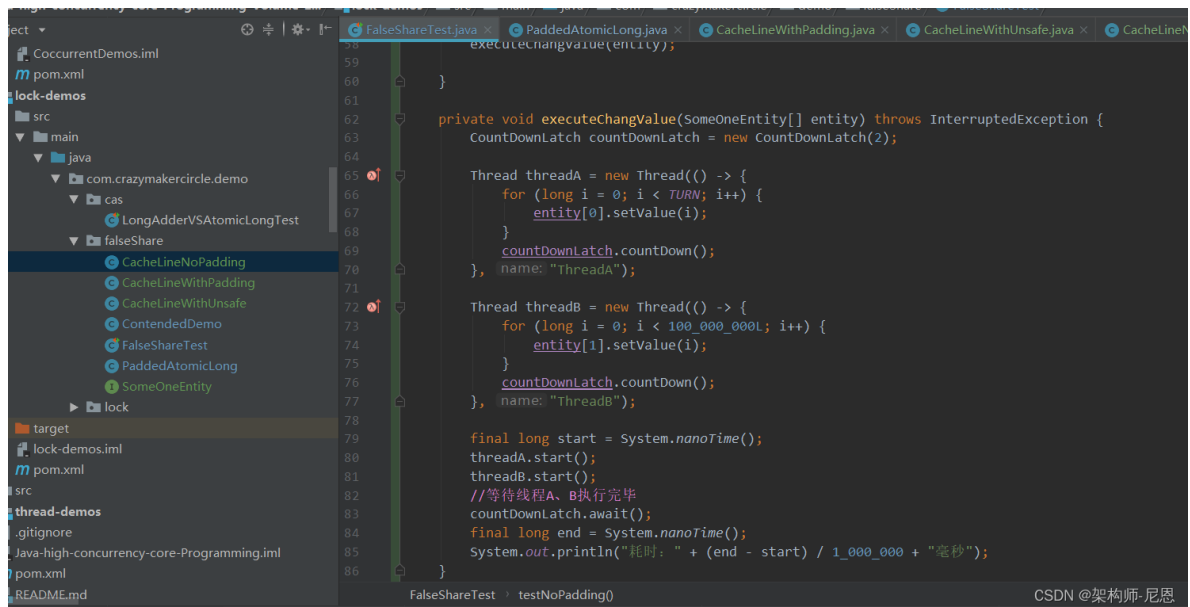
entity类



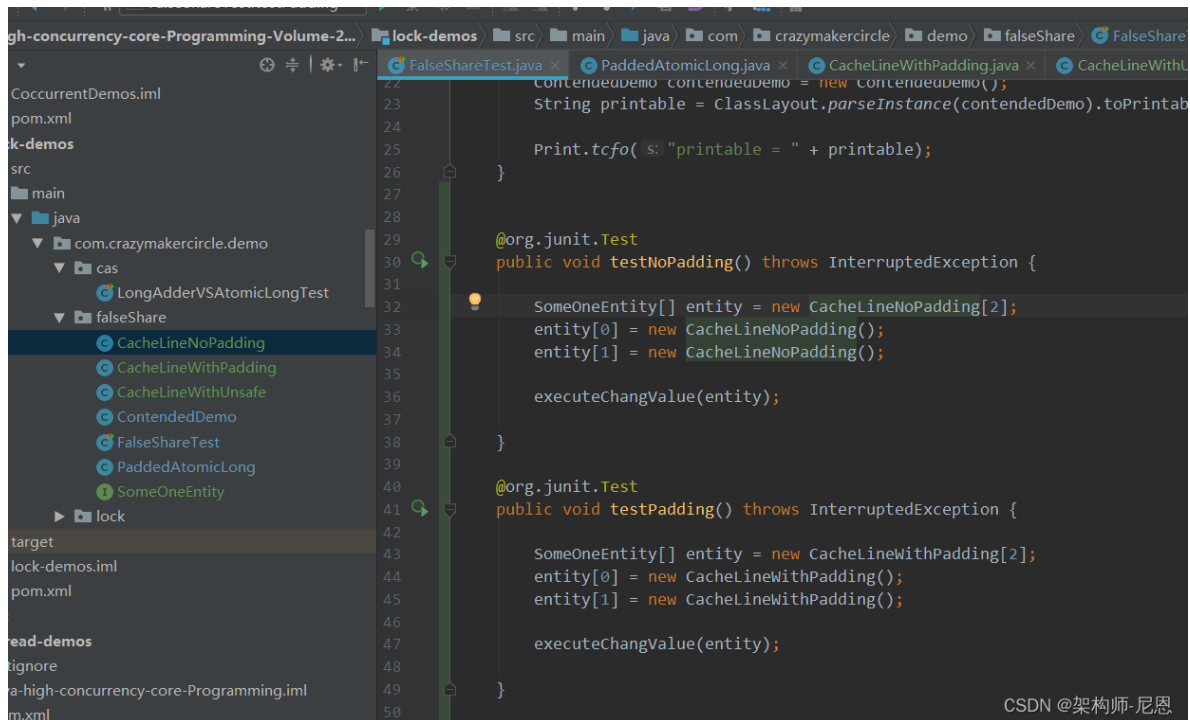
```
1 package com.crazymakercircle.demo.falseShare;
2
3 public class CacheLineNoPadding implements SomeoneEntity {
4
5     public volatile long x = 1L;
6
7     @Override
8     public void setValue(long value) {
9         x=value;
10    }
11 }
```

CSDN @架构师-尼恩

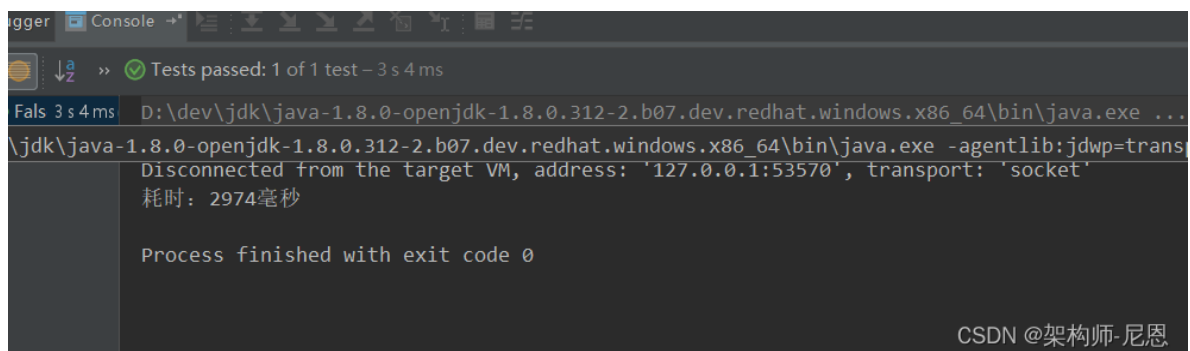
并行的执行数据修改，这里抽取成为了一个通用的方法



测试用例

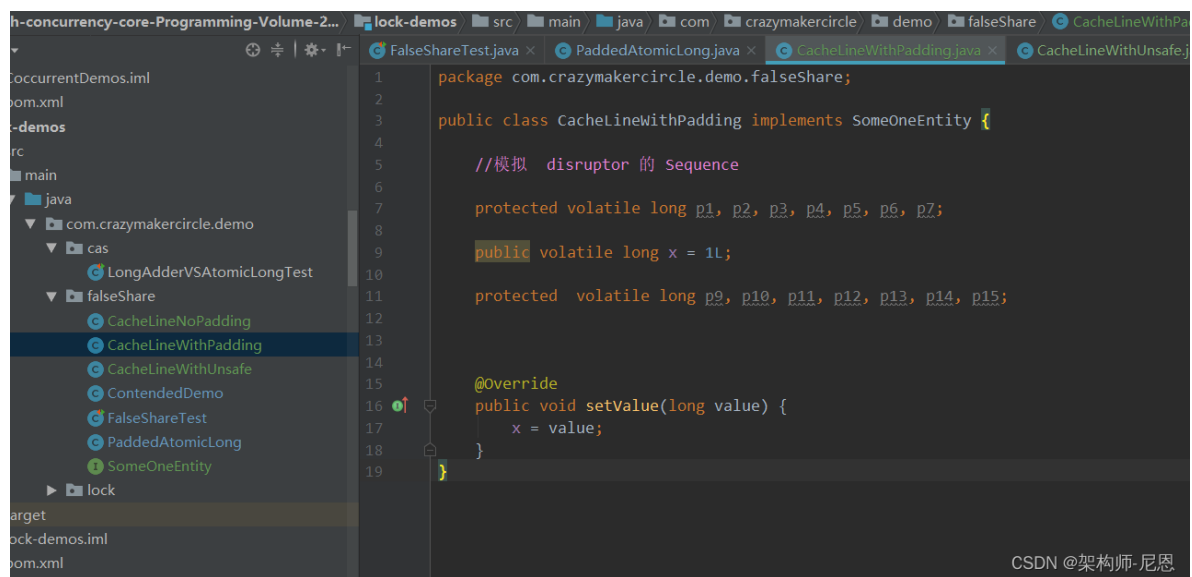


执行的时间



消除伪共享场景下的 耗时计算

entity类



The screenshot shows an IDE with a project named 'lock-demos'. The left sidebar displays a file tree with the following structure:

- com.crazymakercircle.demo
 - cas
 - LongAdderVSAAtomicLongTest
 - falseShare
 - CacheLineNoPadding
 - CacheLineWithPadding (selected)
 - CacheLineWithUnsafe
 - ContendedDemo
 - FalseShareTest
 - PaddedAtomicLong
 - SomeOneEntity

The main editor displays the code for `CacheLineWithPadding` in the `com.crazymakercircle.demo.falseShare` package. The code is as follows:

```
package com.crazymakercircle.demo.falseShare;

public class CacheLineWithPadding implements SomeOneEntity {

    //模拟 disruptor 的 Sequence

    protected volatile long p1, p2, p3, p4, p5, p6, p7;

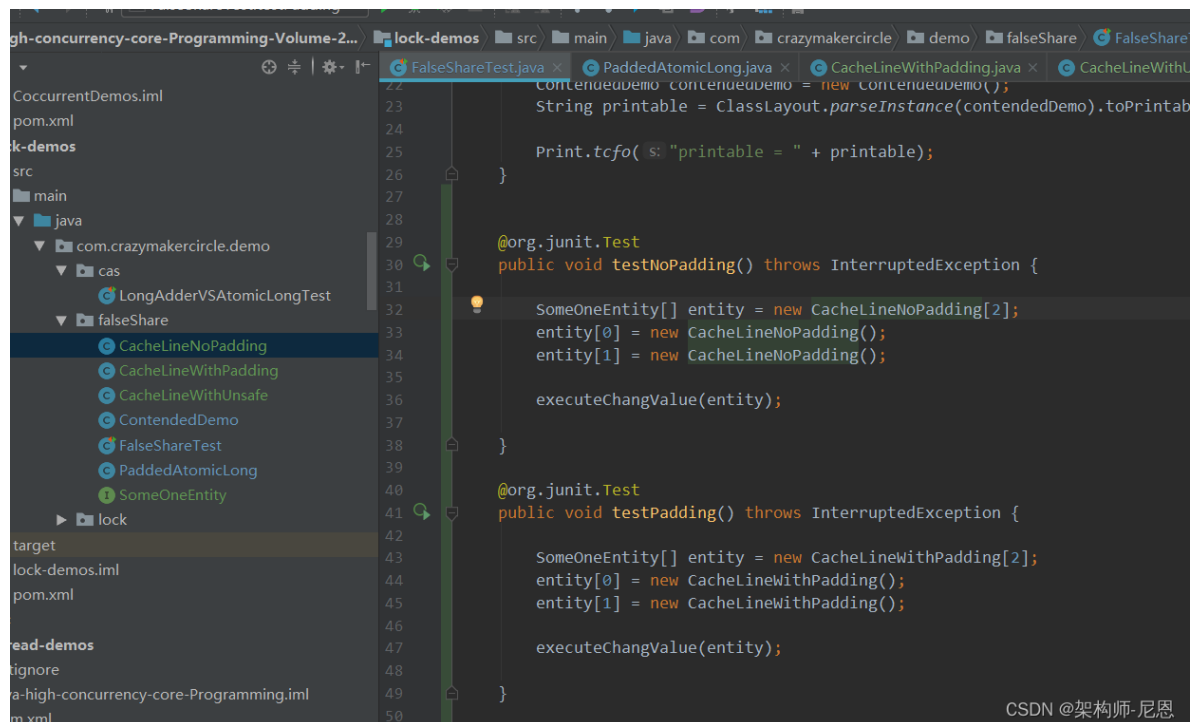
    public volatile long x = 1L;

    protected volatile long p9, p10, p11, p12, p13, p14, p15;

    @Override
    public void setValue(long value) {
        x = value;
    }
}
```

The bottom right corner of the IDE window contains the text: CSDN @架构师-尼恩

测试用例



The screenshot shows the same IDE with the `FalseShareTest` class selected in the file tree. The code in the editor is as follows:

```
ContendedDemo contendedDemo = new ContendedDemo();
String printable = ClassLayout.parseInstance(contendedDemo).toPrintable();

Print.tcfo("s: \"printable = \" + printable);
}

@org.junit.Test
public void testNoPadding() throws InterruptedException {
    SomeOneEntity[] entity = new CacheLineNoPadding[2];
    entity[0] = new CacheLineNoPadding();
    entity[1] = new CacheLineNoPadding();

    executeChangeValue(entity);
}

@org.junit.Test
public void testPadding() throws InterruptedException {
    SomeOneEntity[] entity = new CacheLineWithPadding[2];
    entity[0] = new CacheLineWithPadding();
    entity[1] = new CacheLineWithPadding();

    executeChangeValue(entity);
}
```

The bottom right corner of the IDE window contains the text: CSDN @架构师-尼恩

消除伪共享场景下的 耗时计算 （550ms）

```
Console →
>> Tests passed: 1 of 1 test – 591 ms

ms D:\dev\jdk\java-1.8.0-openjdk-1.8.0.312-2.b07.dev.redhat.windows.x86_64\bin\java.exe ...
ms Connected to the target VM, address: '127.0.0.1:53558', transport: 'socket'
Disconnected from the target VM, address: '127.0.0.1:53558', transport: 'socket'
耗时: 550毫秒

Process finished with exit code 0

CSDN @架构师-尼恩
```

使用unsafe访问变量的耗时计算

entity

```
lock-demos  src  main  java  com  crazymakercircle  demo  falseShare  FalseShareTest
FalseShareTest.java  PaddedAtomicLong.java  CacheLineWithPadding.java  SomeOneEntity.java  CacheLineWithUnsafe.java
import sun.misc.Unsafe;

public class CacheLineWithUnsafe implements SomeOneEntity {

    //模拟 disruptor 的 Sequence
    protected volatile long p1, p2, p3, p4, p5, p6, p7;

    public volatile long x = 1L;

    protected volatile long p9, p10, p11, p12, p13, p14, p15;

    private static final Unsafe UNSAFE;
    private static final long VALUE_OFFSET;

    static {
        UNSAFE = Util.getUnsafe();
        try {
            VALUE_OFFSET = UNSAFE.objectFieldOffset(CacheLineWithUnsafe.class.getDeclaredField( name: "x"));
        } catch (final Exception e) {
            throw new RuntimeException(e);
        }
    }

    @Override
    public void setValue(long value) {
        // UNSAFE.putLongVolatile(this, VALUE_OFFSET, value);
        UNSAFE.putOrderedLong( o: this, VALUE_OFFSET, value);
    }

CacheLineWithUnsafe  setValue()

CSDN @架构师-尼恩
```

```
java  com.crazymakercircle.demo  falseShare  CacheLineWithUnsafe
LongAdderVSAtomicLongTest  CacheLineNoPadding  CacheLineWithPadding  CacheLineWithUnsafe  ContendedDemo  FalseShareTest  PaddedAtomicLong
FalseShareTest.testPaddingAndUnsafe()
@org.junit.Test
public void testPaddingAndUnsafe() throws InterruptedException {
    SomeOneEntity[] entity = new CacheLineWithUnsafe[2];
    entity[0] = new CacheLineWithUnsafe();
    entity[1] = new CacheLineWithUnsafe();

    executeChangValue(entity);
}

FalseShareTest  testPaddingAndUnsafe()

bugger  Console →
>> Tests passed: 1 of 1 test – 96 ms

False$ 96 ms D:\dev\jdk\java-1.8.0-openjdk-1.8.0.312-2.b07.dev.redhat.windows.x86_64\bin\java.exe ...
test: 96 ms Connected to the target VM, address: '127.0.0.1:53646', transport: 'socket'
Disconnected from the target VM, address: '127.0.0.1:53646', transport: 'socket'
耗时: 54毫秒

Process finished with exit code 0

CSDN @架构师-尼恩
```

使用unsafe访问变量的耗时计算:

54ms

性能总结

消除伪共享场景，比存在伪共享场景 的性能，性能提升 6倍左右

实验数据，从 3000ms 提升 到 500ms

使用 unsafe 取消内存可见性，比消除伪共享场景，性能提升 10 倍左右

实验数据，从 500ms 提升 到 50ms

通过实验的对比，可见Java 的性能，是可以大大优化的，尤其在高性能组件

以上实操的 详细介绍，请参见《100wqps 日志平台实操》

JDK源码中如何解决 伪共享问题

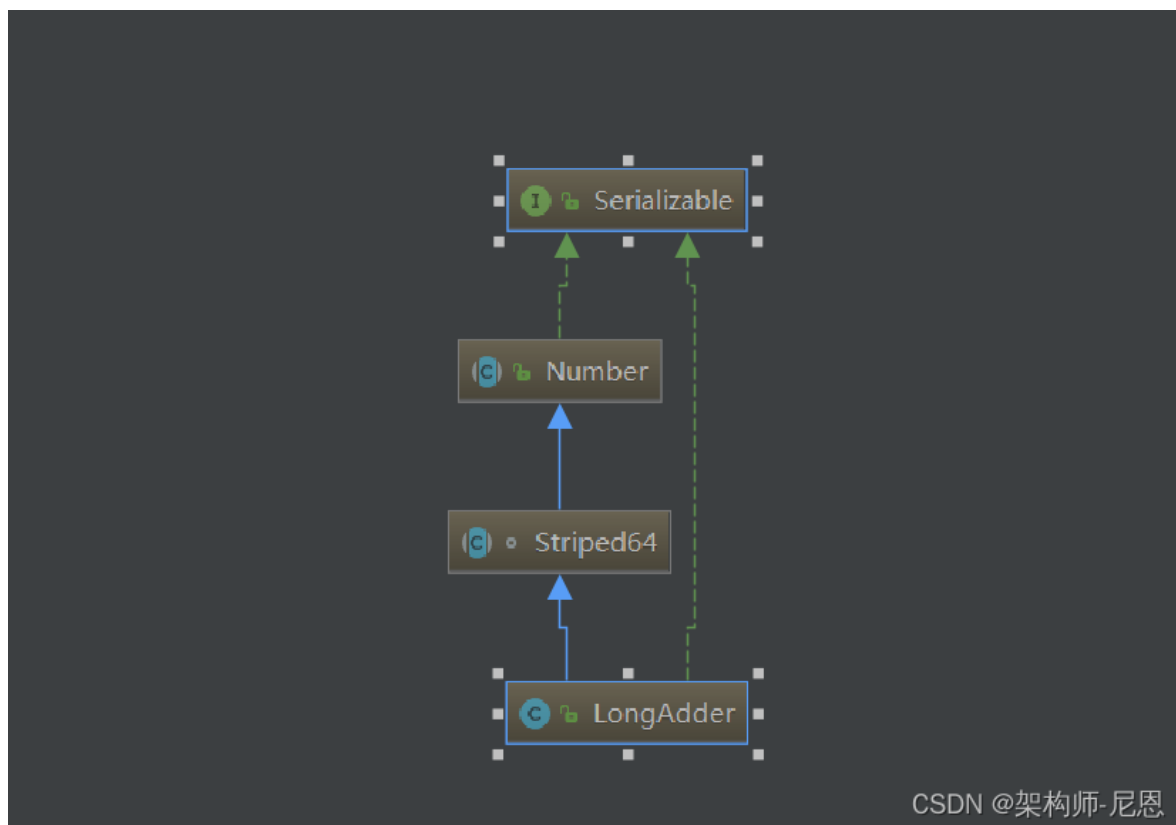
在LongAdder在java8中的实现已经采用了@Contended。

LongAdder以及 Striped64如何解决伪共享问题

LongAdder是大家常用的 高并发累加器

通过分而治之的思想，实现 超高并发累加。

LongAdder的 结构如下：



CSDN @架构师-尼恩

Striped64是在java8中添加用来支持累加器的并发组件，它可以在并发环境下使用来做某种计数，

Striped64的设计思路是在竞争激烈的时候尽量分散竞争，

Striped64维护了一个base Count和一个Cell数组，计数线程会首先试图更新base变量，如果成功则退出计数，否则会认为当前竞争是很激烈的，那么就会通过Cell数组来分散计数，

Striped64根据线程来计算哈希，然后将不同的线程分散到不同的Cell数组的index上，然后这个线程的计数内容就会保存在该Cell的位置上面，

基于这种设计，最后的总计数需要结合base以及散落在Cell数组中的计数内容。

这种设计思路类似于java7的ConcurrentHashMap实现，也就是所谓的分段锁算法，

ConcurrentHashMap会将记录根据key的hashCode来分散到不同的segment上，

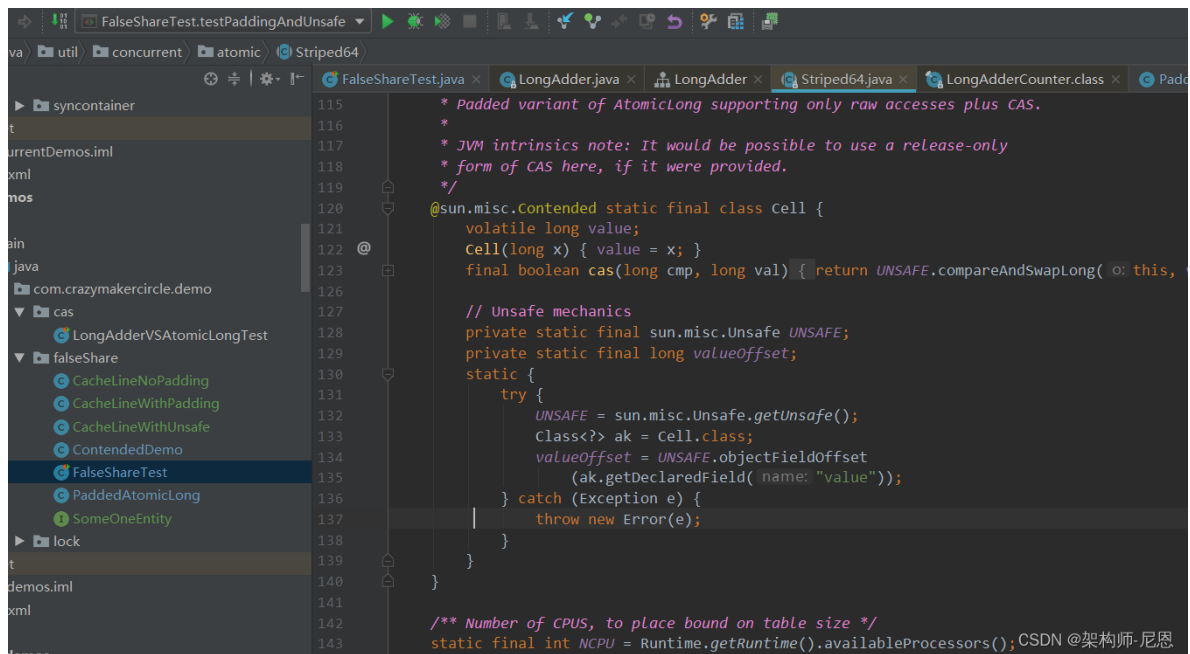
线程想要操作某个记录，只需要锁住这个记录对应的segment就可以了，而其他segment并不会被锁住，其他线程任然可以去操作其他的segment，

这样就显著提高了并发度，

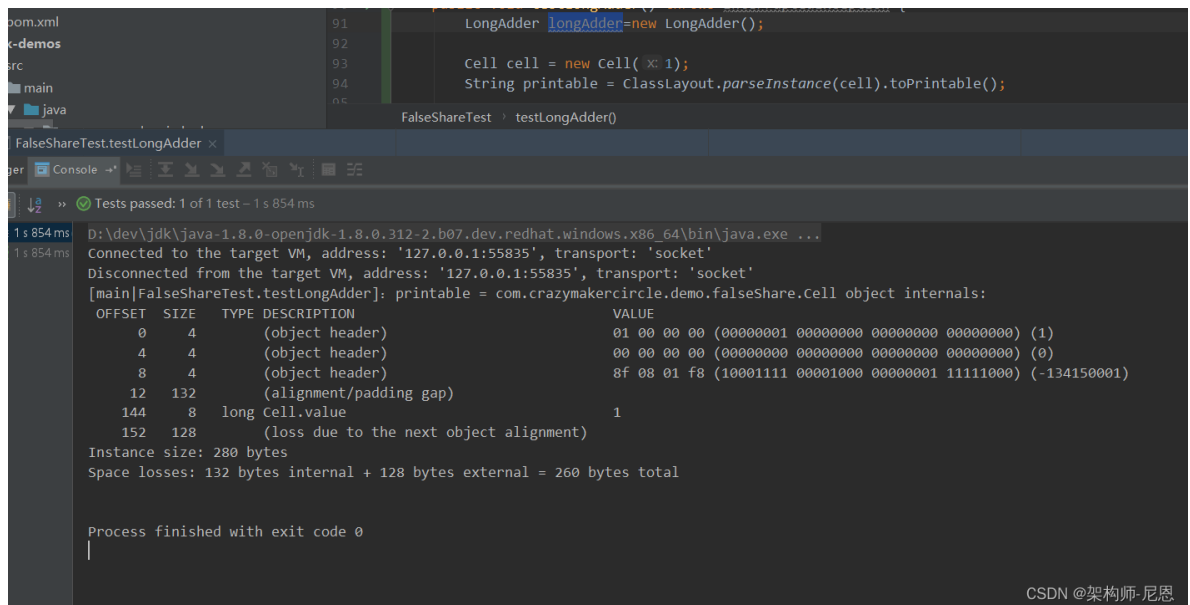
虽然如此，java8中的ConcurrentHashMap实现已经抛弃了java7中分段锁的设计，而采用更为轻量级的CAS来协调并发，效率更佳。

Cell元素如何消除伪共享

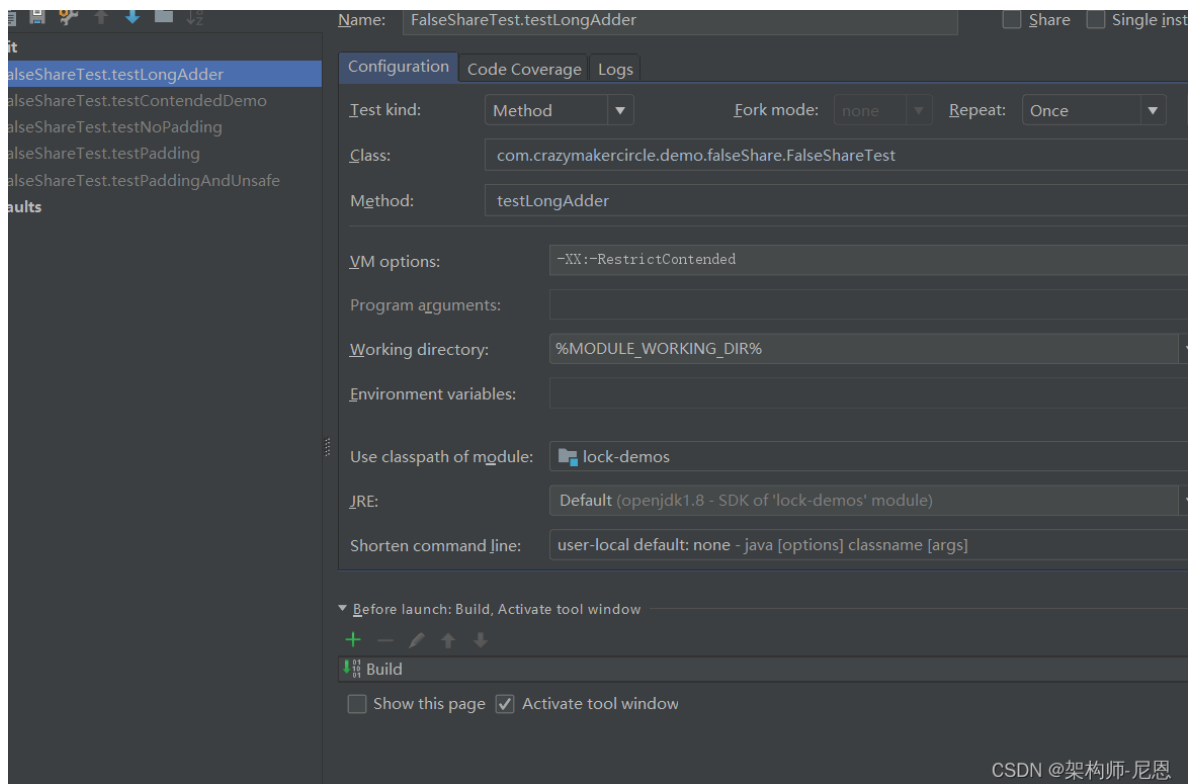
Striped64 中的Cell元素，是如何消除伪共享的呢？



可以打印一下 cell 的 内存结构



当然，别忘记加上 vm 选项：-XX:-RestrictContended



对于伪共享，我们在实际开发中该怎么做？

通过上面大篇幅的介绍，我们已经知道伪共享的对程序的影响。

那么，在实际的生产开发过程中，我们一定要通过缓存行填充去解决掉潜在的伪共享问题吗？

其实并不一定。

首先就是多次强调的，伪共享是很隐蔽的，我们暂时无法从系统层面上通过工具来探测伪共享事件。

其次，不同类型的计算机具有不同的微架构（如 32 位系统和 64 位系统的 java 对象所占自己数就不一样），如果设计到跨平台的设计，那就更难以把握了，一个确切的填充方案只适用于一个特定的操作系统。

还有，缓存的资源是有限的，如果填充会浪费珍贵的 cache 资源，并不适合大范围应用。

3 Disruptor框架是如何解决伪共享问题的？

在Disruptor中有一个重要的类Sequence，该类包装了一个volatile修饰的long类型数据value，

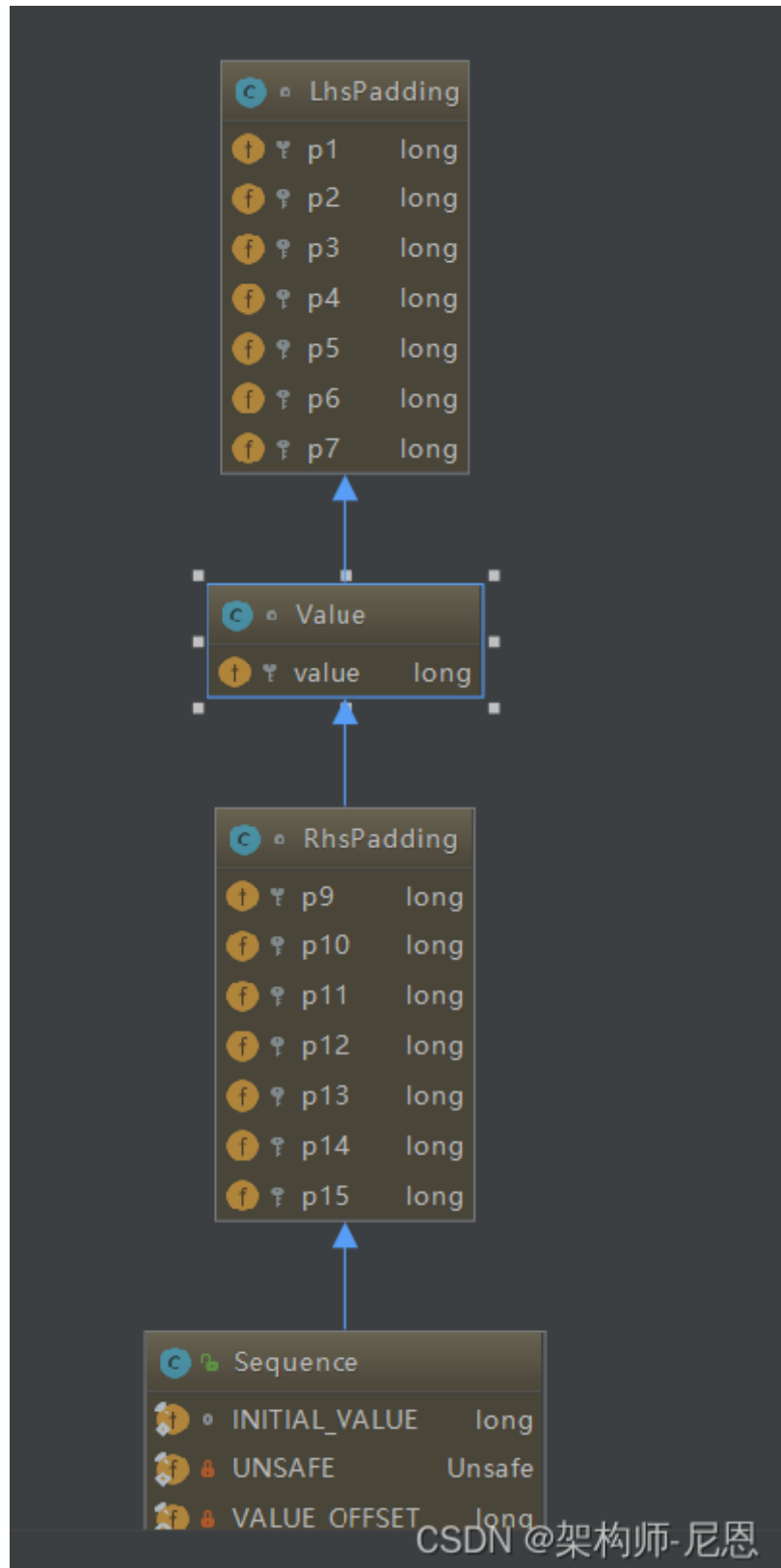
Sequence的结构和源码

无论是Disruptor中的基于数组实现的缓冲区RingBuffer，还是生产者，消费者，都有各自独立的Sequence，

Sequence的用途是啥呢？

- 在RingBuffer缓冲区中，Sequence标示着写入进度，例如每次生产者要写入数据进缓冲区时，都要调用RingBuffer.next () 来获得下一个可使用的相对位置。
- 对于生产者和消费者来说，Sequence标示着它们的事件序号。

Sequence的结构图如下



来看看Sequence类的源码：


```

class LhsPadding {
    protected long p1, p2, p3, p4, p5, p6, p7;
}

class Value extends LhsPadding {
    protected volatile long value;
}

class RhsPadding extends Value {
    protected long p9, p10, p11, p12, p13, p14, p15;
}

public class Sequence extends RhsPadding {
    static final long INITIAL_VALUE = -1L;
    private static final Unsafe UNSAFE;
    private static final long VALUE_OFFSET;
    static {
        UNSAFE = Util.getUnsafe();
        try {
            VALUE_OFFSET =
UNSAFE.objectFieldOffset(Value.class.getDeclaredField("value"));
        } catch (final Exception e) {
            throw new RuntimeException(e);
        }
    }

    public Sequence() {
        this(INITIAL_VALUE);
    }

    public Sequence(final long initialValue) {
        UNSAFE.putOrderedLong(this, VALUE_OFFSET, initialValue);
    }
}

```

2: Disruptor 的使用实战

我们从一个简单的例子开始学习Disruptor：

生产者传递一个long类型的值给消费者，而消费者消费这个数据的方式仅仅是把它打印出来。

定义一个Event和工厂

首先定义一个Event来包含需要传递的数据：

```
public class LongEvent {
    private long value;
    public long getValue() {
        return value;
    }

    public void setValue(long value) {
        this.value = value;
    }
}
```

由于需要让Disruptor为我们创建事件，我们同时还声明了一个EventFactory来创建Event对象。

```
public class LongEventFactory implements EventFactory {
    @Override
    public Object newInstance() {
        return new LongEvent();
    }
}
```

定义事件处理器（消费者）

我们还需要一个事件消费者，也就是一个事件处理器。

这个例子中，事件处理器的工作，就是简单地把事件中存储的数据打印到终端：

```
/**
 * 类似于消费者
 * disruptor会回调此处理器的方法
 */
static class LongEventHandler implements EventHandler<LongEvent> {
    @Override
    public void onEvent(LongEvent longEvent, long l, boolean b) throws
Exception {
        System.out.println(longEvent.getValue());
    }
}
```

disruptor会回调此处理器的方法

定义事件源(生产者)

事件都会有一个生成事件的源，类似于 生产者的角色，

如何产生事件，然后发出事件呢？

通过从 环形队列中 获取 序号，通过序号获取 对应的 事件对象，将数据填充到 事件对象，再通过 序号将 事件对象 发布出去。

一段生产者的代码如下：

```
// 事件生产者：业务代码
// 通过从 环形队列中 获取 序号，通过序号获取 对应的 事件对象，将数据填充到 事件对象，再
通过 序号将 事件对象 发布出去。
```

```

static class LongEventProducer {
    private final RingBuffer<LongEvent> ringBuffer;

    public LongEventProducer(RingBuffer<LongEvent> ringBuffer) {
        this.ringBuffer = ringBuffer;
    }

    /**
     * onData用来发布事件，每调用一次就发布一次事件事件
     * 它的参数会通过事件传递给消费者
     *
     * @param data
     */
    public void onData(long data) {

        // step1: 通过从 环形队列中 获取 序号
        //可以把ringBuffer看做一个事件队列，那么next就是得到下面一个事件槽
        long sequence = ringBuffer.next();

        try {

            //step2: 通过序号获取 对应的 事件对象， 将数据填充到 事件对象，
            //用上面的索引，取出一个空的事件用于填充
            LongEvent event = ringBuffer.get(sequence); // for the sequence
            event.setValue(data);
        } finally {

            //step3: 再通过 序号将 事件对象 发布出去。
            //发布事件
            ringBuffer.publish(sequence);
        }
    }
}

```

很明显的是：

当用一个简单队列来发布事件的时候会牵涉更多的细节，这是因为事件对象还需要预先创建。

发布事件最少需要三步：

step1: 获取下一个事件槽。

如果我们使用RingBuffer.next()获取一个事件槽，那么一定要发布对应的事件。

step2: 通过序号获取 对应的 事件对象， 将数据填充到 事件对象，

step3: 再通过 序号将 事件对象 发布出去。

发布事件的时候要使用try/finally保证事件一定会被发布

如果不能发布事件，那么就会引起Disruptor状态的混乱。

尤其是在多个事件生产者的情况下会导致事件消费者失速，从而不得不重启应用才能恢复。

Disruptor 3.0提供了lambda式的API。

这样可以把一些复杂的操作放在Ring Buffer，所以在Disruptor3.0以后的版本最好使用Event Publisher或者Event Translator(事件转换器)来发布事件。

组装起来

最后一步就是把所有的代码组合起来完成一个完整的事件处理系统。

```
@org.junit.Test
public void testSimpleDisruptor() throws InterruptedException {
    // 消费者线程池
    Executor executor = Executors.newCachedThreadPool();
    // 事件工厂
    LongEventFactory eventFactory = new LongEventFactory();
    // 环形队列大小，2的指数
    int bufferSize = 1024;

    // 构造 分裂者（事件分发者）
    Disruptor<LongEvent> disruptor = new Disruptor<LongEvent>(eventFactory,
        bufferSize, executor);

    // 连接 消费者 处理器
    disruptor.handleEventsWith(new LongEventHandler());
    // 开启 分裂者（事件分发）
    disruptor.start();

    // 获取环形队列，用于生产 事件
    RingBuffer<LongEvent> ringBuffer = disruptor.getRingBuffer();

    LongEventProducer producer = new LongEventProducer(ringBuffer);

    for (long i = 0; true; i++) {
        //发布事件
        producer.onData(i);
        Thread.sleep(1000);
    }
}
```

事件转换器

Disruptor3.0以后，提供了事件转换器，帮助填充 LongEvent 的业务数据

下面是一个例子

```
static class LongEventProducerWithTranslator {
    //一个translator可以看做一个事件初始化器，publicEvent方法会调用它
    //填充Event
    private static final EventTranslatorOneArg<LongEvent, Long> TRANSLATOR =
        new EventTranslatorOneArg<LongEvent, Long>() {
            public void translateTo(LongEvent event, long sequence, Long
data) {
                event.setValue(data);
            }
        };

    private final RingBuffer<LongEvent> ringBuffer;

    public LongEventProducerWithTranslator(RingBuffer<LongEvent> ringBuffer)
{
}
```

```

        this.ringBuffer = ringBuffer;
    }

    public void onData(Long data) {
        ringBuffer.publishEvent(TRANSLATOR, data);
    }
}

```

使用事件转换器的好处，省了从 环形队列 获取 序号，然后拿到事件 填充数据，再发布序号 中的第二步骤

给 事件 填充 数据 的动作，在 EventTranslatorOneArg 完成

Disruptor提供了不同的接口去产生一个Translator对象：

- EventTranslator,
- EventTranslatorOneArg,
- EventTranslatorTwoArg,

很明显，Translator中方法的参数是通过RingBuffer来传递的。

使用 事件转换器 转换器的进行事件的 生产与消费 代码，大致如下：

```

@org.junit.Test
public void testSimpleDisruptorWithTranslator() throws InterruptedException
{
    // 消费者线程池
    Executor executor = Executors.newCachedThreadPool();
    // 事件工厂
    LongEventFactory eventFactory = new LongEventFactory();
    // 环形队列大小，2的指数
    int bufferSize = 1024;

    // 构造 分裂者 （事件分发者）
    Disruptor<LongEvent> disruptor = new Disruptor<LongEvent>(eventFactory,
bufferSize, executor);

    // 连接 消费者 处理器
    disruptor.handleEventsWith(new LongEventHandler());
    // 开启 分裂者（事件分发）
    disruptor.start();

    // 获取环形队列，用于生产 事件
    RingBuffer<LongEvent> ringBuffer = disruptor.getRingBuffer();

    LongEventProducerWithTranslator producer = new
LongEventProducerWithTranslator(ringBuffer);

    for (long i = 0; true; i++) {
        //发布事件
        producer.onData(i);
        Thread.sleep(1000);
    }
}

```

上面写法的另一个好处是，Translator可以分离出来并且更加容易单元测试。

通过Java 8 Lambda使用Disruptor

Disruptor在自己的接口里面添加对于Java 8 Lambda的支持。

大部分Disruptor中的接口都符合Functional Interface的要求（也就是在接口中仅仅有一个方法）。

所以在Disruptor中，可以广泛使用Lambda来代替自定义类。

```
@org.junit.Test
public void testSimpleDisruptorWithLambda() throws InterruptedException {
    // 消费者线程池
    Executor executor = Executors.newCachedThreadPool();
    // 环形队列大小，2的指数
    int bufferSize = 1024;

    // 构造 分裂者（事件分发者）
    Disruptor<LongEvent> disruptor = new Disruptor<LongEvent>
(LongEvent::new, bufferSize, executor);

    // 连接 消费者 处理器
    // 可以使用lambda来注册一个EventHandler
    disruptor.handleEventsWith((event, sequence, endOfBatch) ->
System.out.println("Event: " + event.getValue()));
    // 开启 分裂者（事件分发）
    disruptor.start();

    // 获取环形队列，用于生产 事件
    RingBuffer<LongEvent> ringBuffer = disruptor.getRingBuffer();

    LongEventProducerWithTranslator producer = new
LongEventProducerWithTranslator(ringBuffer);

    for (long i = 0; true; i++) {
        //发布事件
        producer.onData(i);
        Thread.sleep(1000);
    }
}
```

由于在Java 8中方法引用也是一个lambda，因此还可以把上面的代码改成下面的代码：

```
public static void handleEvent(LongEvent event, long sequence, boolean
endOfBatch)
{
    System.out.println(event.getValue());
}

@org.junit.Test
public void testSimpleDisruptorWithMethodRef() throws InterruptedException {
    // 消费者线程池
    Executor executor = Executors.newCachedThreadPool();
    // 环形队列大小，2的指数
    int bufferSize = 1024;
```

```

// 构造 分裂者 (事件分发者)
Disruptor<LongEvent> disruptor = new Disruptor<LongEvent>
(LongEvent::new, bufferSize, executor);

// 连接 消费者 处理器
// 可以使用lambda来注册一个EventHandler
disruptor.handleEventsWith(LongEventDemo::handleEvent);
// 开启 分裂者 (事件分发)
disruptor.start();

// 获取环形队列, 用于生产 事件
RingBuffer<LongEvent> ringBuffer = disruptor.getRingBuffer();

LongEventProducerWithTranslator producer = new
LongEventProducerWithTranslator(ringBuffer);

for (long i = 0; true; i++) {
    //发布事件
    producer.onData(i);
    Thread.sleep(1000);
}
}
}

```

构造Disruptor对象的几个要点

在构造Disruptor对象, 有几个核心的要点:

- 1: 事件工厂(Event Factory)定义了如何实例化事件(Event), Disruptor 通过 EventFactory 在 RingBuffer 中预创建 Event 的实例。
- 2: ringBuffer这个数组的大小, 一般根据业务指定成2的指数倍。
- 3: 消费者线程池, 事件的处理是在构造的线程池里来进行处理的。
- 4: 指定等待策略, Disruptor 定义了 com.lmax.disruptor.WaitStrategy 接口用于抽象 **Consumer 如何等待Event事件**。

Disruptor 提供了多个 WaitStrategy 的实现, 每种策略都具有不同性能和优缺点, 根据实际运行环境的 CPU 的硬件特点选择恰当的策略, 并配合特定的 JVM 的配置参数, 能够实现不同的性能提升。

- BlockingWaitStrategy 是最低效的策略, 但其对**CPU的消耗最小**并且在各种不同部署环境中能提供更加一致的性能表现;
- SleepingWaitStrategy 的性能表现跟 BlockingWaitStrategy 差不多, 对 CPU 的消耗也类似, 但其对生产者线程的影响最小, 适合用于异步日志类似的场景;
- YieldingWaitStrategy 的性能是最好的, 适合用于低延迟的系统。在要求极高性能且**事件处理线数小于 CPU 逻辑核心数的场景中**, 推荐使用此策略; 。

Disruptor如何实现高性能?

使用Disruptor, 主要用于对性能要求高、延迟低的场景, 它通过“榨干”机器的性能来换取处理的高性能。

Disruptor实现高性能主要体现了去掉了锁，采用CAS算法，同时内部通过环形队列实现有界队列。

- 环形数据结构
数组元素不会被回收，避免频繁的GC，所以，为了避免垃圾回收，采用数组而非链表。
同时，数组对处理器的缓存机制更加友好。
- 元素位置定位
数组长度 2^n ，通过位运算，加快定位的速度。
下标采取递增的形式。不用担心index溢出的问题。
index是long类型，即使100万QPS的处理速度，也需要30万年才能用完。
- 无锁设计
采用CAS无锁方式，保证线程的安全性
每个生产者或者消费者线程，会先申请可以操作的元素在数组中的位置，申请到之后，直接在该位置写入或者读取数据。整个过程通过原子变量CAS，保证操作的线程安全。
- 属性填充：
通过添加额外的无用信息，避免伪共享问题

Disruptor和BlockingQueue比较:

- **BlockingQueue:** FIFO队列.生产者Producer向队列中发布publish一个事件时,消费者Consumer能够获取到通知.如果队列中没有消费的事件,消费者就会被阻塞,直到生产者发布新的事件
- Disruptor可以比BlockingQueue做到更多:
 - Disruptor队列中同一个事件可以有多个消费者,消费者之间既可以并行处理,也可以形成依赖图相互依赖,按照先后次序进行处理
 - Disruptor可以预分配用于存储事件内容的内存空间
 - Disruptor使用极度优化和无锁的设计实现极高性能的目标

如果你的项目有对性能要求高，对延迟要求低的需求，并且需要一个无锁的有界队列，来实现生产者/消费者模式，那么Disruptor是你的不二选择。

原理：Disruptor 的内部Ring Buffer环形队列

RingBuffer是什么

RingBuffer 是一个环(首尾相连的环)，用做在不同上下文(线程)间传递数据的buffer。

RingBuffer 拥有一个序号，这个序号指向数组中下一个可用元素。



Disruptor使用环形队列的优势：

Disruptor框架就是一个使用CAS操作的内存队列，与普通的队列不同，

Disruptor框架使用的是一个基于数组实现的环形队列，无论是生产者向缓冲区里提交任务，还是消费者从缓冲区里获取任务执行，都使用CAS操作。

使用环形队列的优势：

第一，简化了多线程同步的复杂度。

学数据结构的时候，实现队列都要两个指针head和tail来分别指向队列的头和尾，对于一般的队列是这样，

想象下，如果有多个生产者同时往缓冲区队列中提交任务，某一生产者提交新任务后，tail指针都要做修改的，那么多个生产者提交任务，头指针不会做修改，但会对tail指针产生冲突，

例如某一生产者P1要做写入操作，在获得tail指针指向的对象值V后，执行compareAndSet () 方法前，tail指针被另一生产者P2修改了，这时生产者P1执行compareAndSet () 方法，发现tail指针指向的值V和期望值E不同，导致冲突。

同样，如果多个消费者不断从缓冲区中获取任务，不会修改尾指针，但会造成队列头指针head的冲突问题（因为队列的FIFO特点，出列会从头指针出开始）。

环形队列的一个特点就是只有一个指针，只通过一个指针来实现出列和入列操作。

如果使用两个指针head和tail来管理这个队列，有可能会出现“伪共享”问题（伪共享问题在下面我会详细说），

因为创建队列时，head和tail指针变量常常在同一个缓存行中，多线程修改同一缓存行中的变量就容易出现伪共享问题。

第二，由于使用的是环形队列，那么队列创建时大小就被固定了，

Disruptor框架中的环形队列本来也就是基于数组实现的，使用数组的话，减少了系统对内存空间管理的压力，

因为数组不像链表，Java会定期回收链表中一些不再引用的对象，而数组不会出现空间的新分配和回收问题。

关闭Disruptor

- **disruptor.shutdown()** : 关闭Disruptor. 方法会阻塞,直至所有的事件都得到处理
- **executor.shutdown()** : 关闭Disruptor使用的线程池. 如果线程池需要关闭,必须进行手动关闭,Disruptor在shutdown时不会自动关闭使用的线程池

3: Disruptor 的使用场景分析

Disruptor 它可以用来作为高性能的有界内存队列，适用于两大场景：

- 生产者消费者场景
- 发布订阅 场景

生产者消费者场景。Disruptor的最常用的场景就是“生产者-消费者”场景，对场景的就是“一个生产者、多个消费者”的场景，并且要求顺序处理。

备注，这里和JCTool 的 MPSC 队列，刚好相反，MPSC 使用于多生产者，单消费者场景

发布订阅 场景：Disruptor也可以认为是观察者模式的一种实现，实现发布订阅模式。

当前业界开源组件使用Disruptor的包括Log4j2、Apache Storm等，

Disruptor 使用细分场景

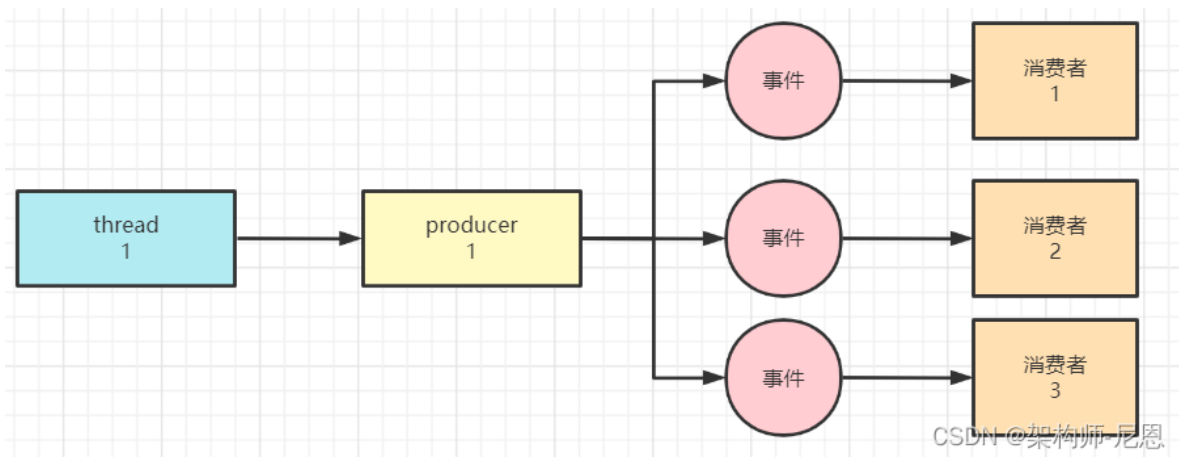
Disruptor是一个优秀的并发框架，可以使用在多个生产者单消费者场景

- 单生产者多消费者场景
- 多生产者单消费者场景
- 单生产者多消费者场景
- 多个消费者串行消费场景
- 菱形方式执行场景
- 链式并行执行场景
- 多组消费者相互隔离场景
- 多组消费者航道执行模式

单生产者多消费者并行场景

在并发系统中提高性能最好的方式之一就是单一写者原则，对Disruptor也是适用的。

如果在生产者单消费者 需求中仅仅有一个事件生产者，那么可以设置为单一生产者模式来提高系统的性能。



ProducerType 的类型

ProducerType 定义了生产者的类型，两类

```
1  package com.lmax.disruptor.dsl;
16
17
18  /**
19   * Defines producer types to support creation of RingBuffer with correct sequencer and publisher.
20   */
21  public enum ProducerType
22  {
23      /**
24       * Create a RingBuffer with a single event publisher to the RingBuffer
25       */
26      SINGLE,
27
28      /**
29       * Create a RingBuffer supporting multiple event publishers to the one RingBuffer
30       */
31      MULTI
32  }
33
```

在这种场景下，ProducerType 的类型的 SINGLE

单生产者多消费者并行场景的参考代码

参考的代码如下：

```
LongEventProducerWithTranslator.java x LongEventSceneDemo.java x Disruptor.java x WorkHandler.java x LongEventHandler.java x

@org.junit.Test
public void testSimpleProducerDisruptorWithMethodRef() throws InterruptedException {
    // 消费者线程池
    Executor executor = Executors.newCachedThreadPool();
    // 环形队列大小, 2的指数
    int bufferSize = 1024;
    // 构造 分裂者 (事件分发者)
    Disruptor<LongEvent> disruptor = new Disruptor<LongEvent>(LongEvent::new, bufferSize,
        executor,
        ProducerType.SINGLE, //多个生产者
        new YieldingWaitStrategy());
    // 连接 消费者 处理器
    // 可以使用lambda来注册一个EventHandler
    disruptor.handleEventsWith(LongEventSceneDemo::handleEvent,
        LongEventSceneDemo::handleEvent);
    // 开启 分裂者 (事件分发)
    disruptor.start();
    // 获取环形队列, 用于生产 事件
    RingBuffer<LongEvent> ringBuffer = disruptor.getRingBuffer();

    //1生产者, 并发生产数据
    LongEventProducerWithTranslator producer = new LongEventProducerWithTranslator(ringBuffer);
    Thread thread = new Thread() { @Override public void run() {
        for (long i = 0; true; i++) { /*发布事件*/ producer.onData(i); ThreadUtil.sleepSeconds(1); } }};
    thread.start();
    ThreadUtil.sleepSeconds(5);
}
```

CSDN @架构师-尼恩

执行结果:

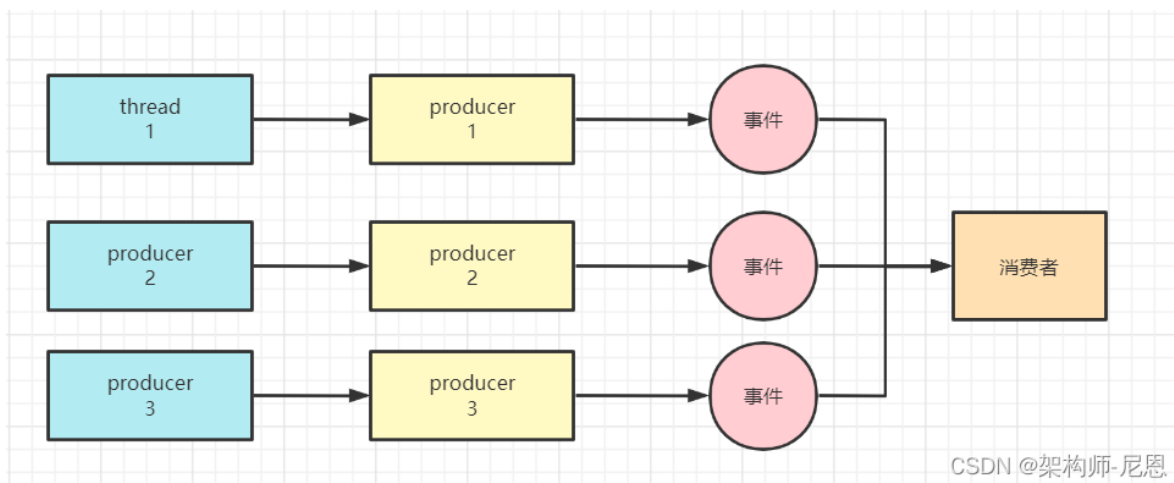
```
Tests passed: 1 of 1 test - 5 s 59 ms
D:\dev\jdk\java-1.8.0-openjdk-1.8.0.312-2.b07.dev.redhat.windows.x86_64\bin\java.exe ...
Connected to the target VM, address: '127.0.0.1:56848', transport: 'socket'
0
0
0
1
1
1
2
2
2
3
3
3
4
4
4
Disconnected from the target VM, address: '127.0.0.1:56848', transport: 'socket'
Process finished with exit code 0

CSDN @架构师-尼恩
```

以上用例的具体减少, 请参见 尼恩《100wqps 日志平台实操, 视频》

多生产者单消费者场景

该场景较为简单, 就是多个生产者, 单个消费者



其实，消费者也可以是多

ProducerType 的类型

ProducerType 定义了生产者的类型，两类

```
1  package com.lmax.disruptor.dsl;
2
3  /**
4   * Defines producer types to support creation of RingBuffer with correct sequencer and publisher.
5   */
6  public enum ProducerType
7  {
8      /**
9       * Create a RingBuffer with a single event publisher to the RingBuffer
10      */
11      SINGLE,
12
13      /**
14       * Create a RingBuffer supporting multiple event publishers to the one RingBuffer
15      */
16      MULTI
17  }
```

CSDN @架构师-尼恩

在这种场景下，ProducerType 的类型的 MULTI

多生产者场景的要点

在代码编写维度，多生产者单消费者场景的要点如下：

- 创建Disruptor 的时候，将ProducerType.SINGLE改为ProducerType.MULTI，
- 编写多线程生产者的相关代码即可。

多生产者场景的参考代码

参考的代码如下：

```
Volume-2... | CoccurentDemos | src | main | java | com | crazymakercircle | disruptor | LongEventDemo
LongEventDemo.java | CopyOnWriteArrayListTest.java | NotSafePlus.java | LongEventFactory.java | ContendedDemo.java

8  import java.util.concurrent.Executors;
9
10 public class LongEventDemo {
11
12
13     @org.junit.Test
14     public void testSimpleDisruptor() throws InterruptedException {
15         // 消费者线程池
16         Executor executor = Executors.newCachedThreadPool();
17         // 事件工厂
18         LongEventFactory eventFactory = new LongEventFactory();
19         // 环形队列大小, 2的指数
20         int bufferSize = 1024;
21
22         // 构造 分裂者 (事件分发者)
23         Disruptor<LongEvent> disruptor = new Disruptor<>(eventFactory, bufferSize, executor);
24
25         // 连接 消费者 处理器
26         disruptor.handleEventsWith(new LongEventHandler());
27         // 开启 分裂者 (事件分发)
28         disruptor.start();
29
30         // 获取环形队列, 用于生产 事件
31         RingBuffer<LongEvent> ringBuffer = disruptor.getRingBuffer();
32
33         LongEventProducer producer = new LongEventProducer(ringBuffer);
34
35         for (long i = 0; true; i++) {
36             //发布事件
37             producer.onData(i);
38             Thread.sleep( millis: 1000);
39         }
40     }
41 }
```

Translator

emo1
test

CSDN @架构师-尼恩

运行的结果如下

```
LongEventSceneDemo.testSimpleDisruptorWithMethodRef
CoccurentDemos | src | main | java | com | crazymakercircle | disruptor | LongEventDemo
LongEventDemo.java | CopyOnWriteArrayListTest.java | NotSafePlus.java | LongEventFactory.java | ContendedDemo.java

35     for (long i = 0; true; i++) {
36         //发布事件
37     }
38     LongEventDemo.testSimpleDisruptor()

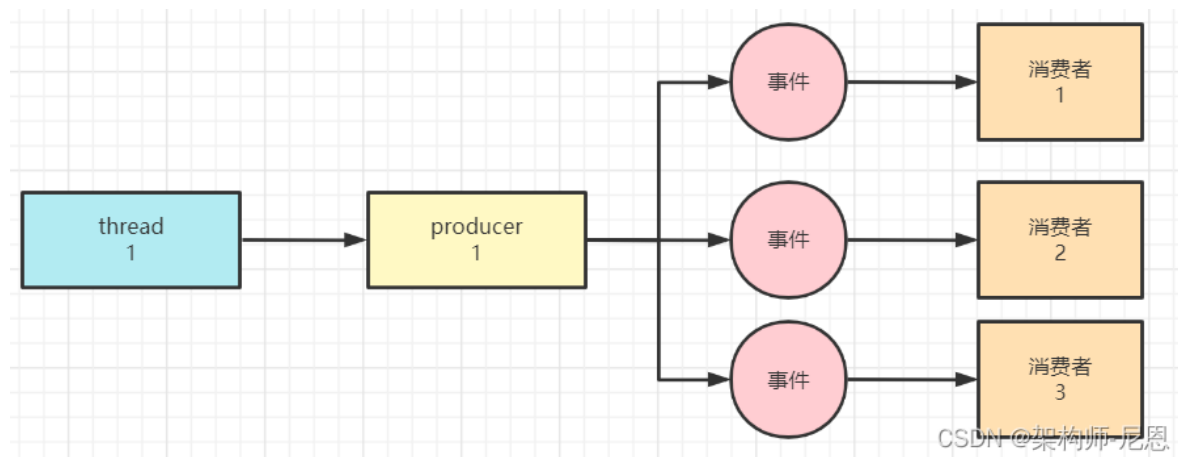
LongEventSceneDemo.testSimpleDisruptorWithMethodRef
Debugger | Console
Tests passed: 1 of 1 test - 5 s 230 ms
5 s 230 ms
D:\dev\jdk\java-1.8.0-openjdk-1.8.0.312-2.b07.dev.redhat.windows.x86_64\bin\java.exe ...
Connected to the target VM, address: '127.0.0.1:51918', transport: 'socket'
0
0
0
1
1
1
2
2
2
3
3
3
4
4
4
Disconnected from the target VM, address: '127.0.0.1:51918', transport: 'socket'
Process finished with exit code 0

CSDN @架构师-尼恩
```

以上用例的具体减少, 请参见 尼恩《100wqps 日志平台实操, 视频》

单生产者多消费者竞争场景

该场景中，生产者为一个，消费者为多个，多个消费者之间，存在着竞争关系，
也就是说，对于同一个事件event，多个消费者 不重复消费



disruptor如何设置多个竞争消费者？

首先，得了解一下，disruptor框架的两个设置消费者的方法
大概有两点：

- 消费者需要 实现 `WorkHandler` 接口，而不是 `EventHandler` 接口
- 使用 `handleEventsWithWorkerPool` 设置 disruptor的消费者，而不是 `handleEventsWith` 方法

在disruptor框架调用start方法之前，有两个方法设置消费者：

- `disruptor.handleEventsWith(EventHandler ... handlers)`，将多个EventHandler的实现类传入方法，封装成一个EventHandlerGroup，实现多消费者消费。
- `disruptor.handleEventsWithWorkerPool(WorkHandler ... handlers)`，将多个WorkHandler的实现类传入方法，封装成一个EventHandlerGroup实现多消费者消费。

那么，以上的Disruptor类的handleEventsWith，handleEventsWithWorkerPool方法的联系及区别是什么呢？

相同的在于：

两者共同点都是，将多个消费者封装到一起，供框架消费事件。

第一个不同点在于：

对于某一条事件 event，

`handleEventsWith` 方法返回的EventHandlerGroup，Group中的每个消费者都会对 event 进行消费，各个消费者之间不存在竞争。

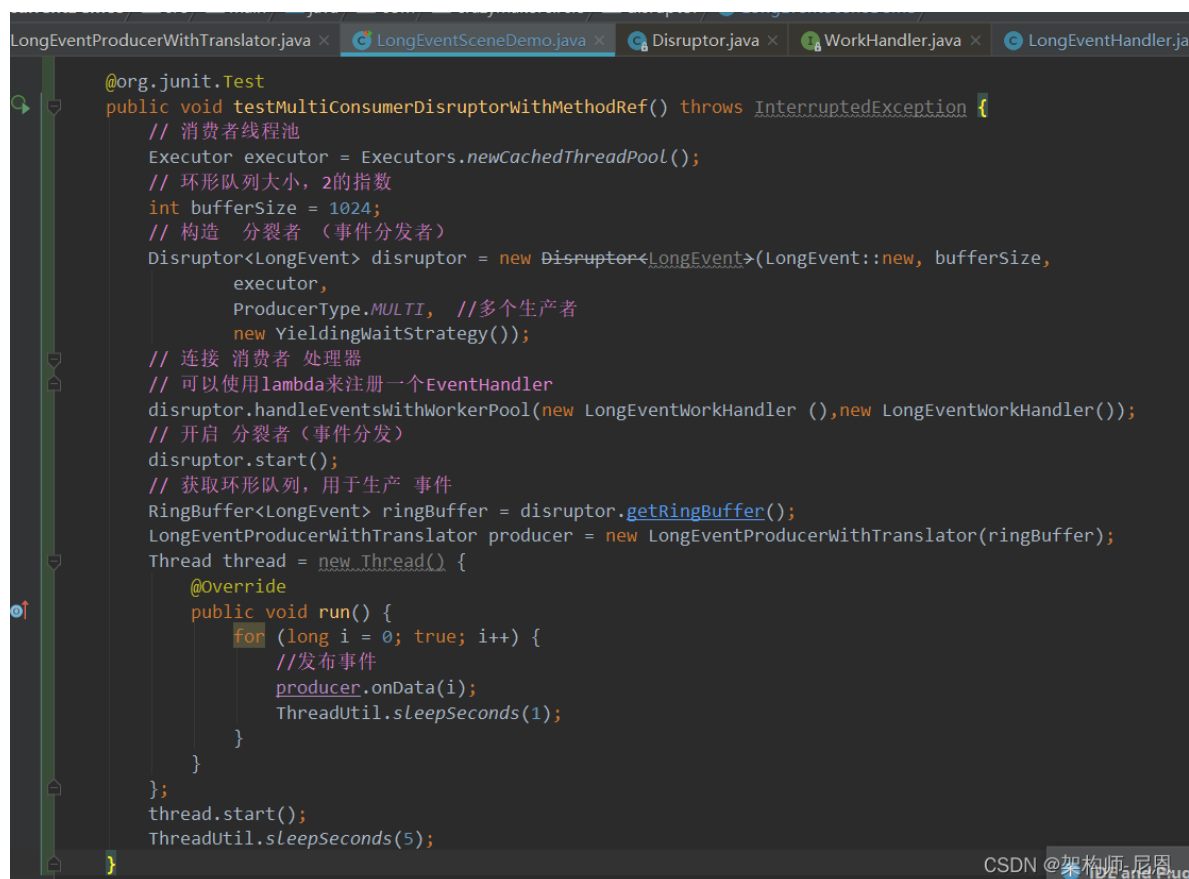
`handleEventsWithWorkerPool`方法返回的EventHandlerGroup，Group的消费者对于同一条事件 event 不重复消费；也就是，如果c0消费了事件m，则c1不再消费事件m。

另外一个不同:

在设置消费者的时候, Disruptor类的handleEventsWith, handleEventsWithWorkerPool方法所传入的形参不同。对于独立消费的消费, 应当实现EventHandler接口。对于不重复消费的消费, 应当实现WorkHandler接口。

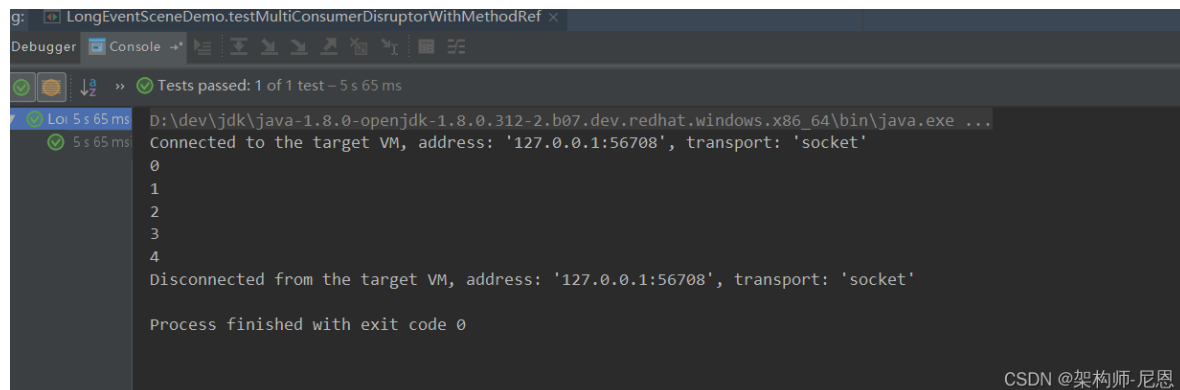
因此, 根据消费者集合是否独立消费事件, 可以对不同的接口进行实现。也可以对两种接口同时实现, 具体消费流程由disruptor的方法调用决定。

演示代码如下:



```
@org.junit.Test
public void testMultiConsumerDisruptorWithMethodRef() throws InterruptedException {
    // 消费者线程池
    Executor executor = Executors.newCachedThreadPool();
    // 环形队列大小, 2的指数
    int bufferSize = 1024;
    // 构造 分裂者 (事件分发者)
    Disruptor<LongEvent> disruptor = new Disruptor<LongEvent>(LongEvent::new, bufferSize,
        executor,
        ProducerType.MULTI, //多个生产者
        new YieldingWaitStrategy());
    // 连接 消费者 处理器
    // 可以使用lambda来注册一个EventHandler
    disruptor.handleEventsWithWorkerPool(new LongEventWorkHandler (), new LongEventWorkHandler());
    // 开启 分裂者 (事件分发)
    disruptor.start();
    // 获取环形队列, 用于生产 事件
    RingBuffer<LongEvent> ringBuffer = disruptor.getRingBuffer();
    LongEventProducerWithTranslator producer = new LongEventProducerWithTranslator(ringBuffer);
    Thread thread = new Thread() {
        @Override
        public void run() {
            for (long i = 0; true; i++) {
                //发布事件
                producer.onData(i);
                ThreadUtil.sleepSeconds(1);
            }
        }
    };
    thread.start();
    ThreadUtil.sleepSeconds(5);
}
```

执行结果



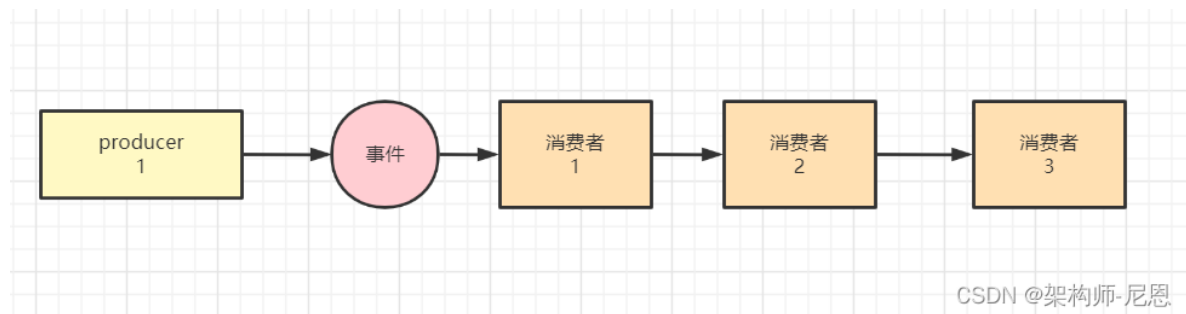
```
g: LongEventSceneDemo.testMultiConsumerDisruptorWithMethodRef x
Debugger Console
Tests passed: 1 of 1 test - 5 s 65 ms
5 s 65 ms D:\dev\jdk\java-1.8.0-openjdk-1.8.0.312-2.b07.dev.redhat.windows.x86_64\bin\java.exe ...
5 s 65 ms Connected to the target VM, address: '127.0.0.1:56708', transport: 'socket'
0
1
2
3
4
Disconnected from the target VM, address: '127.0.0.1:56708', transport: 'socket'
Process finished with exit code 0
```

以上用例的具体减少, 请参见 尼恩《100wqps 日志平台实操, 视频》

多个消费者串行消费场景

在 多个消费者串行消费场景中，多个消费者，可以按照次序，消费消息。

比如：一个用户注册的Event，需要有一个Handler来存储信息，一个Hanlder来发邮件等等。



多个消费者串行消费场景案例

```
@org.junit.Test
public void testMultiSerialConsumerDisruptorWithMethodRef() throws InterruptedException {
    // 消费者线程池
    Executor executor = Executors.newCachedThreadPool();
    // 环形队列大小，2的指数
    int bufferSize = 1024;
    // 构造 分裂者（事件分发者）
    Disruptor<LongEvent> disruptor = new Disruptor<LongEvent>(LongEvent::new, bufferSize,
        executor,
        ProducerType.SINGLE, //多个生产者
        new YieldingWaitStrategy());
    // 连接 消费者 处理器
    // 可以使用lambda来注册一个EventHandler
    disruptor.handleEventsWith(LongEventSceneDemo::handleEvent)
        .then(LongEventSceneDemo::handleEvent2)
        .then(LongEventSceneDemo::handleEvent3);
    // 开启 分裂者（事件分发）
    disruptor.start();
    // 获取环形队列，用于生产 事件
    RingBuffer<LongEvent> ringBuffer = disruptor.getRingBuffer();
    //1生产者，并发生产数据
    LongEventProducerWithTranslator producer = new LongEventProducerWithTranslator(ringBuffer);
    Thread thread = new Thread() {@Override public void run() {
        for (long i = 0; true; i++) { /*发布事件*/producer.onData(i);ThreadUtil.sleepSeconds(1); } }};
    thread.start();
    ThreadUtil.sleepSeconds(5);
}
```

CSDN @架构师-尼恩

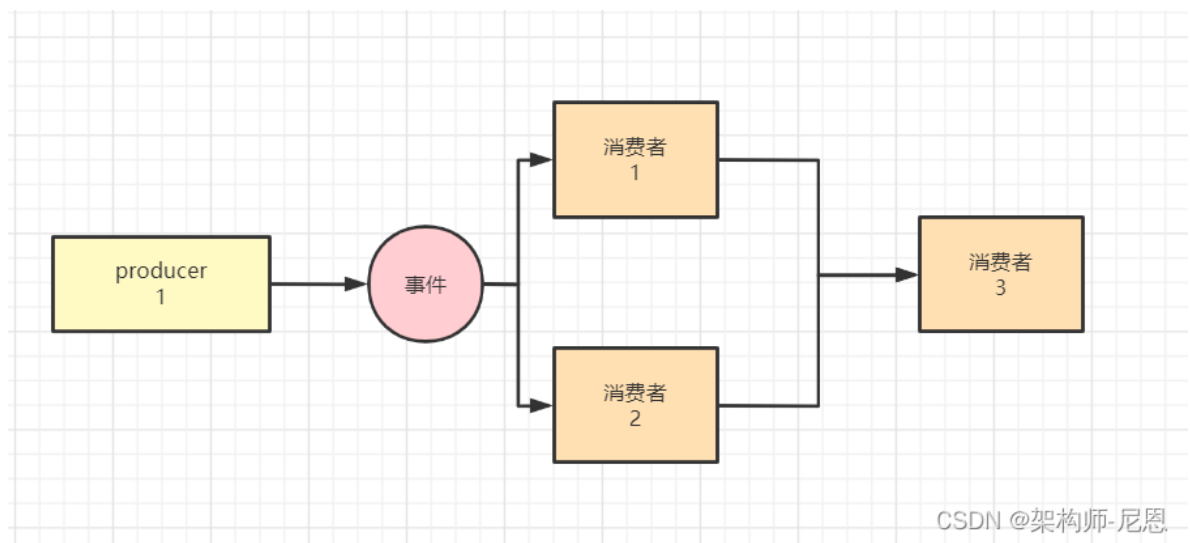
执行结果

```
LongEventSceneDemo.testWithSerialConsumerDisruptorWithMethodRef ...
Debugger Console
Tests passed: 1 of 1 test - 5 s 66 ms
5 s 66 ms D:\dev\jdk\java-1.8.0-openjdk-1.8.0.312-2.b07.dev.redhat.windows.x86_64\bin\java.exe ...
5 s 66 ms Connected to the target VM, address: '127.0.0.1:57169', transport: 'socket'
[pool-1-thread-1|LongEventSceneDemo.handleEvent]: 0
[pool-1-thread-2|LongEventSceneDemo.handleEvent2]: 0
[pool-1-thread-3|LongEventSceneDemo.handleEvent3]: 0
[pool-1-thread-1|LongEventSceneDemo.handleEvent]: 1
[pool-1-thread-2|LongEventSceneDemo.handleEvent2]: 1
[pool-1-thread-3|LongEventSceneDemo.handleEvent3]: 1
[pool-1-thread-1|LongEventSceneDemo.handleEvent]: 2
[pool-1-thread-2|LongEventSceneDemo.handleEvent2]: 2
[pool-1-thread-3|LongEventSceneDemo.handleEvent3]: 2
[pool-1-thread-1|LongEventSceneDemo.handleEvent]: 3
[pool-1-thread-2|LongEventSceneDemo.handleEvent2]: 3
[pool-1-thread-3|LongEventSceneDemo.handleEvent3]: 3
[pool-1-thread-1|LongEventSceneDemo.handleEvent]: 4
[pool-1-thread-2|LongEventSceneDemo.handleEvent2]: 4
[pool-1-thread-3|LongEventSceneDemo.handleEvent3]: 4
Disconnected from the target VM, address: '127.0.0.1:57169', transport: 'socket'
Process finished with exit code 0
CSDN @架构师-尼恩
```

菱形方式执行场景

场景特点

先并发，后串行



菱形方式执行场景案例

```

9
10
11 @org.junit.Test
12 public void testCurrentThenSerialConsumerDisruptorWithMethodRef() throws InterruptedException {
13     // 消费者线程池
14     Executor executor = Executors.newCachedThreadPool();
15     // 环形队列大小，2的指数
16     int bufferSize = 1024;
17     // 构造 分裂者 (事件分发者)
18     Disruptor<LongEvent> disruptor = new Disruptor<LongEvent>(LongEvent::new, bufferSize,
19         executor,
20         ProducerType.SINGLE, //多个生产者
21         new YieldingWaitStrategy());
22     // 连接 消费者 处理器
23     // 可以使用lambda来注册一个EventHandler
24     disruptor.handleEventsWith(LongEventSceneDemo::handleEvent, LongEventSceneDemo::handleEvent2)
25         .then(LongEventSceneDemo::handleEvent3);
26     // 开启 分裂者 (事件分发)
27     disruptor.start();
28     // 获取环形队列，用于生产 事件
29     RingBuffer<LongEvent> ringBuffer = disruptor.getRingBuffer();
30     //1生产者，并发生产数据
31     LongEventProducerWithTranslator producer = new LongEventProducerWithTranslator(ringBuffer);
32     Thread thread = new Thread() { @Override public void run() { for (long i = 0; true; i++)
33         { /*发布事件*/ producer.onData(i); ThreadUtil.sleepSeconds(1); } }
34     };
35     thread.start();
36     ThreadUtil.sleepSeconds(5);
37 }
38
39 }

```

CSDN @架构师-尼恩

执行结果

```

>> Tests passed: 1 of 1 test - 5 s 64 ms
D:\dev\jdk\java-1.8.0-openjdk-1.8.0.312-2.b07.dev.redhat.windows.x86_64\bin\java.exe -agent
[pool-1-thread-2|LongEventSceneDemo.handleEvent2]: 0
[pool-1-thread-1|LongEventSceneDemo.handleEvent]: 0
[pool-1-thread-3|LongEventSceneDemo.handleEvent3]: 0
[pool-1-thread-1|LongEventSceneDemo.handleEvent]: 1
[pool-1-thread-2|LongEventSceneDemo.handleEvent2]: 1
[pool-1-thread-3|LongEventSceneDemo.handleEvent3]: 1
[pool-1-thread-2|LongEventSceneDemo.handleEvent2]: 2
[pool-1-thread-1|LongEventSceneDemo.handleEvent]: 2
[pool-1-thread-3|LongEventSceneDemo.handleEvent3]: 2
[pool-1-thread-1|LongEventSceneDemo.handleEvent]: 3
[pool-1-thread-2|LongEventSceneDemo.handleEvent2]: 3
[pool-1-thread-3|LongEventSceneDemo.handleEvent3]: 3
[pool-1-thread-2|LongEventSceneDemo.handleEvent2]: 4
[pool-1-thread-1|LongEventSceneDemo.handleEvent]: 4
[pool-1-thread-3|LongEventSceneDemo.handleEvent3]: 4
Disconnected from the target VM, address: '127.0.0.1:49515', transport: CSDN@架构师-尼恩

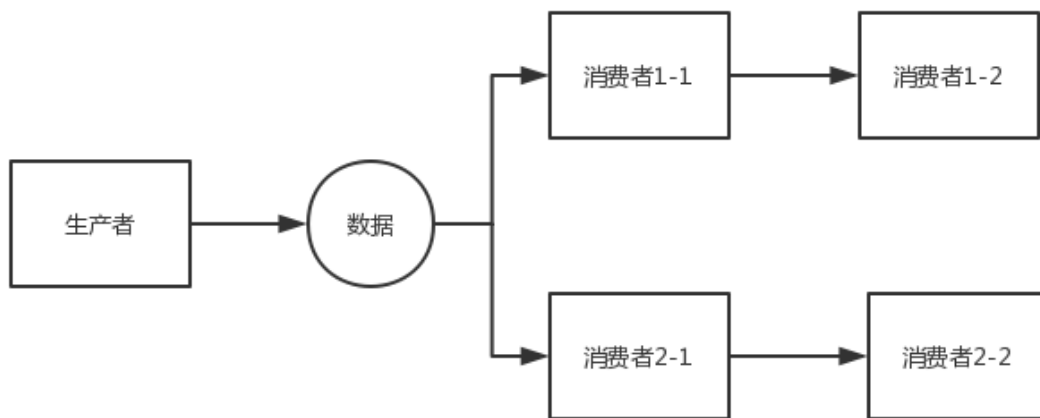
```

链式并行执行场景

场景特点

多组消费者形成 并行链，特点是：

- 链内 串行
- 链间 并行



场景案例

```
occurentDemos / src / main / java / com / crazymakercircle / disruptor / LongEventSceneDemo /
LongEventProducerWithTranslator.java x LongEventSceneDemo.java x EventHandlerGroup.java x Disruptor.java x WorkHandler.java x
@org.junit.Test
public void testlinkSerialConsumerDisruptorWithMethodRef() throws InterruptedException {
    // 消费者线程池
    Executor executor = Executors.newCachedThreadPool();
    // 环形队列大小, 2的指数
    int bufferSize = 1024;
    // 构造 分裂者 (事件分发者)
    Disruptor<LongEvent> disruptor = new Disruptor<LongEvent>(LongEvent::new, bufferSize,
        executor,
        ProducerType.SINGLE, //多个生产者
        new YieldingWaitStrategy());
    // 连接 消费者 处理器
    // 可以使用lambda来注册一个EventHandler
    disruptor.handleEventsWith(LongEventSceneDemo::handleEvent1)
        .then(LongEventSceneDemo::handleEvent2);
    disruptor.handleEventsWith(LongEventSceneDemo::handleEvent3)
        .then(LongEventSceneDemo::handleEvent4);
    // 开启 分裂者 (事件分发)
    disruptor.start();
    // 获取环形队列, 用于生产 事件
    RingBuffer<LongEvent> ringBuffer = disruptor.getRingBuffer();
    //1生产者, 并发生产数据
    LongEventProducerWithTranslator producer = new LongEventProducerWithTranslator(ringBuffer);
    Thread thread = new Thread() {
        @Override
        public void run() {
            for (long i = 0; true; i++) { producer.onData(i); ThreadUtil.sleepSeconds(1); }
        }
    };
    thread.start();
    ThreadUtil.sleepSeconds(5);
}
```

CSDN @架构师-尼恩

执行结果

```
rebug: LongEventSceneDemo.testLinkSerialConsumerDisruptorWithMethodRef
Debugger Console
Tests passed: 1 of 1 test - 5 s 62 ms
D:\dev\jdk\java-1.8.0-openjdk-1.8.0.312-2.b07.dev.redhat.windows.x86_64\bin\java.exe ...
Connected to the target VM, address: '127.0.0.1:49609', transport: 'socket'
[pool-1-thread-1|LongEventSceneDemo.handleEvent1]: 0
[pool-1-thread-3|LongEventSceneDemo.handleEvent3]: 0
[pool-1-thread-2|LongEventSceneDemo.handleEvent2]: 0
[pool-1-thread-4|LongEventSceneDemo.handleEvent4]: 0
[pool-1-thread-3|LongEventSceneDemo.handleEvent3]: 1
[pool-1-thread-1|LongEventSceneDemo.handleEvent1]: 1
[pool-1-thread-4|LongEventSceneDemo.handleEvent4]: 1
[pool-1-thread-2|LongEventSceneDemo.handleEvent2]: 1
[pool-1-thread-3|LongEventSceneDemo.handleEvent3]: 2
[pool-1-thread-1|LongEventSceneDemo.handleEvent1]: 2
[pool-1-thread-4|LongEventSceneDemo.handleEvent4]: 2
[pool-1-thread-2|LongEventSceneDemo.handleEvent2]: 2
[pool-1-thread-1|LongEventSceneDemo.handleEvent1]: 3
[pool-1-thread-3|LongEventSceneDemo.handleEvent3]: 3
[pool-1-thread-2|LongEventSceneDemo.handleEvent2]: 3
[pool-1-thread-4|LongEventSceneDemo.handleEvent4]: 3
[pool-1-thread-1|LongEventSceneDemo.handleEvent1]: 4
[pool-1-thread-3|LongEventSceneDemo.handleEvent3]: 4
[pool-1-thread-2|LongEventSceneDemo.handleEvent2]: 4
[pool-1-thread-4|LongEventSceneDemo.handleEvent4]: 4
Disconnected from the target VM, address: '127.0.0.1:49609', transport: 'socket'
```

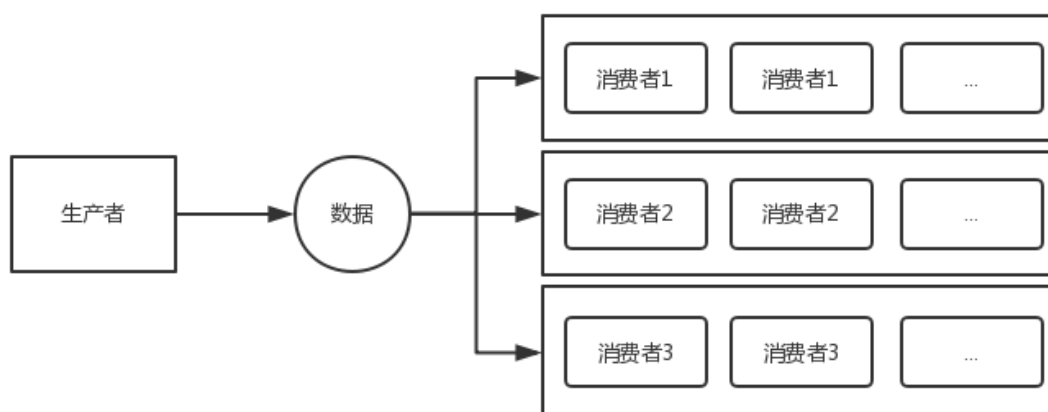
CSDN @架构师-尼恩

多组消费者相互隔离场景

场景特点

多组消费者 相互隔离，特点是：

- 组内 相互竞争
- 组间 相互隔离



场景案例

```
@org.junit.Test
public void testIsolateDisruptorWithMethodRef() throws InterruptedException {
    // 消费者线程池
    Executor executor = Executors.newCachedThreadPool();
    // 环形队列大小, 2的指数
    int bufferSize = 1024;
    // 构造 分裂者 (事件分发者)
    Disruptor<LongEvent> disruptor = new Disruptor<LongEvent>(LongEvent::new, bufferSize,
        executor,
        ProducerType.SINGLE, //多个生产者
        new YieldingWaitStrategy());
    // 连接 消费者 处理器
    // 可以使用lambda来注册一个EventHandler
    disruptor.handleEventsWithWorkerPool(new LongEventWorkHandler(), new LongEventWorkHandler());
    disruptor.handleEventsWithWorkerPool(new LongEventWorkHandler2(), new LongEventWorkHandler2());
    // 开启 分裂者 (事件分发)
    disruptor.start();
    // 获取环形队列, 用于生产 事件
    RingBuffer<LongEvent> ringBuffer = disruptor.getRingBuffer();
    // 1生产者, 并发生产数据
    LongEventProducerWithTranslator producer = new LongEventProducerWithTranslator(ringBuffer);
    Thread thread = new Thread() {
        @Override
        public void run() { for (long i = 0; true; i++) { producer.onData(i); ThreadUtil.sleepSeconds(1); } }
    };
    thread.start();
    ThreadUtil.sleepSeconds(5);
}
```

CSDN @架构师-尼恩

执行结果

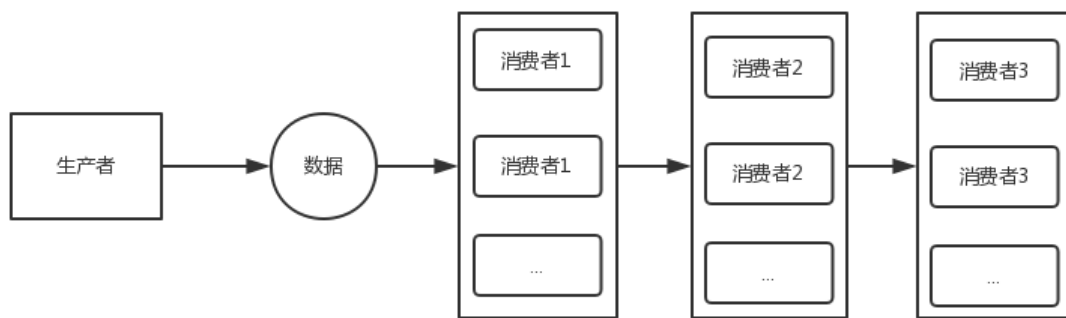
```
er Console
Tests passed: 1 of 1 test - 5 s 68 ms
D:\dev\jdk\java-1.8.0-openjdk-1.8.0.312-2.b07.dev.redhat.windows.x86_64\bin\java.exe ...
Connected to the target VM, address: '127.0.0.1:51217', transport: 'socket'
[pool-1-thread-3|LongEventWorkHandler2.onEvent]: 0
[pool-1-thread-1|LongEventWorkHandler.onEvent]: 0
[pool-1-thread-4|LongEventWorkHandler2.onEvent]: 1
[pool-1-thread-2|LongEventWorkHandler.onEvent]: 1
[pool-1-thread-1|LongEventWorkHandler.onEvent]: 2
[pool-1-thread-3|LongEventWorkHandler2.onEvent]: 2
[pool-1-thread-4|LongEventWorkHandler2.onEvent]: 3
[pool-1-thread-2|LongEventWorkHandler.onEvent]: 3
[pool-1-thread-3|LongEventWorkHandler2.onEvent]: 4
[pool-1-thread-1|LongEventWorkHandler.onEvent]: 4
Disconnected from the target VM, address: '127.0.0.1:51217', transport: 'socket'
Process finished with exit code 0
CSDN @架构师-尼恩
```

多组消费者航道执行模式

场景特点

多组消费者形成 并行链, 特点是:

- 组内 相互竞争
- 组之间串行依次执行



场景案例

组之间串行依次执行,组内有多个实例竞争执行

```
EventSceneDemo.java x EventHandlerGroup.java x Disruptor.java x WorkHandler.java x LongEventHandler.java x
public void testChannelModelDisruptorWithMethodRef() throws InterruptedException {
    // 消费者线程池
    Executor executor = Executors.newCachedThreadPool();
    // 环形队列大小, 2的指数
    int bufferSize = 1024;
    // 构造 分裂者 (事件分发者)
    Disruptor<LongEvent> disruptor = new Disruptor<LongEvent>(LongEvent::new, bufferSize,
        executor,
        ProducerType.SINGLE, //多个生产者
        new YieldingWaitStrategy());
    // 连接 消费者 处理器
    // 可以使用lambda来注册一个EventHandler
    disruptor.handleEventsWithWorkerPool(new LongEventWorkHandler(), new LongEventWorkHandler())
        .thenHandleEventsWithWorkerPool(new LongEventWorkHandler2(), new LongEventWorkHandler2());
    // 开启 分裂者 (事件分发)
    disruptor.start();
    // 获取环形队列, 用于生产 事件
    RingBuffer<LongEvent> ringBuffer = disruptor.getRingBuffer();
    // 1生产者, 并生产数据
    LongEventProducerWithTranslator producer = new LongEventProducerWithTranslator(ringBuffer);
    Thread thread = new Thread() {
        @Override
        public void run() { for (long i = 0; true; i++) { producer.onData(i); ThreadUtil.sleepSeconds(1); } }
    };
    thread.start();
    ThreadUtil.sleepSeconds(5);
}
```

CSDN @架构师-尼恩

执行效果

```
ger Console →
Tests passed: 1 of 1 test - 5 s 68 ms
D:\dev\jdk\java-1.8.0-openjdk-1.8.0.312-2.b07.dev.redhat.windows.x86_64\bin\java.exe ...
Connected to the target VM, address: '127.0.0.1:51360', transport: 'socket'
[pool-1-thread-1|LongEventWorkHandler.onEvent]: 0
[pool-1-thread-3|LongEventWorkHandler2.onEvent]: 0
[pool-1-thread-2|LongEventWorkHandler.onEvent]: 1
[pool-1-thread-4|LongEventWorkHandler2.onEvent]: 1
[pool-1-thread-1|LongEventWorkHandler.onEvent]: 2
[pool-1-thread-3|LongEventWorkHandler2.onEvent]: 2
[pool-1-thread-2|LongEventWorkHandler.onEvent]: 3
[pool-1-thread-4|LongEventWorkHandler2.onEvent]: 3
[pool-1-thread-1|LongEventWorkHandler.onEvent]: 4
[pool-1-thread-3|LongEventWorkHandler2.onEvent]: 4
Disconnected from the target VM, address: '127.0.0.1:51360', transport: 'socket'

Process finished with exit code 0

CSDN @架构师-尼恩
```

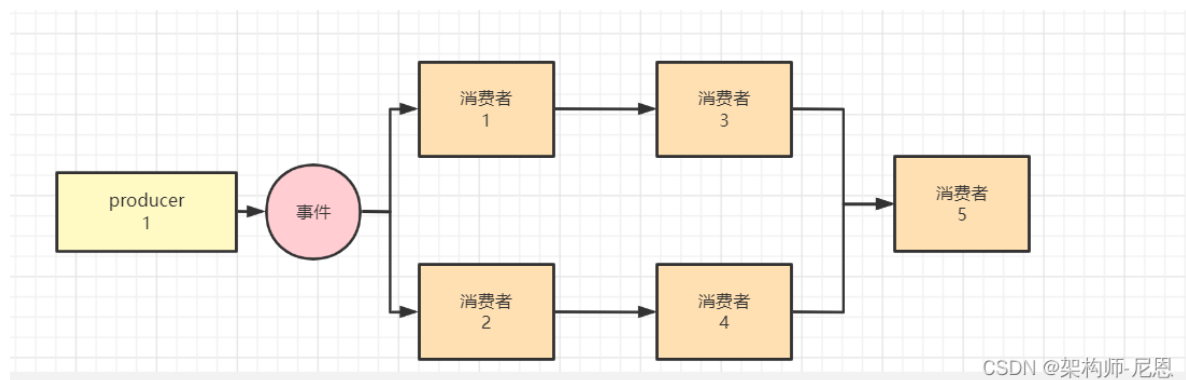
六边形执行顺序

这是一种比较复杂的场景

场景特点

单边内部是有序的

边和边之间是并行的



CSDN @架构师-尼恩

参考代码

```
@org.junit.Test
public void testHexagonConsumerDisruptorwithMethodRef() throws
InterruptedException {
    // 消费者线程池
    Executor executor = Executors.newCachedThreadPool();
    // 环形队列大小, 2的指数
    int bufferSize = 1024;
    // 构造 分裂者 (事件分发者)
    Disruptor<LongEvent> disruptor = new Disruptor<LongEvent>
(LongEvent::new, bufferSize,
    executor,
    ProducerType.SINGLE, //多个生产者
    new YieldingWaitStrategy());

    EventHandler consumer1 = new LongEventHandlerWithName("consumer 1");
    EventHandler consumer2 = new LongEventHandlerWithName("consumer 2");
    EventHandler consumer3 = new LongEventHandlerWithName("consumer 3");
    EventHandler consumer4 = new LongEventHandlerWithName("consumer 4");
    EventHandler consumer5 = new LongEventHandlerWithName("consumer 5");
    // 连接 消费者 处理器
    // 可以使用lambda来注册一个EventHandler

    disruptor.handleEventsWith(consumer1,consumer2);
    disruptor.after(consumer1).handleEventsWith(consumer3);
    disruptor.after(consumer2).handleEventsWith(consumer4);
    disruptor.after(consumer3,consumer4).handleEventsWith(consumer5);
    // 开启 分裂者 (事件分发)
```

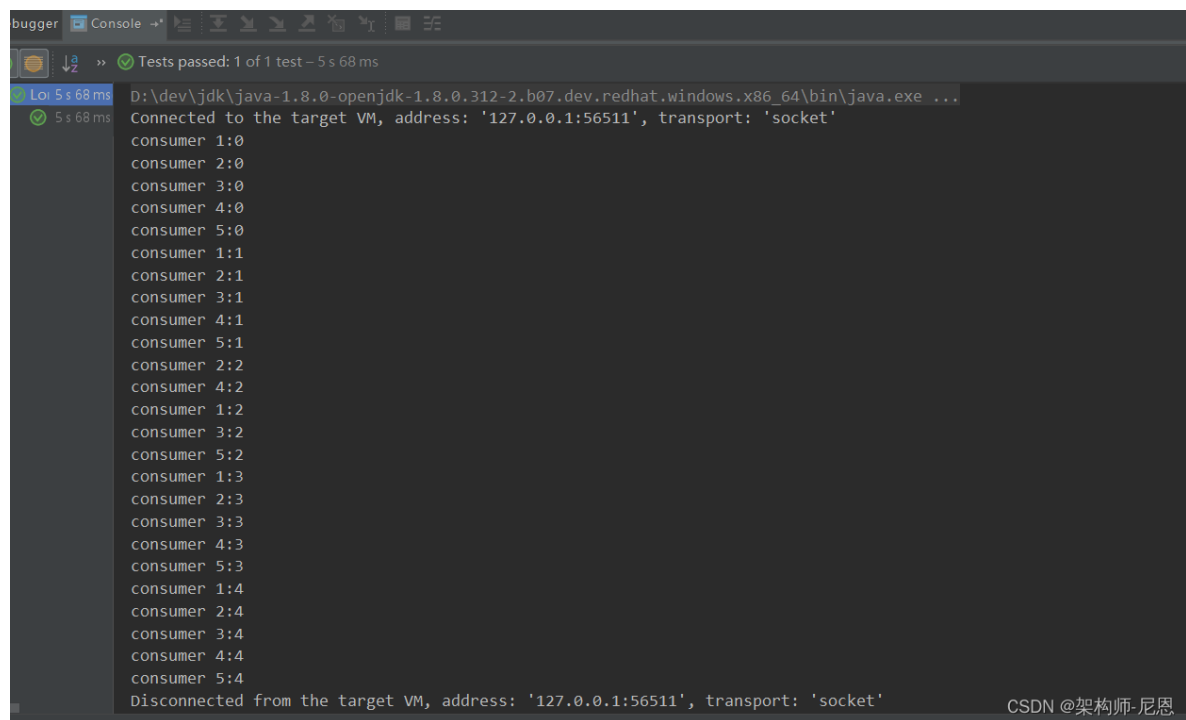


```

disruptor.start();
// 获取环形队列，用于生产 事件
RingBuffer<LongEvent> ringBuffer = disruptor.getRingBuffer();
//1生产者，并发生产数据
LongEventProducerWithTranslator producer = new
LongEventProducerWithTranslator(ringBuffer);
Thread thread = new Thread() {
    @Override
    public void run() {
        for (long i = 0; true; i++) {
            producer.onData(i);
            ThreadUtil.sleepSeconds(1);
        }
    }
};
thread.start();
ThreadUtil.sleepSeconds(5);
}

```

执行结果



The screenshot shows a debugger window with a console tab. The console output indicates that a test passed in 5 s 68 ms. Below this, the output of a Java process is shown, displaying a sequence of consumer messages in the format 'consumer id:index', where 'id' ranges from 1 to 5 and 'index' ranges from 0 to 4. The output ends with a 'Disconnected from the target VM' message. The source code for the producer thread from the previous block is visible in the background.

```

bugger Console →
Tests passed: 1 of 1 test - 5 s 68 ms
Lo: 5 s 68 ms
Connected to the target VM, address: '127.0.0.1:56511', transport: 'socket'
consumer 1:0
consumer 2:0
consumer 3:0
consumer 4:0
consumer 5:0
consumer 1:1
consumer 2:1
consumer 3:1
consumer 4:1
consumer 5:1
consumer 2:2
consumer 4:2
consumer 1:2
consumer 3:2
consumer 5:2
consumer 1:3
consumer 2:3
consumer 3:3
consumer 4:3
consumer 5:3
consumer 1:4
consumer 2:4
consumer 3:4
consumer 4:4
consumer 5:4
Disconnected from the target VM, address: '127.0.0.1:56511', transport: 'socket'
CSDN @架构师-尼恩

```

4：架构师视角，深入Disruptor源码分析

Disruptor其实是“生产者-消费者”模型一种典型的应用场合，它的功能其实就是一种有界队列。

核心概念

Ring Buffer

如其名，环形的缓冲区。

曾经 RingBuffer 是 Disruptor 中的最主要的对象，但从3.0版本开始，其职责被简化为仅仅负责对通过 Disruptor 进行交换的数据（事件）进行存储和更新。

在一些更高级的应用场景中，Ring Buffer 可以由用户的自定义实现来完全替代。

Sequence

通过顺序递增的序号来编号管理通过其进行交换的数据（事件），对数据(事件)的处理过程总是沿着序号逐个递增处理。

Sequence 采用缓存行填充的方式对long类型的一层包装，用以代表事件的序号。

一个 Sequence 用于跟踪标识某个特定的事件处理器(RingBuffer/Consumer)的处理进度。

虽然一个 AtomicLong 也可以用于标识进度，但定义 Sequence 来负责该问题还有另一个目的，那就是防止不同的 Sequence 之间的CPU缓存伪共享(Flase Sharing)问题。

另外，Sequence 通过 unsafe 的cas方法从而避免了锁的开销。

Sequencer

Sequencer 是 Disruptor 的真正核心。

生产者与缓存RingBuffer之间的桥梁、

此接口有两个实现类 SingleProducerSequencer、MultiProducerSequencer，它们定义在生产者和消费者之间快速、正确地传递数据的并发算法。

Sequence Barrier

消费者与消费者直接的隔离屏障。

消费者之间，并不是通过 RingBuffer 进行加锁互斥隔离，而是通过 Sequence Barrier 来管理依赖次序关系，从而能减少RingBuffer上的并发冲突；

在一定程度上，Sequence Barrier 类似与 aqs 同步队列

Sequence Barrier 用于保持对 RingBuffer 的 main published Sequence 和Consumer依赖的其它 Consumer的 Sequence 的引用。

Sequence Barrier 还定义了：Consumer 是否还有可处理的事件的逻辑。

Wait Strategy

定义 Consumer 如何进行等待下一个事件的策略。（注：Disruptor 定义了多种不同的策略，针对不同的场景，提供了不一样的性能表现）

Event

在 Disruptor 的语义中，生产者和消费者之间进行交换的数据被称为事件(Event)。

它不是一个被 Disruptor 定义的特定类型，而是由 Disruptor 的使用者定义并指定。

EventProcessor

事件处理器，是消费者线程池Executor的调度单元，

EventProcessor 是对事件业务处理EventHandler与异常处理ExceptionHandler等的一层封装；

EventProcessor 持有特定消费者(Consumer)的 Sequence，并提供事件循环(Event Loop)，用于调用业务事件处理实现EventHandler

EventHandler

Disruptor 定义的事件处理接口，由用户实现，用于处理事件，是 Consumer 的真正实现。

Producer

即生产者，只是泛指调用 Disruptor 发布事件的用户代码，Disruptor 没有定义特定接口或类型

RingBuffer

基于数组的缓存实现，也是创建sequencer与定义WaitStrategy的入口；

Disruptor

Disruptor的使用入口。

持有RingBuffer、消费者线程池Executor、消费者仓库ConsumerRepository等引用。

Disruptor的无锁架构

并发领域的一个典型场景是生产者消费者模型，生产者消费者模型的经典方式，是使用queue作为生产者线程与消费者线程之间共享数据的方法，但是，经典方式对于queue的读写避免不了读写锁的竞争。

通过序号屏障对依赖关系的管理，RingBuffer实现了事件缓存的无锁架构。

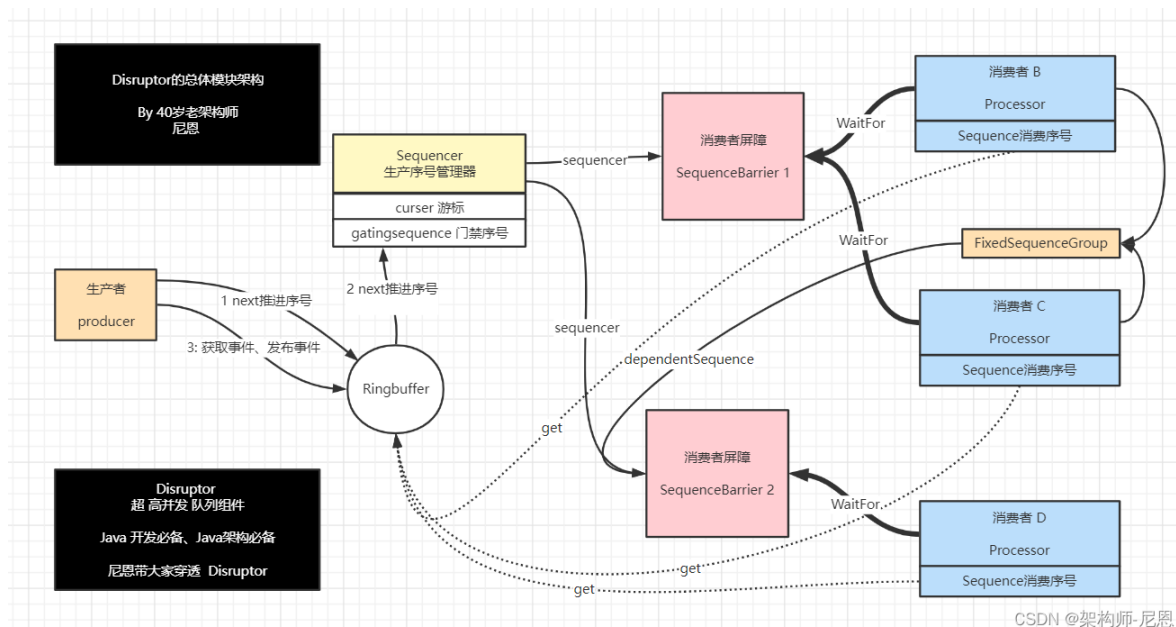
Disruptor使用环形缓冲区RingBuffer作为共享数据的媒介。

生产者通过Sequencer控制RingBuffer，以及唤醒等待事件的消费者，

消费者通过SequenceBarrier监听RingBuffer的可消费事件。

考虑一个场景，一个生产者A与三个消费者B、C、D,同时D的事件处理需要B与C先完成。

则该模型结构如下：



Disruptor 中，生产者与Sequencer有关系，由生产者通过Sequencer控制RingBuffer的写入。

RingBuffer是Disruptor高性能的一个亮点。RingBuffer就是一个大数组，事件以循环覆盖的方式写入。

与常规RingBuffer拥有2个首尾指针的方式不同，Disruptor的RingBuffer只有一个指针(或称序号)，指向数组下一个可写入的位置，该序号在Disruptor源码中就是**Sequencer中的cursor**，

如何管理消费者和生产者之间的依赖关系呢？

还是通过SequenceBarrier 进行依赖管理，

消费者的 processer，通过 SequenceBarrier 获取生产者的 生产 序号

如何管理消费者与消费者之间的依赖关系呢？

每个消费者拥有各自独立的事件序号Sequence，消费者之间不通过Sequence在共享竞态，或者说依赖管理。

消费者与消费者之间的依赖关系是，通过SequenceBarrier 进行依赖管理。

依赖关系管理的例子

在上面的例子中：

SequenceBarrier1 监听 RingBuffer 的序号 cursor，消费者B与C通过SequenceBarrier1等待可消费事件。

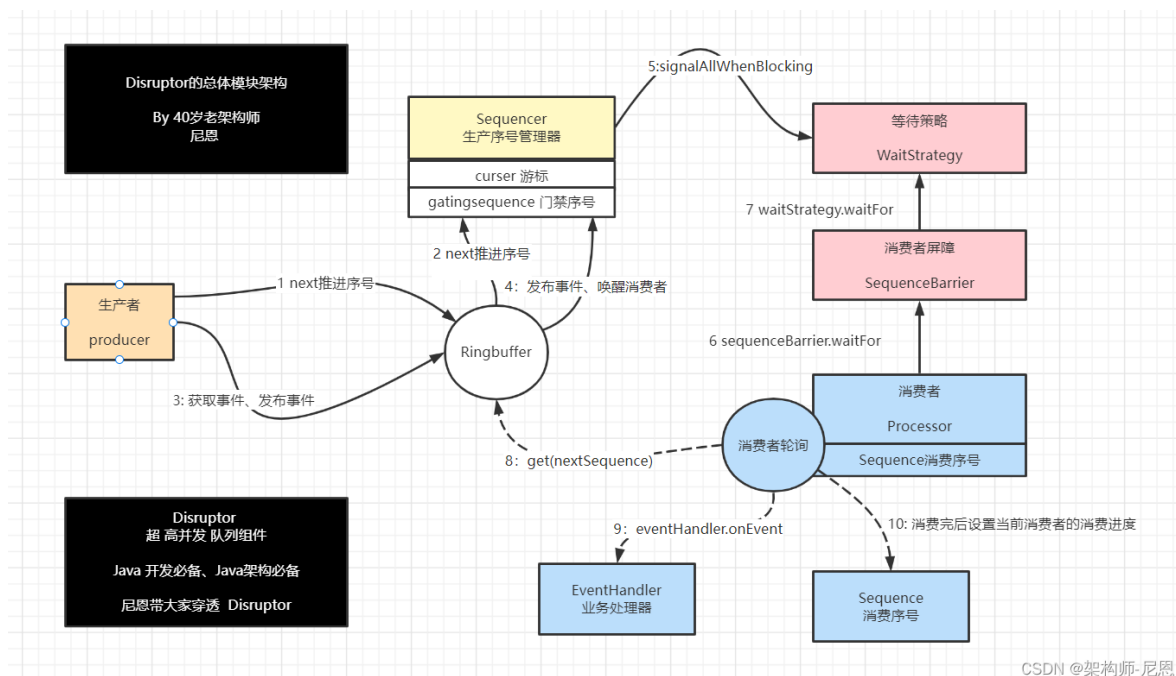
SequenceBarrier2 除了监听 RingBuffer 的序号cursor，同时也监听B与C的序号Sequence，从而将最小的序号返回给消费者D，由此实现了D依赖B与C的逻辑。

如何避免未消费事件的写入覆盖呢？

为了避免未消费事件的写入覆盖，生产者的 Sequencer需要监听所有消费者的消息处理进度，也就是 gatingSequences。

Disruptor的总体模块架构

结合执行流程进行梳理



核心类Sequence

用来表达 event 序列号的对象，但这里为什么不直接用 long 呢？

为了高并发下的可见性，肯定不能直接用 long 的，至少也是 volatile long。

但 Disruptor 觉得 volatile long 还是不够用，所以创造了 Sequence 类。

Sequence的内部实现主要是 volatile long，

```
volatile long value;
```

除此以外还支持以下特性：

- CAS 更新
- order writes (Store/Store barrier，改动不保证立即可见) vs volatile writes (Store/Load barrier，改动保证立即可见)
- 在 volatile 字段 附近添加 padding 解决伪共享问题

简单理解就是高并发下优化的 long 类型。

比如在对 EventProcessor.sequence 的更新中都是用的 order writes，不保证立即可见，但速度快很多。

在这个场景里，造成的结果是显示的消费进度可能比实际上慢，导致生产者有可能在可以生产的情况下没有去生产。

但生产者看的是多个消费者中最慢的那个消费进度，所以影响可能没有那么大。

核心类Sequencer

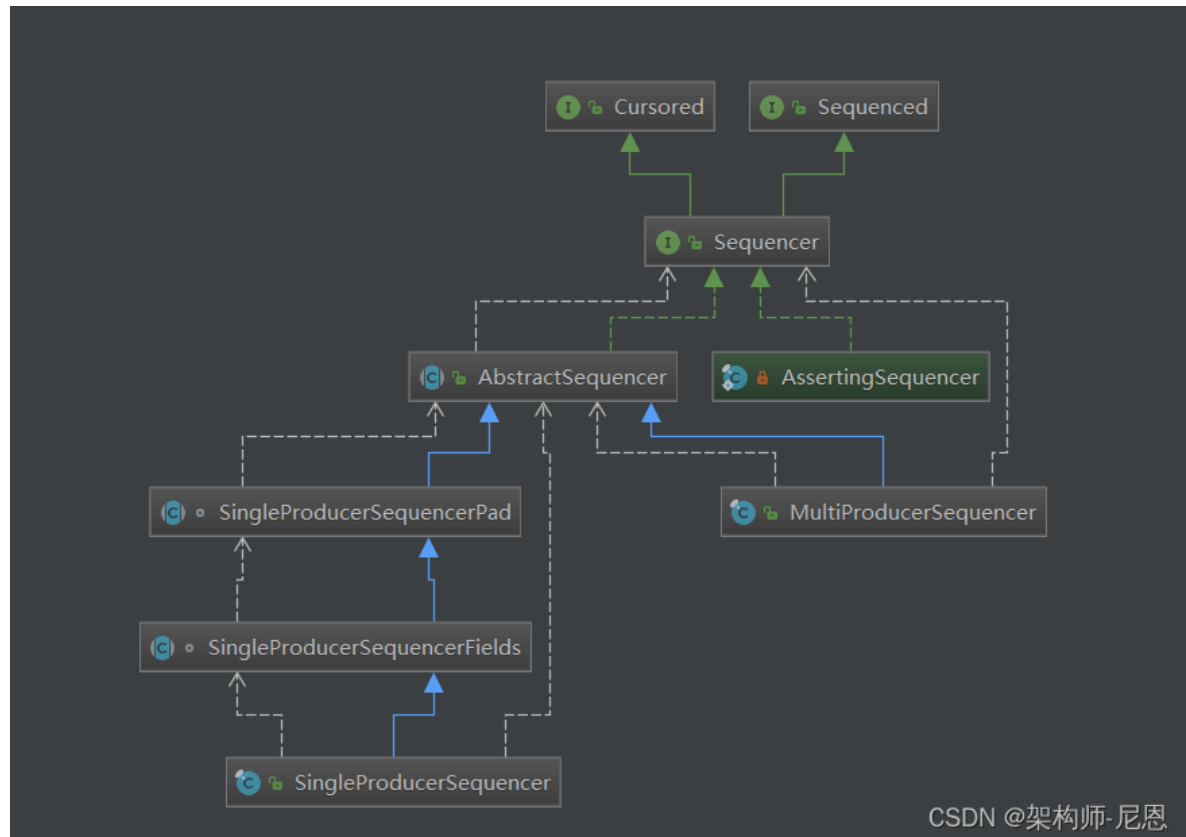
Sequencer 是生产者与缓存RingBuffer之间的桥梁、是 Disruptor 的真正核心。

Sequencer 负责在生产者和消费者之间快速、正确地传递数据的序号。

生产者发布 event 的时候首先需要预定一个 sequence，Sequencer 就是计算和发布 sequence 的。

Sequencer 的实现类

Sequencer 接口有两个重要实现类 SingleProducerSequencer、MultiProducerSequencer ,



CSDN @架构师-尼恩

SingleProducerSequencer

生产者发布事件的步骤:

- 通过 Sequencer.next(n) 来预定下面 n 个可以写入的位置序号
- 根据序号获取事件, 然后修改事件数据, 然后发布 event。

因为 RingBuffer 是环形的, 一个 size 为 1024 的 RingBuffer, 当拿到的序号 Sequence 为 1024 时, 相当于又要去写 0 位置,

问题来了, 假如之前的 0 位置的数据, 还没被消费呢?

此时, 不能直接写, 如果写的话, 老数据就会被覆盖了。

如何解决数据覆盖的问题呢?

答案就是使用: Sequencer。Sequencer 在内部维护了一个 gatingSequences 数组:

```
volatile Sequence[] gatingSequences = new Sequence[0];
```

gatingSequences 数据里边, 记录的是消费者的 Sequence ,

每个消费者会维护一个自己的 Sequence 对象, 来记录自己已经消费到的序列位置。

每添加一个消费者, 都会把消费者的 Sequence 引用添加到 gatingSequences 中。

通过访问 gatingSequences, Sequencer 可以得知消费的最慢的消费者消费到了哪个位置。

8个消费者的例子，
`gatingSequences=[7, 8, 9, 10, 3, 4, 5, 6, 11]`
最慢的消费完了3，此时可以写seq 19的数据，但不能写seq 20。

在 `next(n)`方法里，如果计算出的下一个 event 的 Sequence 值减去 `bufferSize`

```
long nextValue = this.nextValue;long nextSequence = nextValue + n;  
long wrapPoint = nextSequence - bufferSize;
```

得出来的 `wrapPoint > min(gatingSequences)`，说明即将写入的位置上，之前的 event 还有消费者没有消费，这时 `SingleProducerSequencer` 会等待并自旋。

```
while (wrapPoint > (minSequence = Util.getMinimumSequence(gatingSequences,  
nextValue))) {  
    LockSupport.parkNanos(1L);  
}
```

举个例子，`gatingSequences=[7, 8, 9, 10, 3, 4, 5, 6, 11]`，RingBuffer size 16 的情况下，如果算出来的 `nextSequence` 是 20，`wrapPoint` 是 $20-16=4$ ，这时 `gatingSequences` 里最小的是 3。

说明下一个打算写入的位置是 `wrapPoint 4`，但最慢的消费者才消费到 3，你不能去覆盖之前 4 上的数据，这时只能等待，等消费者把之前的 4 消费掉。

为什么 `wrapPoint = nextSequence - bufferSize`，而不是 `bufferSize` 的 n 倍呢，因为消费者只能落后生产者一圈，不然就已经存在数据覆盖了。

等到 `SingleProducerSequencer` 自旋到下一个位置所有人都消费过的时候，它就可以从 `next` 方法中返回，生产者拿着 `sequence` 就可以继续去发布。

MultiProducerSequencer

`MultiProducerSequencer` 是在多个生产者的场合使用的，多个生产者的情况下存在竞争，导致它的实现更加复杂。

```

int[] availableBuffer;
int indexMask;
int indexShift;
public MultiProducerSequencer(int bufferSize, final waitStrategy waitStrategy){

    super(bufferSize, waitStrategy);
    availableBuffer = new int[bufferSize];
    indexMask = bufferSize - 1;
    indexShift = Util.log2(bufferSize);
    initialiseAvailableBuffer();
}

```

数据结构上多出来的主要就是这个 availableBuffer，用来记录 RingBuffer 上哪些位置有数据可以读。

还是从 Sequencer.next(n)说起，计算下一个数据位 Sequence 的逻辑是一样的，包括消费者落后导致 Sequencer 自旋等待的逻辑。不同的是因为有多多个 publisher 同时访问 Sequencer.next(n)方法，所以在确定最终位置的时候用了一个 CAS 操作，如果失败了就自旋再来一次。

```

cursor.compareAndSet(current, next)

```

另一个不同的地方是 publish(final long sequence) 方法，SingleProducer 的版本很简单，就是移动了一下 cursor。

```

public void publish(long sequence){

    cursor.set(sequence);
    waitStrategy.signalAllWhenBlocking();

}

```

MultiProducer 的版本则是

```

public void publish(final long sequence){

    setAvailable(sequence);
    waitStrategy.signalAllWhenBlocking();

}

```

setAvailable 做了什么事呢，它去设置 availableBuffer 的状态位了。给定一个 sequence，先计算出对应的数组下标 index，然后计算出在那个 index 上要写的数据 availabilityFlag，最后执行

```

availableBuffer[index]=availabilityFlag

```


根据 calculateAvailabilityFlag(sequence) 方法计算出来的 availabilityFlag 实际上是该 sequence 环绕 RingBuffer 的圈数。

```
availableBuffer=[6, 6, 6, 6, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5]例子：前4个已经走到第6圈。
```

availableBuffer 主要用于判断一个 sequence 下的数据是否可用

```
public boolean isAvailable(long sequence){
    int index = calculateIndex(sequence);
    int flag = calculateAvailabilityFlag(sequence);
    long bufferAddress = (index * SCALE) + BASE;
    return UNSAFE.getIntVolatile(availableBuffer, bufferAddress) == flag;
}
```

作为比较，来看一下 SingleProducer 的方法

```
public boolean isAvailable(long sequence){
    return sequence <= cursor.get();
}
```

在单个生产者的场景下，publishEvent 的时候才会推进 cursor，所以只要 sequence<=cursor，就说明数据是可消费的。

多个生产者的场景下，在 next(n)方法中，就已经通过 cursor.compareAndSet(current, next) 移动 cursor 了，此时 event 还没有 publish，所以 cursor 所在的位置不能保证 event 一定可用。

在 publish 方法中是去 setAvailable(sequence)了，所以 availableBuffer 是数据是否可用的标志。那为什么值要写成圈数呢，应该是避免把上一轮的数据当成这一轮的数据，错误判断 sequence 是否可用。

另一个值得一提的是 getHighestPublishedSequence 方法，这个是消费者用来查询最高可用 event 数据的位置。

```
public long getHighestPublishedSequence(long lowerBound, long availableSequence)
{
    for (long sequence = lowerBound; sequence <= availableSequence; sequence++)
    {
        if (!isAvailable(sequence))
        {
            return sequence - 1;
        }
    }
    return availableSequence;
}
```

range, 依次查询 availableBuffer, 直到发现不可用的 Sequence, 那么该 Sequence 之前的都是可用的。或全部都是可用的。

单生产者的版本:

```
public long getHighestPublishedSequence(long lowerBound, long
availableSequence)
{
    return availableSequence;
}
```

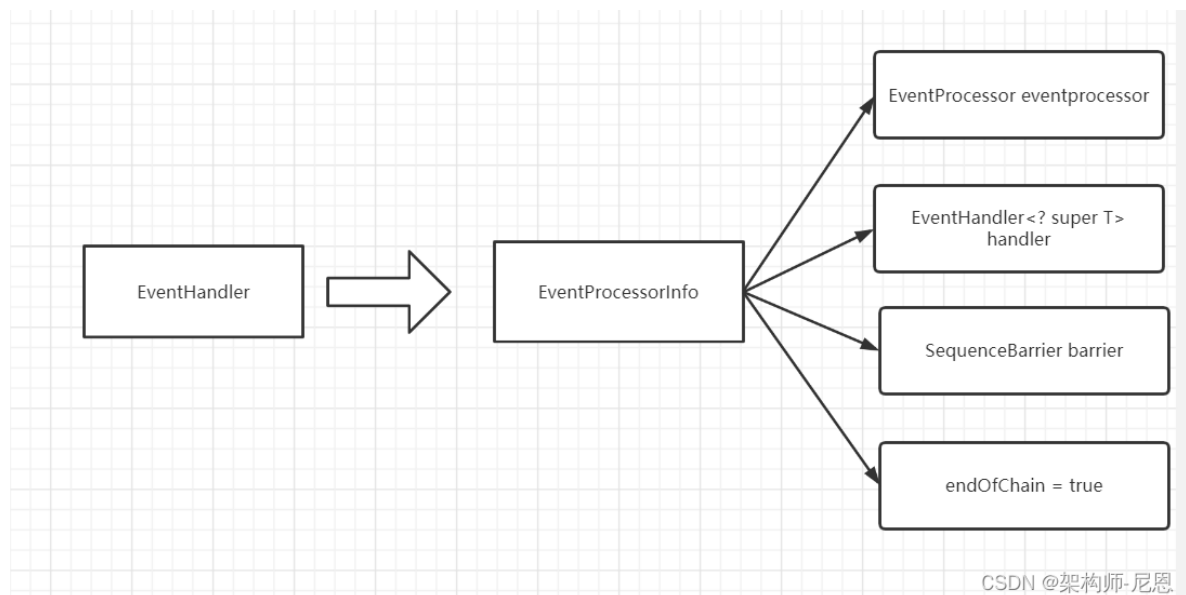
说完了生产者, 下面来看看消费者

核心类消费者仓库 和消费者信息

消费者仓库 ConsumerRepository

Disruptor中, 通过ConsumerRepository来管理所有消费者, 主要维护了以下结构:

- EventHandler 到 消费者处理器 信息的映射, 用于信息查询

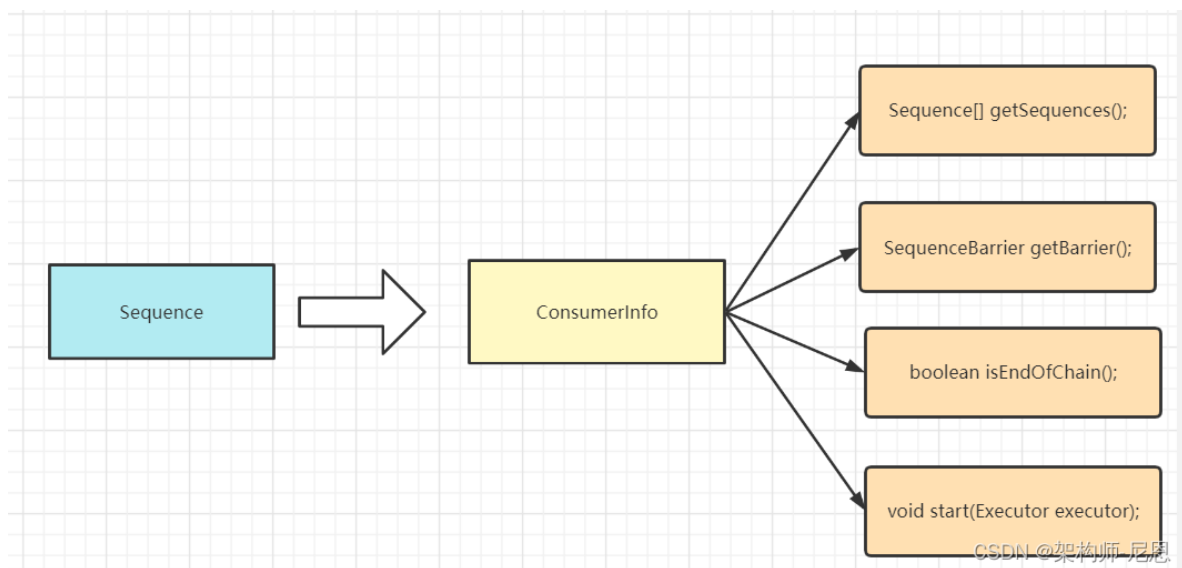


- Sequence 到消费者信息的映射

ConsumerInfo 和 Sequence 是一对多 关系

一个 ConsumerInfo 消费者 可能有多个Sequence

但是 一个Sequence只从属一个消费者。

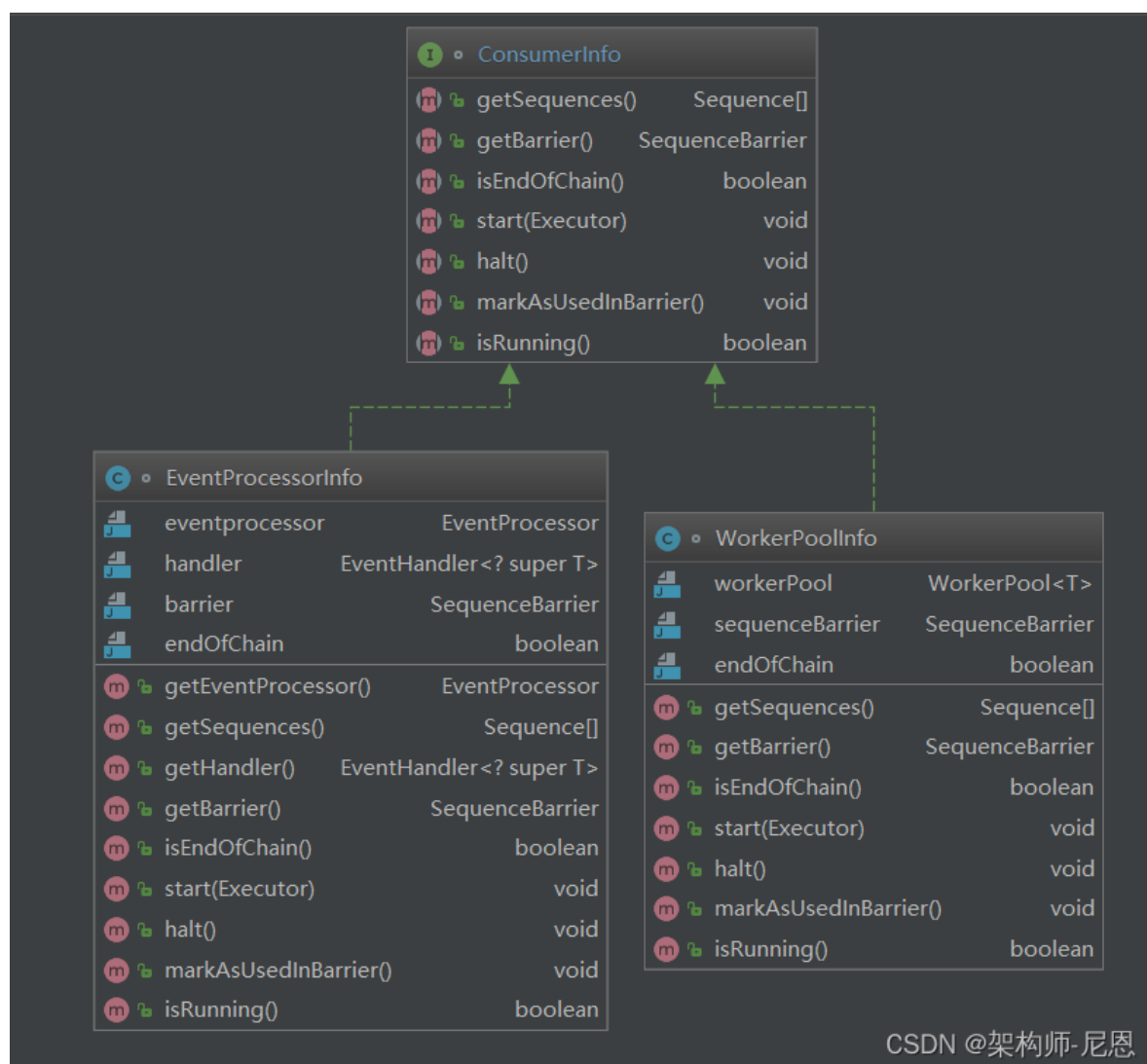


核心代码如下：

```
26  */
27
28  /**
29   * 提供 ConsumerInfo 消费者信息 仓库 机制，将 EventHandler 与 EventProcessor 关联起来。
30   * 传递给Disruptor的每一个EventHandler最终都会关联到一个 EventProcessor。
31   */
32  class ConsumerRepository<T> implements Iterable<ConsumerInfo>
33  {
34      /**
35       * EventHandler 到 消费者处理器 信息的映射，用于信息查询
36       */
37      private final Map<EventHandler<?>, EventProcessorInfo<T>> eventProcessorInfoByEventHandler =
38          new IdentityHashMap<>();
39
40      /**
41       * Sequence 到消费者信息的映射，ConsumerInfo 和 Sequence 是一对多 关系
42       * 一个 ConsumerInfo 消费者 可能有多个Sequence{@link WorkerPool}，
43       * 但是 一个Sequence只从属一个消费者。
44       */
45      private final Map<Sequence, ConsumerInfo> eventProcessorInfoBySequence =
46          new IdentityHashMap<>();
47
48      /**
49       * 消费者信息列表
50       */
51      private final Collection<ConsumerInfo> consumerInfos = new ArrayList<>();
52  }
```

消费者的信息 ConsumerInfo

ConsumerRepository用于维护Disruptor的所有消费者的信息，管理的集合类里主要有ConsumerInfo接口，



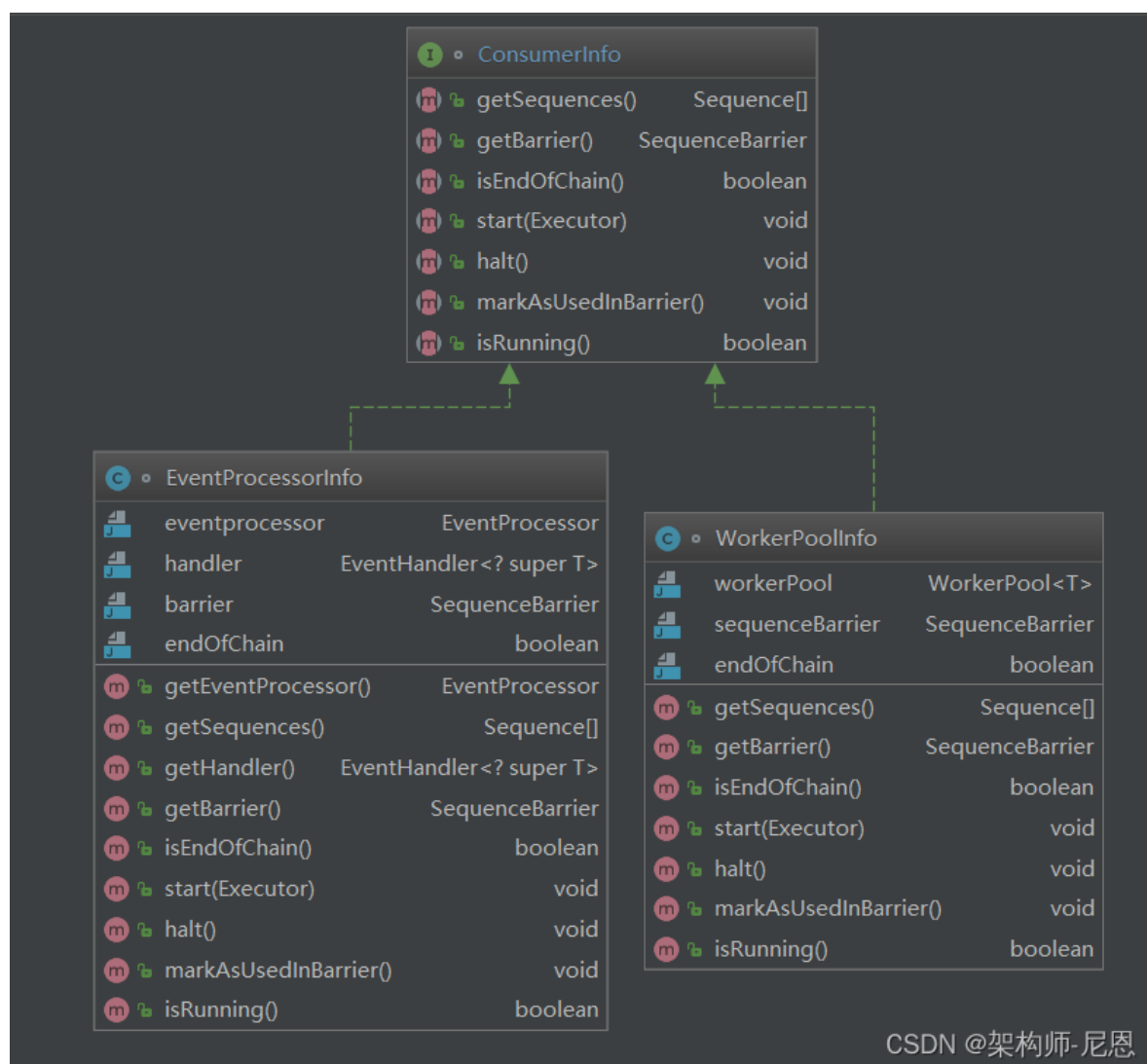
消费者的信息 实现类

ConsumerInfo 维护了消费者信息的抽象，目前主要有两个实现类：

- EventProcessorInfo 单事件处理器消费者信息
- WorkerPoolInfo 线程池消费者信息对象/工作者池信息。

EventProcessorInfo ： 一个单线程的消费者(只有一个EventProcessor)， 代理EventHandler， 管理处理事件以外的其他事情(如： 拉取事件， 等待事件...)

WorkerPoolInfo 表示： WorkPool整体是一个消费者， 是一个多线程的消费者， 每个生产者publish的事件只会被WorkerPool里的某一个WorkProcessor消费



CSDN @架构师-尼恩

WorkerPoolInfo多线程消费者信息

WorkerPoolInfo 包含了一个 WorkerPool 类型的成员

WorkerPool 和 处理器 没有任何 继承关系，是一个独立的类

协作者模式下，所有的消费者共用一个workSequence，通过CAS写workSequence

多线程消费者信息, 包含了多个 工作处理器，多个 工作处理器，放在下面的数组中

```
// workProcessors are created to wrap each of the provided workHandlersprivate
final workProcessor<?>[] workProcessors;
```

由 WorkerPool的start方法 启动 WorkProcessor 工作处理器

```
1 }
2
3 /**
4  * Start the worker pool processing events in sequence.
5  *
6  * @param executor providing threads for running the workers.
7  * @return the {@link RingBuffer} used for the work queue.
8  * @throws IllegalStateException if the pool has already been started and not halted yet
9  */
10 public RingBuffer<T> start(final Executor executor)
11 {
12     if (!started.compareAndSet( expect: false, update: true))
13     {
14         throw new IllegalStateException("WorkerPool has already been started and cannot be restarted until halted.");
15     }
16     // 生产者生产前，先获取生产者的cursor赋值给消费者们共用的workSequence，后续workSequence在消费者WorkProcessor的run方法会执行++操作
17     final long cursor = ringBuffer.getCursor();
18     workSequence.set(cursor);
19
20     for (WorkProcessor<?> processor : workProcessors)
21     {
22         // 同样，每个消费者线程启动前，给每个消费者的消费进度sequence赋值生产者的cursor初值
23         processor.getSequence().set(cursor);
24         // 【注意】如果有10个消费者共同消费，然后只有一个消费者线程的话，
25         // 此时生产者生产一个数据后生产者就会被阻塞；因为有其他9个消费者的消费sequence处于0状态，
26         // 此时生产者不能超过最慢消费者sequence，所以生产者不得不阻塞。
27         executor.execute(processor);
28     }
29
30     return ringBuffer;
31 }
```

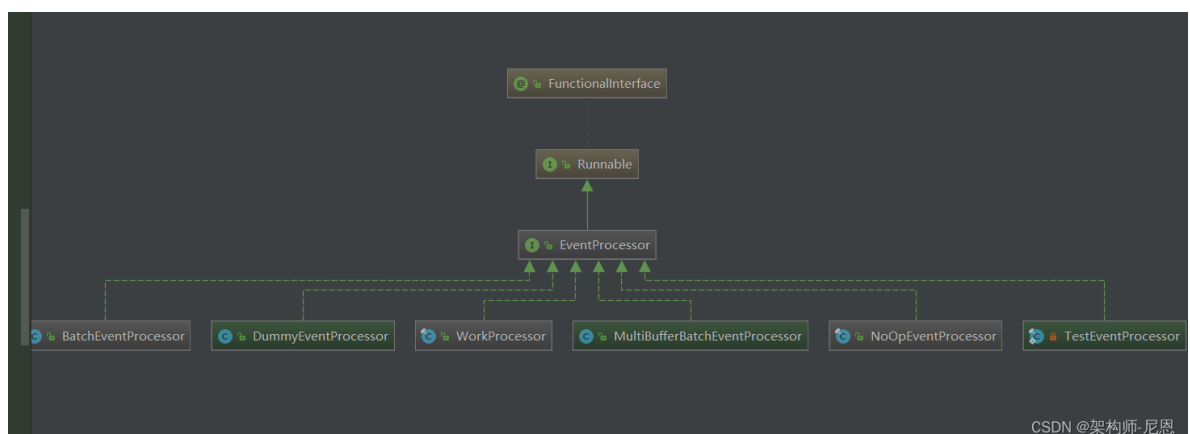
CSDN @架构师-尼恩

消费者处理器

Disruptor的消费者依赖EventProcessor 事件处理器。

handler和processer都可以翻译为“处理器”，但是process侧重于 处理执行，实际执行，processer与cpu有关系，一个processer事件处理器关联一个执行线程而handle侧重于 业务处理器，表示用户逻辑的处理，process表示 handler 的执行过程handle和process 的关系，类似于 程序 与进程的关系

消费者处理器类型比较多，uml图如下：



CSDN @架构师-尼恩

主要的消费者处理器类型，两种：

- BatchEventProcessor 单线程批处理消费者，同一批次添加的消费者，会消费每一个event
- WorkProcessor 消费者池，同一批次添加的消费者，每个event只会被其中一个processer 消费。

WorkProcessor 通过 WorkerPool 进行管理

BatchEventProcessor单线程批处理事件

在使用BatchEventProcessor时，通过Disruptor#handleEventsWith方法可以获取一个EventHandlerGroup，再通过EventHandlerGroup的and和then方法可以构建一个复杂的消费者链。

事件消费者组EventHandlerGroup

EventHandlerGroup表示一组事件消费者，内部持有了Disruptor类实例disruptor，其大部分功能都是通过调用disruptor实现，其实可以算是Disruptor这个辅助类的一部分。

EventHandlerGroup.java

设置批处理程序以使用环形缓冲区中的事件。这些处理程序仅在此组中的每个{@link EventProcessor}处理完事件后处理事件。

该方法通常用作链的一部分。例如，如果处理程序A必须在处理程序B dw.handleEventsWith (A) 之前处理事件。那么 (B)

@param处理将处理事件的批处理程序。

@return {@link EventHandlerGroup}，可用于在创建的事件处理器上设置事件处理器障碍。

```
95     @SafeVarargs
96     public final EventHandlerGroup<T> then(final EventHandler<? super T>... handlers)
97     {
98         return handleEventsWith(handlers);
99     }
```

设置批处理程序以处理来自环形缓冲区的事件。

这些处理程序仅在此组中的每个{@link EventProcessor}处理完事件后处理事件。

该方法通常用作链的一部分。例如，如果A必须在Bdw.after (A) .handleEventsWith (B) 之前处理事件

@param处理将处理事件的批处理程序。

@return {@link EventHandlerGroup}，可用于在创建的事件处理器上设置事件处理器障碍。

```
148     @SafeVarargs
149     public final EventHandlerGroup<T> handleEventsWith(final EventHandler<? super T>... handlers)
150     {
151         return disruptor.createEventProcessors(sequences, handlers);
152     }
```

```
// 由EventHandlerGroup调用时，barrierSequences是EventHandlerGroup实例的序列，
// 也就是上一个事件处理器组的序列，作为当前事件处理的门控，防止后边的消费链超前
// 如果第一次调用handleEventsWith，则barrierSequences为空
EventHandlerGroup<T> createEventProcessors(final Sequence[] barrierSequences,
                                           final EventHandler<? super T>[]
                                           eventHandlers) {
    checkNotStarted();
```

```

        // 对应此事件处理器组的序列组
        final Sequence[] processorSequences = new
Sequence[eventHandlers.length];
        final SequenceBarrier barrier = ringBuffer.newBarrier(barrierSequences);

        for (int i = 0, eventHandlersLength = eventHandlers.length; i <
eventHandlersLength; i++) {
            final EventHandler<? super T> eventHandler = eventHandlers[i];

            final BatchEventProcessor<T> batchEventProcessor =
                new BatchEventProcessor<(ringBuffer, barrier, eventHandler);

            if (exceptionHandler != null) {
                batchEventProcessor.setExceptionHandler(exceptionHandler);
            }

            consumerRepository.add(batchEventProcessor, eventHandler, barrier);
            processorSequences[i] = batchEventProcessor.getSequence();
        }
        // 每次添加完事件处理器后，更新门控序列，以便后续调用链的添加
        // 所谓门控，是指后续消费链的消费，不能超过前边
        updateGatingSequencesForNextInChain(barrierSequences,
processorSequences);

        return new EventHandlerGroup<(this, consumerRepository,
processorSequences);
    }

```

```

// 为消费链下一组消费者，更新门控序列
// barrierSequences是上一组事件处理器组的序列（如果本次是第一次，则为空数组），本组不能超过上
组序列值
// processorSequences是本次要设置的事件处理器组的序列
private void updateGatingSequencesForNextInChain(final Sequence[]
barrierSequences, final Sequence[] processorSequences)
{
    if (processorSequences.length > 0)
    {
        // 将本组序列添加到Sequencer中的gatingSequences中
        ringBuffer.addGatingSequences(processorSequences);
        // 将上组序列从Sequencer中的gatingSequences中，gatingSequences一直保存消
费链末端消费者的序列组
        for (final Sequence barrierSequence : barrierSequences)
        {
            ringBuffer.removeGatingSequence(barrierSequence);
        }
        // 取消标记上一组消费者为消费链末端

        consumerRepository.unMarkEventProcessorsAsEndOfChain(barrierSequences);
    }
}

```

可以看到，使用BatchEventProcessor构建消费者链时的逻辑都在 createEventProcessors 方法中。

ConsumerRepository 类主要保存消费者的各种关系，如通过EventHandler引用获取EventProcessorInfo信息，通过Sequence获取ConsumerInfo信息等。

因为要使用引用做key，所以数据结构使用IdentityHashMap。

IdentityHashMap和HashMap最大的不同，就是使用==而不是equals比较key。

这个createEventProcessors方法接收两个参数，barrierSequences表示当前消费者组的屏障序列数组，如果当前消费者组是第一组，则取一个空的序列数组；否则，barrierSequences就是上一组消费者组的序列数组。

createEventProcessors方法的另一个参数eventHandlers，这个参数是代表事件消费逻辑的EventHandler数组。

Disruptor为每个EventHandler实现类都创建了一个对应的BatchEventProcessor。

在构建BatchEventProcessor时需要以下传入三个构造参数：dataProvider是数据存储结构如RingBuffer；sequenceBarrier用于跟踪生产者游标，协调数据处理；

eventHandler是用户实现的事件处理器，也就是实际的消费者。

注意，Disruptor并非为每个BatchEventProcessor都创建一个新的SequenceBarrier，而是每个消费者组共用一个SequenceBarrier。

BatchEventProcessor定义，请参见源码仓库。

至于为什么要叫做BatchEventProcessor，可以看看在run()方法里每次waitFor获取的availableSequence是当前能够使用的最大值，然后再循环处理这些数据。

这样当消费者有瞬时抖动，导致暂时落后生产者时，可在下一次循环中，批量处理所有落后的事件。

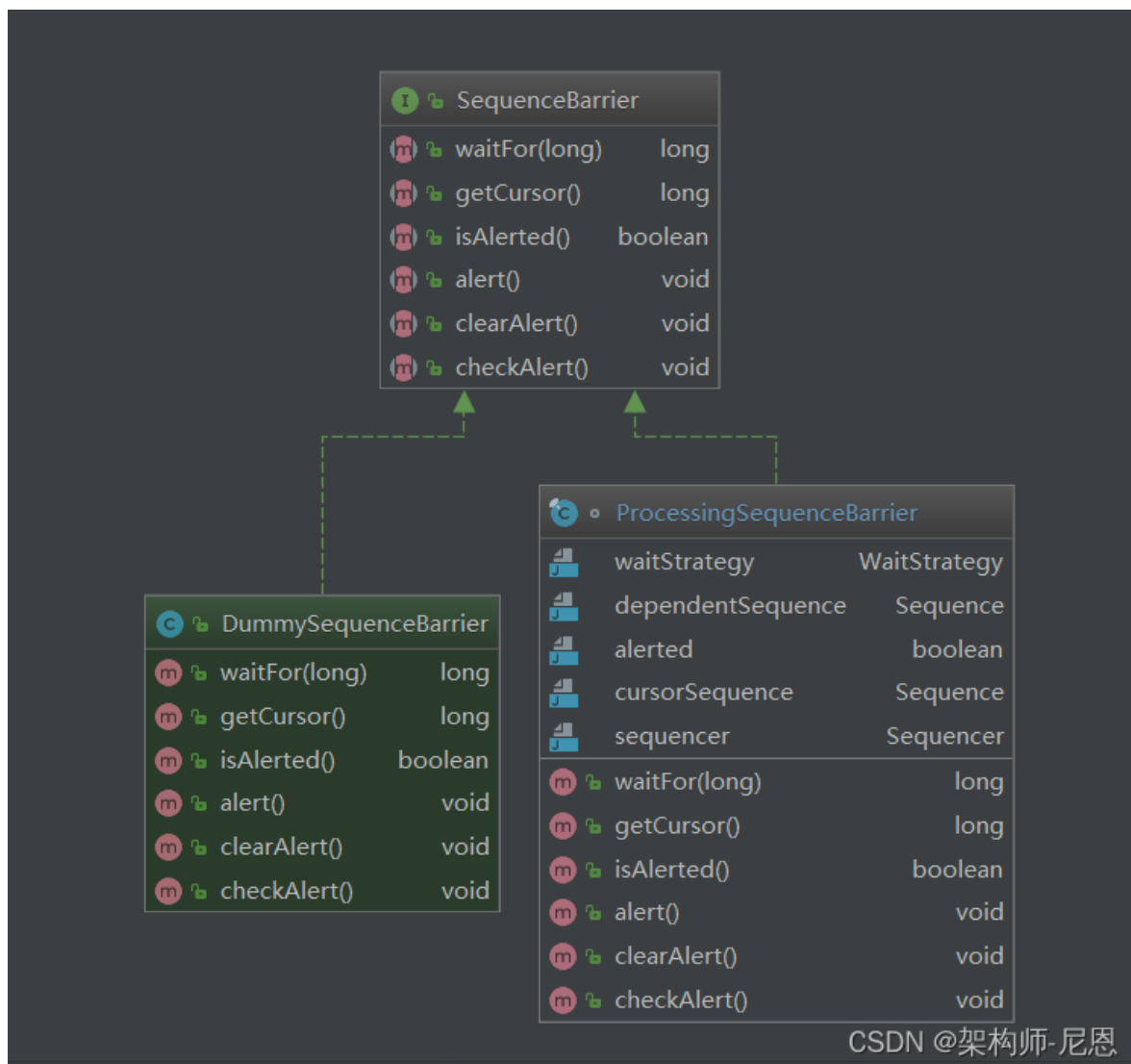
可以看出：

BatchEventProcessor可以处理超时，可以处理中断，可以通过用户实现的异常处理类处理异常，同时，发生异常之后再次启动，不会漏消费，也不会重复消费。

SequenceBarrier协调屏障

SequenceBarrier：一个协调屏障，

SequenceBarrier用来跟踪发布者(publisher)的游标(cursor)和事件处理器(EventProcessor)的序列号(sequence)。



CSDN @架构师-尼恩

disruptor管理两种依赖关系

disruptor需要管理两种依赖关系：

- 生产者与消费者之间的依赖关系
- 以及消费者与消费者之间的依赖关系

消费者与消费者之间的依赖关系 ,使用的 是sequenceBarrier 的 dependentSequence 来管理

消费者对生产者之间的依赖关系 ,使用的 是sequenceBarrier 的 sequencer 来管理

生产者对最慢+末端消费者直接的依赖关系，使用门禁序号 gatingSequence 来管理

消除锁和CAS操作

- **Disruptor**中,通过联合使用**SequenceBarrier**和**Sequence**, 协调和管理消费者和生产者之间的处理关系,避免了锁和**CAS**操作
- **Disruptor**中的各个消费者和生产者持有自己的序号**Sequence**,

序号**Sequence**需要满足以下条件:

- **条件一:** 消费者的序号**Sequence**的数值必须小于生产者的序号**Sequence**的数值

- **条件二:** 消费者的序号**Sequence**的数值必须小于依赖关系中前置的消费者的序号**Sequence**的数值
- **条件三:** 生产者的序号**Sequence**的数值不能大于消费者正在消费的序号**Sequence**的数值,防止生产者速度过快,将还没有来得及消费的事件消息覆盖
- **条件一和条件二**在**SequenceBarrier**中的**waitFor()** 方法中实现:
- 条件三是针对生产者建立的**SequenceBarrier**,逻辑判定发生在生产者从**RingBuffer**获取下一个可用的entry时,**RingBuffer**会将获取下一个可用的entry委托给**Sequencer**处理:

SequenceBarrier的几个方法

`long waitFor(long sequence) throws AlertException, InterruptedException, TimeoutException;`
等待给定的序列号可用, 用来消费。

`long getCursor();`

获取当前能读取到的游标(cursor)值。

`boolean isAlerted();`

barrier当前的状态是否是警报(alert)状态。

`void alert();`

提醒EventProcessor, 一个状态发生了变化, 直到清除之前, 一直处于这种状态下。

`void clearAlert();`

清除当前的警报状态。

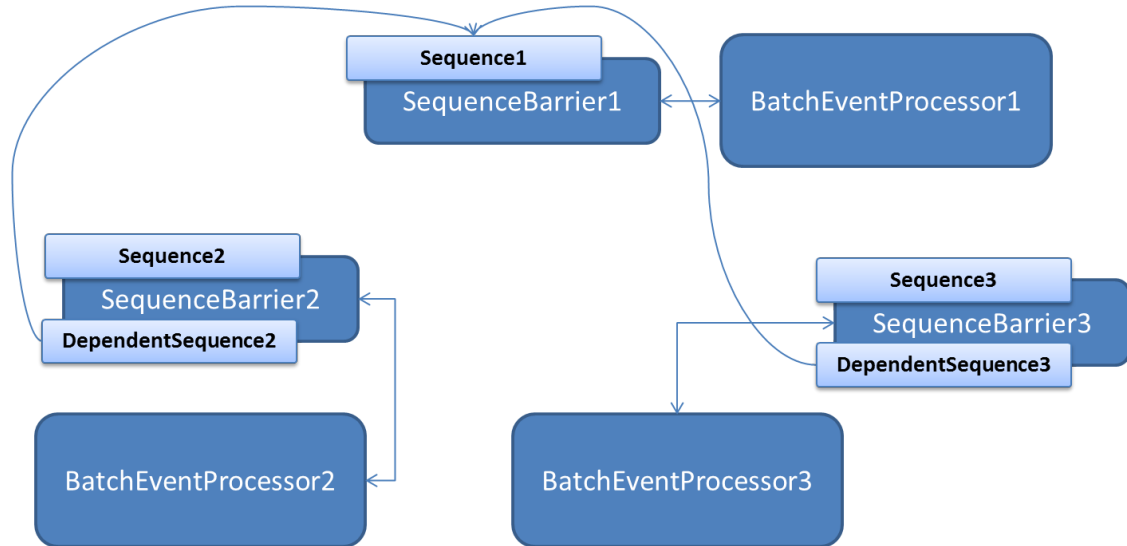
`void checkAlert() throws AlertException;`

检查是否提出了警报, 如果提出了, 就抛出异常。

SequenceBarrier进行依赖消费

1. 不同的BatchEventProcessor之间通过SequenceBarrier进行依赖消费。

原理如下图所示：



假设我们有三个消费者BatchEventProcessor1, BatchEventProcessor2, BatchEventProcessor3.

1需要先于2和3消费, 那么构建BatchEventProcessor和SequenceBarrier时,

我们需要让BatchEventProcessor2和BatchEventProcessor3的SequenceBarrier的dependentSequence中加入SequenceBarrier1的sequence。

其实这里2和3共用一个SequenceBarrier就行。

ProcessingSequenceBarrier

接下来看下它的实现类 - ProcessingSequenceBarrier

SequenceBarrier只有一个重要的实现类, 就是ProcessingSequenceBarrier。

ProcessingSequenceBarrier有以下几个重要的属性:

- 生产者Sequencer,
- 消费定位cursorSequence,
- 等待策略waitStrategy,
- 还有一组依赖sequence: dependentSequence

```

final class ProcessingSequenceBarrier implements SequenceBarrier {
    private final WaitStrategy waitStrategy; //等待策略waitStrategy

    private final Sequence dependentSequence; // 一组依赖sequence
    private volatile boolean alerted = false;
    private final Sequence cursorSequence; //消费定位cursorSequence
    private final Sequencer sequencer; // 生产者Sequencer

    ProcessingSequenceBarrier(
        final Sequencer sequencer,
        final WaitStrategy waitStrategy,
        final Sequence cursorSequence,
        final Sequence[] dependentSequences) {
        this.sequencer = sequencer;
        this.waitStrategy = waitStrategy;
        // cursorSequence其实持有的是AbstractSequencer的成员变量cursor实例的引用
        this.cursorSequence = cursorSequence;
        // 如果消费者前面依赖的其他消费者链长度为0，其实就是只有一个消费者的情况下，
        // 那么dependentSequence其实就是生产者的cursor光标
        if (0 == dependentSequences.length) {
            dependentSequence = cursorSequence;
        }
        // 如果消费者前面还有被依赖的消费者，那么dependentSequence就是前面消费者的sequence
        // 如果是指定执行顺序链的会执行到这里，
        // 比如 disruptor.after(eventHandler1).handleEventsWith(eventHandler2);
        else {
            dependentSequence = new FixedSequenceGroup(dependentSequences);
        }
    }
}

```

CSDN @架构师-尼恩

(上图是从它的构造器中截取的一部分)

可以看出如果dependentSequence的长度是0，就将cursorSequence指向它，即两者有着相同的引用。

否则，通过FixedSequenceGroup来创建它，即与cursorSequence之间，两者独立存在。

```

public FixedSequenceGroup(Sequence[] sequences)
{
    this.sequences = Arrays.copyOf(sequences, sequences.length);
}

```

然后重点看下如下几个实现方法：

```

@Override
public void alert()
{
    alerted = true;
    waitStrategy.signalAllWhenBlocking();
}

```

```

@Override
public long getCursor() { return dependentSequence.get(); }

```

```

@Override
public long waitFor(final long sequence)
    throws AlertException, InterruptedException, TimeoutException
{
    checkAlert();

    long availableSequence = waitStrategy.waitFor(sequence, cursorSequence, dependentSequence, barrier: this);

    if (availableSequence < sequence)
    {
        return availableSequence;
    }

    return sequencer.getHighestPublishedSequence(sequence, availableSequence);
}

```

https://blog.csdn.net/qq_34561892

RingBuffer预分配内存

RingBuffer使用数组Object[] entries来存储元素:

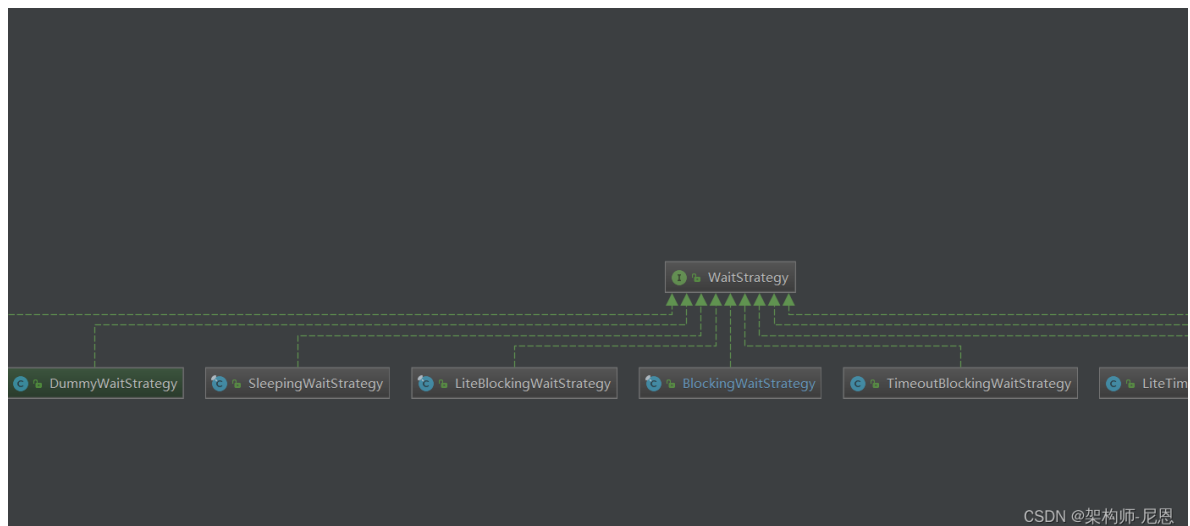
- 初始化RingBuffer时,会将所有数组元素entries的指定为特定的事件Event参数,此时Event中的detail属性为null
- 生产者向RingBuffer写入消息时 ,RingBuffer不是直接将数组元素entries指向Event对象,而是先获取Event对象,更改Event对象中的detail属性
- 消费者在消费时,也是从RingBuffer中读取Event, 读取Event对象中的detail属性
- 由此可见,在生产和消费过程中 ,RingBuffer中的数组元素entries没有发生任何变化,没有产生临时对象,数组中的元素一直存活,直到RingBuffer消亡

```
private void fill(EventFactory<E> eventFactory) {  
    for (int i = 0; i < bufferSize; i++) {  
        // 使用工厂方法初始化数组中的entries元素  
        entries[BUFFER_PAD + i] = eventFactory.newInstance();  
    }  
}
```

通过以上方式,可以最小化JVM中的垃圾回收GC的频率,提升性能

Disruptor的等待策略

uml图



Disruptor默认的等待策略是BlockingWaitStrategy。

这个策略的内部适用一个锁和条件变量来控制线程的执行和等待（Java基本的同步方法）。

BlockingWaitStrategy是最慢的等待策略，但也是CPU使用率最低和最稳定的选项。然而，可以根据不同的部署环境调整选项以提高性能。

SleepingWaitStrategy

和BlockingWaitStrategy一样，SleepingWaitStrategy的CPU使用率也比较低。

它的方式是循环等待并且在循环中间调用LockSupport.parkNanos(1)来睡眠，（在Linux系统上面睡眠时间60μs）。

然而，它的优点在于生产线程只需要计数，而不执行任何指令。并且没有条件变量的消耗。

但是，事件对象从生产者到消费者传递的延迟变大了。

SleepingWaitStrategy最好用在不需要低延迟，而且事件发布对于生产者的影响比较小的情况下。比如异步日志功能。

源码

```
/*
 * Copyright 2011 LMAX Ltd.
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 * http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */
package com.lmax.disruptor;

import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

import com.lmax.disruptor.util.ThreadHints;

/**
 * Blocking strategy that uses a lock and condition variable for {@link
 * EventProcessor}s waiting on a barrier.
 *
 * <p>
 * This strategy can be used when throughput and low-latency are not as
 * important as CPU resource.
 *
 */
public final class BlockingWaitStrategy implements WaitStrategy
{
    private final Lock lock = new ReentrantLock();
    private final Condition processorNotifyCondition = lock.newCondition();

    /**
```

```

*
* @param sequence          to be waited on. 消费者想要消费的下一个序号
* @param cursorSequence    当前ringbuffer最大的生产者序号?
* @param dependentSequence on which to wait.
* @param barrier           the processor is waiting on.
* @return
* @throws AlertException
* @throws InterruptedException
*/
@Override
public long waitFor(long sequence, Sequence cursorSequence, Sequence
dependentSequence, SequenceBarrier barrier)
    throws AlertException, InterruptedException
{
    long availableSequence;
    // cursorSequence:生产者的序号
    // 第一重条件判断: 如果消费者消费速度, 大于生产者生产速度 (即消费者要消费的下一个数据
    已经大于生产者生产的数据时),
    // 那么消费者等待一下
    if (cursorSequence.get() < sequence)
    {
        lock.lock();
        try
        {
            while (cursorSequence.get() < sequence)
            {
                barrier.checkAlert();
                processorNotifyCondition.await();
            }
        }
        finally
        {
            lock.unlock();
        }
    }
    // 第一重条件判断: 自旋等待
    // 即当前消费者线程要消费的下一个sequence, 大于其前面执行链路 (若有依赖关系) 的任何一
    个消费者最小sequence (dependentSequence.get()),
    // 那么这个消费者要自旋等待,
    // 直到前面执行链路 (若有依赖关系) 的任何一个消费者最小
    sequence (dependentSequence.get()) 已经大于等于当前消费者的sequence时,
    // 说明前面执行链路的消费者已经消费完了
    while ((availableSequence = dependentSequence.get()) < sequence)
    {
        barrier.checkAlert();
        ThreadHints.onSpinwait();
    }

    return availableSequence;
}

/**
* 如果生产者新生产一个元素, 那么唤醒所有消费者
*/
@Override
public void signalAllWhenBlocking()
{
    lock.lock();

```



```

        try
        {
            processorNotifyCondition.signalAll();
        }
        finally
        {
            lock.unlock();
        }
    }

    @Override
    public String toString()
    {
        return "BlockingWaitStrategy{" +
            "processorNotifyCondition=" + processorNotifyCondition +
            '}';
    }
}

```

YieldingWaitStrategy

YieldingWaitStrategy是可以被用在低延迟系统中的两个策略之一，这种策略在减低系统延迟的同时也会增加CPU运算量。

YieldingWaitStrategy策略会循环等待sequence增加到合适的值。

循环中调用Thread.yield()允许其他准备好的线程执行。

如果需要高性能而且事件消费者线程比逻辑内核少的时候，推荐使用YieldingWaitStrategy策略。

例如：在开启超线程的时候。

源码

```

/*
 * Copyright 2011 LMAX Ltd.
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 * http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */
package com.lmax.disruptor;

/**

```

```

    * Yielding strategy that uses a Thread.yield() for {@link
    com.lmax.disruptor.EventProcessor}s waiting on a barrier
    * after an initially spinning.
    * <p>
    * This strategy will use 100% CPU, but will more readily give up the CPU than a
    busy spin strategy if other threads
    * require CPU resource.
    */
    public final class YieldingWaitStrategy implements WaitStrategy
    {
        private static final int SPIN_TRIES = 100;

        @Override
        public long waitFor(
            final long sequence, Sequence cursor, final Sequence
            dependentSequence, final SequenceBarrier barrier)
            throws AlertException, InterruptedException
        {
            long availableSequence;
            int counter = SPIN_TRIES;
            // 如果消费者需要消费的下一个序号超过了生产者已生产数据的最大序号，
            // 那么消费者需要等待，否则返回生产者已生产数据的最大序号给消费者消费即可
            while ((availableSequence = dependentSequence.get()) < sequence)
            {
                counter = applyWaitMethod(barrier, counter);
            }

            return availableSequence;
        }

        @Override
        public void signalAllWhenBlocking()
        {
        }

        private int applyWaitMethod(final SequenceBarrier barrier, int counter)
            throws AlertException
        {
            barrier.checkAlert();

            if (0 == counter)
            {
                Thread.yield();
            }
            else
            {
                --counter;
            }

            return counter;
        }
    }
}

```

BusySpinWaitStrategy

BusySpinWaitStrategy是性能最高的等待策略，同时也是对部署环境要求最高的策略。

这个性能最好用在事件处理线程比物理内核数目还要小的时候。例如：在禁用超线程技术的时候。

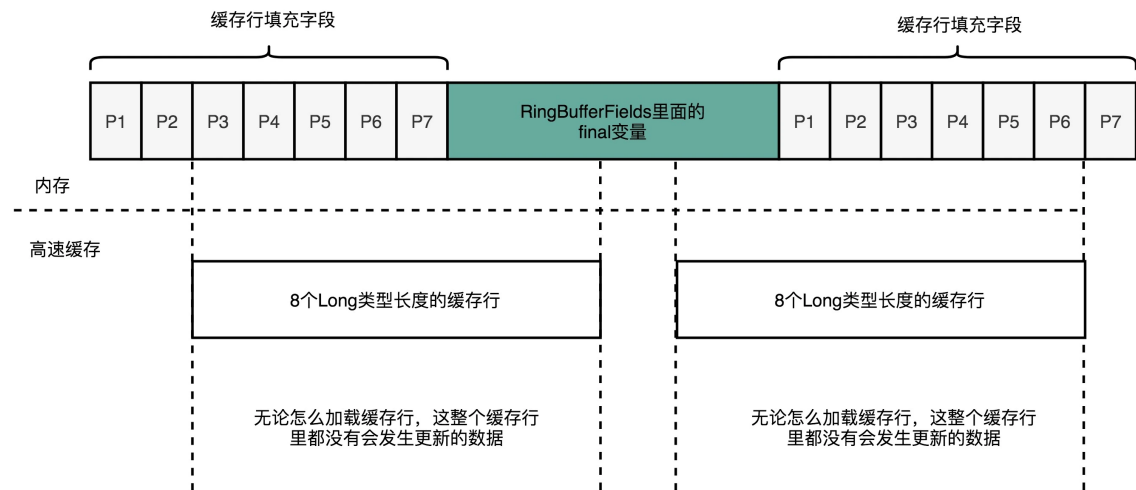
Disruptor 的缓存行填充

RingBuffer 的缓存行填充

Disruptor RingBuffer（环形缓冲区）定义了RingBufferFields类，里面有indexMask和其他几个变量存放RingBuffer的内部状态信息。

```
24  ↓ abstract class RingBufferPad
25      {
26          protected long p1, p2, p3, p4, p5, p6, p7;
27      }
```

```
29  ↓ abstract class RingBufferFields<E> extends RingBufferPad
30      {
31          private static final int BUFFER_PAD;
32          private static final long REF_ARRAY_BASE;
33          private static final int REF_ELEMENT_SHIFT;
34          private static final Unsafe UNSAFE = Util.getUnsafe();
35
36          static
37          {...}
38
39          private final long indexMask;
40          private final Object[] entries;
41          protected final int bufferSize;
42          protected final Sequencer sequencer;
43
44          public final class RingBuffer<E> extends RingBufferFields<E> implements Cursor, EventSequencer<E>, EventSink<E>
45          {
46              public static final long INITIAL_CURSOR_VALUE = Sequence.INITIAL_VALUE;
47              protected long p1, p2, p3, p4, p5, p6, p7;
```



对此，Disruptor利用了缓存行填充，在 RingBufferFields里面定义的变量的前后，分别定义了7个long类型的变量：

- 前面7个来自继承的 RingBufferPad 类
- 后面7个直接定义在 RingBuffer 类

这14个变量无任何实际用途。我们既不读他们，也不写他们。而RingBufferFields里面定义的这些变量都是**final**，第一次写入后就不会再修改。

所以，一旦它被加载到CPU Cache后，只要被频繁读取访问，就不会再被换出Cache。这意味着，对于该值的读取速度，会一直是CPU Cache的访问速度，而非内存的访问速度。

Sequence是如何消除伪共享的

从Sequence的父类Value可以看到，真正使用到的变量是Value类的value，它的前后空间都由8个long型的变量填补了，对于一个大小为64字节的缓存行，它刚好被填补满（一个long型变量value，8个字节加上前/后个7long型变量填补， $7 \times 8 = 56$ ， $56 + 8 = 64$ 字节）。

这样做每次把变量value读进高速缓存中时，都能把缓存行填满，保证每次处理数据时都不会与其他变量发生冲突。

当然，对于大小为64个字节的缓存行来说，如果缓存行大小大于64个字节，那么还是会出现伪共享问题，但是毕竟**非64字节的Cache Line**并不是当前的主流。

参考文献

https://blog.51cto.com/u_11440114/5184594

<https://blog.csdn.net/hxg117/article/details/78064632>

<http://openjdk.java.net/projects/jdk8/features>

<http://beautynbits.blogspot.co.uk/2012/11/the-end-for-false-sharing-in-java.html>

<http://openjdk.java.net/jeps/142>

<http://mechanical-sympathy.blogspot.co.uk/2011/08/false-sharing-java-7.html>

<http://stackoverflow.com/questions/19892322/when-will-jvm-use-intrinsics>

https://blogs.oracle.com/dave/entry/java_contented_annotation_to_help

<https://www.jianshu.com/p/b38ffa33d64d>

<https://blog.csdn.net/MrYushiwen/article/details/123171635>

<https://blog.csdn.net/everyok/article/details/88889057>

硬核推荐：尼恩Java硬核架构班

又名疯狂创客圈社群 VIP

详情：<https://www.cnblogs.com/crazymakercircle/p/9904544.html>



尼恩java
硬核架构班

已经发布

- 《高能性能RPC的基础实操之：从0到1开始IM撸一个IM》
- 《分布式高能性能RPC的基础实操之：千万级用户分布式IM实操-含简历指导》
- 《亿级用户超高并发秒杀实操-含简历指导》
亮点：助力小伙伴搞定70W年薪，N个涨薪50%，**2023春招面试涨薪神器**
- 《横扫全网，工业级elasticsearch底层原理与高并发、高可用架构实操》
亮点：40岁老架构师细致解读，处处透着分布式、高性能中间件的原理和精髓
- 《第1部曲：超级底层：葵花宝典（高性能秘籍）__架构师视角解读OS操作系统》
亮点：大制作解读OS操作系统，并揭秘mmap、pagecache、zerocopy等底层的底层原理
2023春招面试涨薪大神器
- 《Rocketmq视频第2部曲：横扫全网工业级 rocketmq 高可用（HA）底层原理和实操》
亮点：起底式、绞杀式解读 rocketmq如何保障消息的可靠性？
- 《Rocketmq视频第3部曲：超级内功篇、横扫全网 rocketmq 源码学习以及3高架构模式解读》
亮点：大制作解读 Rocketmq源码以及3高架构模式，助力大家内力猛增
- 《Rocketmq视频第4部曲：10Wqps消息推送中台架构、设计、编码、测试实操》
亮点：Netty实操、分库分表实操、Rocketmq工业级使用实操
- 《架构师内功篇：横扫全网 netty 高性能、高并发架构 底层原理、源码学习》
- 《架构师实操篇：redis cluster 工业级高可用实操》
- 《架构师实操篇：100W级别QPS日志平台实操》
- 《彻底穿透：skywalking 源码(代表链路跟踪)+Java agent+bytebuddy 探针》

规划中

《架构师实操篇：基于netty 手写 rpc 框架- 参考 dubbo、seata rpc框架》

《架构师实操篇：go语言学习，以及基于 go 手写 rpc 框架》

《架构师实操篇：千万级任务调度平台 架构与实操- 基于尼恩17年的亿级搜索项目》

《架构师实操篇：工业级 亿级文档搜索 平台 架构与实操- 基于尼恩17年的亿级搜索项目》

特色

会员制

提供技术方向指导，
职业生涯指导，少躺坑，少弯路

简历指导

这个很重要，
对于挪窝涨薪来说

实操性

以上项目，都是老架构师
在生产上实操过的项目

非水货

40岁老架构师，不是水货架构师
《Java高并发三部曲》为证

架构班（社群 VIP）的起源：

最初的视频，主要是给读者加餐。很多的读者，需要一些高质量的实操、理论视频，所以，我就围绕书，和底层，做了几个实操、理论视频，然后效果还不错，后面就做成迭代模式了。

架构班（社群 VIP）的功能：

提供高质量实操项目整刀真枪的架构指导、快速提升大家的：

- 开发水平
- 设计水平
- 架构水平

弥补业务中 CRUD 开发短板，帮助大家尽早脱离具备 3 高能力，掌握：

- 高性能
- 高并发
- 高可用

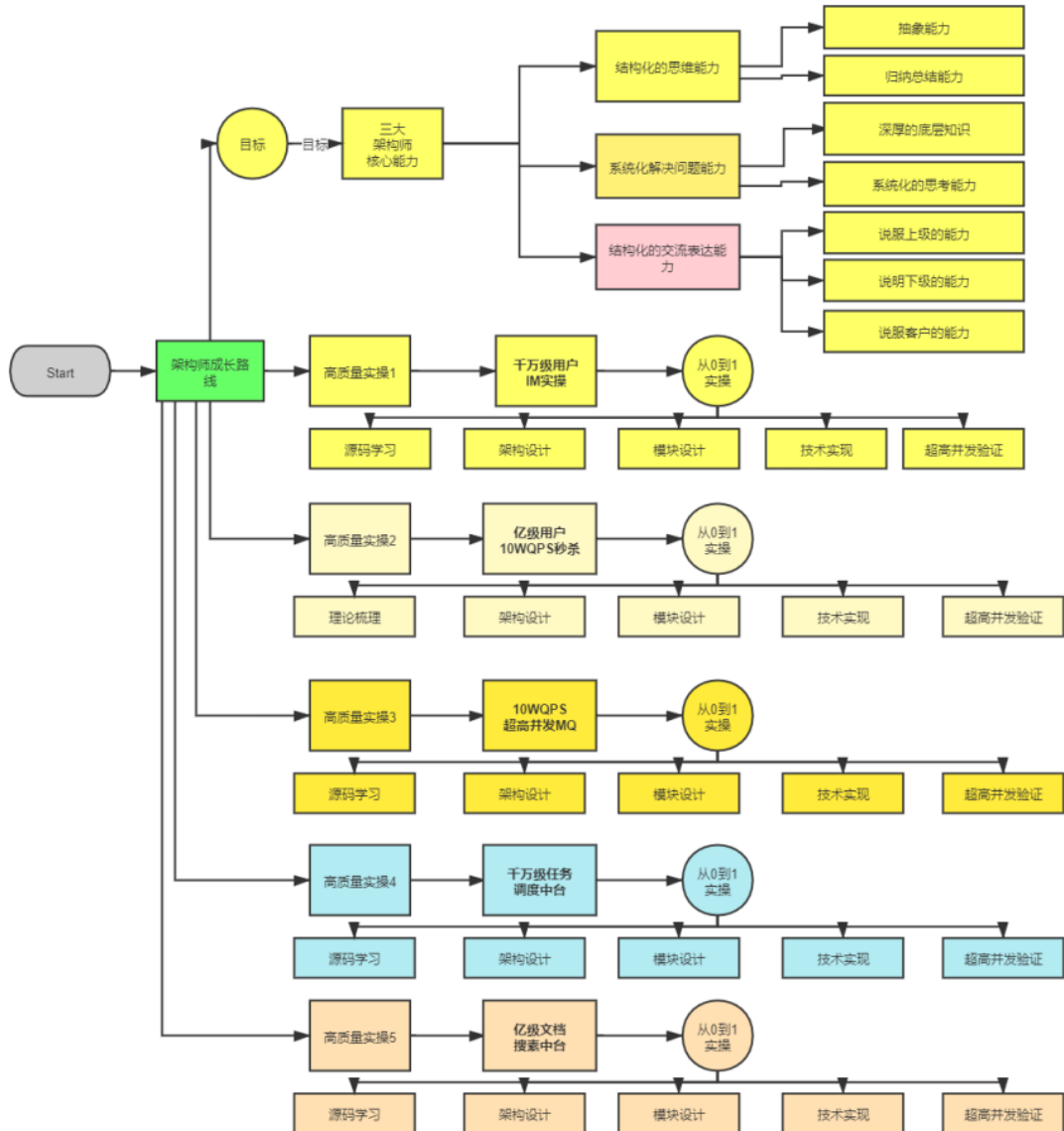
作为一个高质量的架构师成长、人脉社群，把所有的卷王聚焦起来，一起卷：

- 卷高并发实操
- 卷底层原理
- 卷架构理论、架构哲学
- 最终成为顶级架构师，实现人生理想，走向人生巅峰

架构班（社群 VIP）的目的：

- 高质量的实操，大大提升简历的含金量，吸引力，增强面试的召唤率
- 为大家提供九阳真经、葵花宝典，快速提升水平
- 进大厂、拿高薪
- 一路陪伴，提供助学视频和指导，辅导大家成为架构师
- 自学为主，和其他卷王一起，卷高并发实操，卷底层原理、卷大厂面试题，争取狠卷 3 月成高手，狠卷 3 年成为顶级架构师

N 个超高并发实操项目：简历压轴、个顶个精彩



【样章】第 17 章:横扫全网Rocketmq 视频第 2 部曲: 工业级 rocketmq 高可用(HA) 底层原理和实操

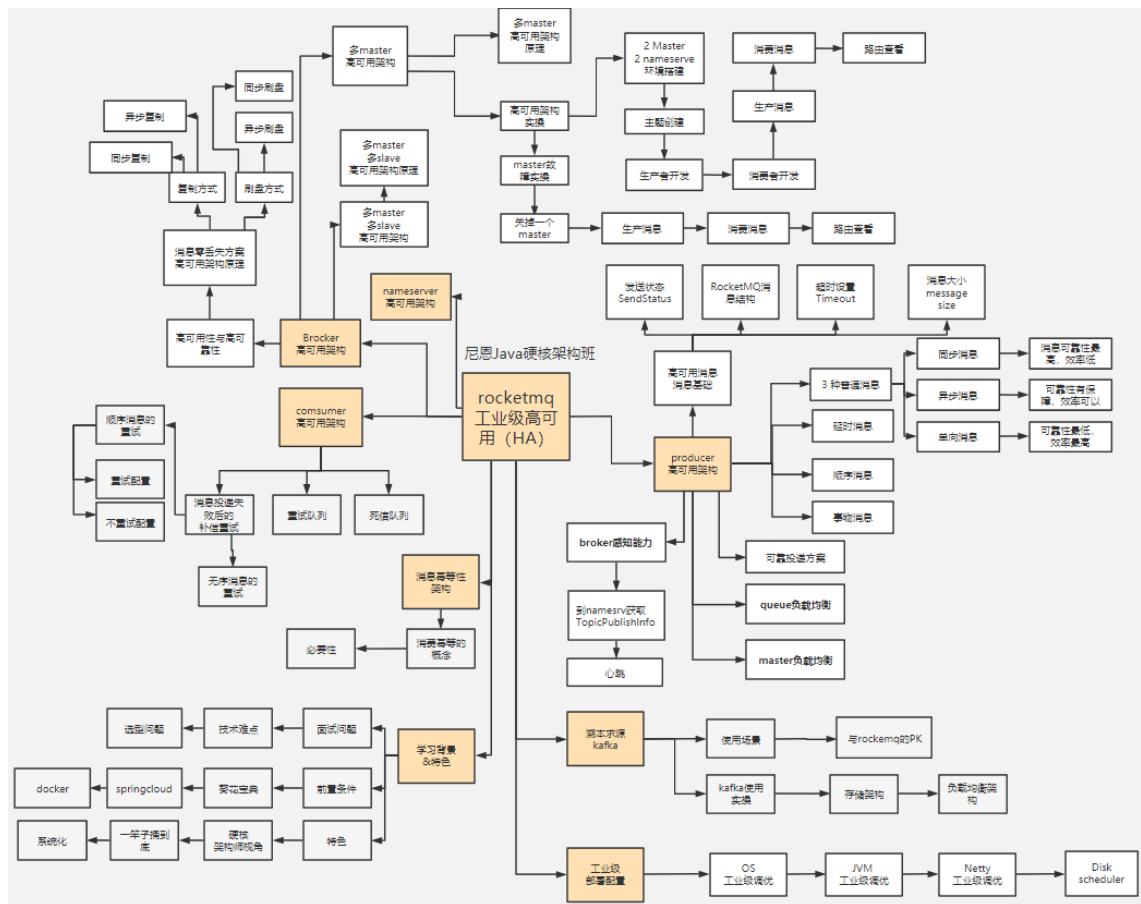
工业级 rocketmq 高可用底层原理, 包含: 消息消费、同步消息、异步消息、单向消息等不同消息的底层原理和源码实现; 消息队列非常底层的主从复制、高可用、同步刷盘、异步刷盘等底层原理。

工业级 rocketmq 高可用底层原理和搭建实操, 包含: 高可用集群的搭建。

解决以下难题:

- 1、技术难题: RocketMQ 如何最大限度的保证消息不丢失的呢? RocketMQ 消息如何做到高可靠投递?
- 2、技术难题: 基于消息的分布式事务, 核心原理不理解
- 3、选型难题: kafka or rocketmq , 该娶谁?

下图链接: <https://www.processon.com/view/6178e8ae0e3e7416bde9da19>



成功案例：2 年翻 3 倍，35 岁卷王成功转型为架构师

详情: <http://topcoder.cloud/forum.php?mod=forumdisplay&fid=43&page=1>

最新 最后发表 热门 精华

成功案例: [1057号卷王] 3年小伙拿到外企offer, 薪酬涨了200%

1 卷王1号 超级版主 前天 17:41

成功案例: [645号卷王] 4年经验卷王逆袭, 被毕业后, 反涨24W

1 卷王1号 超级版主 2022-9-21

成功案例: [878号卷王] 小伙8年经验, 年薪60W

1 卷王1号 超级版主 2022-8-13

年薪70W案例: 通过尼恩的指导, 小伙伴年薪从40W涨到70W

1 卷王1号 超级版主 2022-2-11

成功案例: [493号卷王] 5年小伙拿满意offer, 就业寒冬季逆涨30%

1 卷王1号 超级版主 前天 17:43

成功案例: [250号卷王] 就业极寒时代, 收offer 涨25%

1 卷王1号 超级版主 前天 17:38

成功案例: [612号卷王] 就业极寒时代, 从外包到自研

1 卷王1号 超级版主 前天 17:15

成功案例: [913号卷王] 热烈祝贺6年经验卷王, 年薪40W

1 卷王1号 超级版主 2022-9-21

成功案例: [959号卷王] 4年经验卷王, 喜获百度、Boss直聘等N个优质offer, 最高涨100%

1 卷王1号 超级版主 2022-9-21

成功案例: [529号卷王] 5年经验卷王喜收2大offer, 最高涨5K

1 卷王1号 超级版主 2022-9-21

成功案例: [811号卷王] 热烈祝贺7年经验卷王, 薪酬涨30%

1 卷王1号 超级版主 2022-9-21

成功案例: [287号卷王] 不惧大寒潮, 卷王逆市收4 offer, 涨30%, 可喜可贺

1 卷王1号 超级版主 2022-5-30

成功案例: [1002号卷王] 5月份“被毕业”, 改简历后, 斩获顶级央企Offer, 涨薪7000+

1 卷王1号 超级版主 2022-7-5

成功案例: [7号卷王] 热烈祝贺小伙伴涨薪120%

1 卷王1号 超级版主 2022-8-13

成功案例: [134号卷王] 大三小伙卷1年, 斩获顶级央企Offer, 成功逆袭

1 卷王1号 超级版主 2022-7-6

成功案例: [1008号卷王] 5年经验卷王收42W offer, 月涨8000, 可喜可贺

1 卷王1号 超级版主 2022-5-30

成功案例: [453号卷王] 非全日制 6年卷王喜提3 offer, 年薪30W, 可喜可贺

1 卷王1号 超级版主 2022-5-21

成功案例: [924号卷王] 6年卷王喜提4 offer, 最高涨薪9000, 可喜可贺

1 卷王1号 超级版主 2022-5-21

成功案例: [15号卷王] 4年卷王入职 微软, 涨薪50%, 可喜可贺

1 卷王1号 超级版主 2022-5-12

成功案例: [527号卷王] 4年卷王喜提2 offer, 涨薪50%, 可喜可贺

1 卷王1号 超级版主 2022-5-13

成功案例: [788号卷王] 3年卷王喜提优质Offer, 涨薪60%

1 卷王1号 超级版主 2022-5-11

成功案例: 热烈祝贺: 非全日制卷王, 喜提2个心仪offer, 面3家过2家

1 卷王1号 超级版主 2022-4-21

成功案例: [693号卷王] 二线城市6年卷王喜提4大优质Offer, 含央企offer, 最高薪酬35W

1 卷王1号 超级版主 2022-4-16

成功案例: [85号卷王] 双非2本小伙, 春招大捷, 喜提9个offer, 最高薪酬近30万

1 卷王1号 超级版主 2022-4-14

成功案例: [741号卷王] 卷王逆袭! 6年小伙从很少面试机会到搞定35K*14薪Offer

1 卷王1号 超级版主 2022-4-12

成功案例: [642号卷王] 热烈祝贺, 6年卷王喜提优质国企offer

1 卷王1号 超级版主 2022-4-7

成功案例: [796号卷王] 热烈祝贺, 36岁卷王喜提52万优质offer

1 卷王1号 超级版主 2022-3-25

☐ 成功案例: [15号卷王] 小伙卷1年, 涨薪9K+, 喜收ebay等多个优质offer

① 卷王1号 超级版主 2022-3-24

☐ 成功案例: [821号卷王] 小伙狠卷3个月, 喜提10多个offer

① 卷王1号 超级版主 2022-3-21

☐ 成功案例: [736号卷王] 3年半经验收22k offer, 但是小伙志存高远, 冲击25k+

① 卷王1号 超级版主 2022-3-20

☐ 成功案例: 热烈祝贺1群小卷王offer拿到手软, 甚至拒了阿里offer

① 卷王1号 超级版主 2022-3-16

☐ 简历案例: 简历一改, 腾讯的邀请就来了! 热烈祝贺, 小伙收到一大堆面试邀请

① 卷王1号 超级版主 2022-3-10

☐ 成功案例: 祝贺我国两大超级卷王, 一个过了阿里HR面, 一个过了阿里2面

① 卷王1号 超级版主 2022-3-10

☐ 成功案例: 小伙伴php转Java, 卷1.5年Java, 涨薪50%, 喜收多个优质offer

① 卷王1号 超级版主 2022-3-10

☐ 成功案例: 4年小伙狠卷半年, 拿到 移动、京东 两大顶级offer

 尼恩 超级版主 2022-3-5

☐ 成功案例: [267号卷王] 助力3年经验卷王, 拿到蜂巢的17k x 14薪的offer

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [143号卷王] 二本院校00后卷神, 毕业没到一年跳到字节, 年薪45W

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [494号卷王] 尼恩分布式事务助力卷王拿到 中信银行offer

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [76号卷王] 2线城市卷王, 狠卷1.5年, 喜收22K offer

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [429号卷王] 小伙伴在社群卷5个月, 涨8k+

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [154号卷王] 双非学校毕业卷王, 连拿 京东到家&滴滴 两个大厂 Offer

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [232号卷王] 涨薪10K, 继续卷向食物链顶端

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: 狠卷1年技术, 喜收 腾讯、阿里、微软三大Offer, 最高年薪56W

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [449号卷王] 应届毕业生卷王喜收 滴滴offer, 年薪33W

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [551号卷王] 小伙伴学完后, 成功进入大厂, 并且推荐自己的朋友加VIP学习

① 卷王1号 超级版主 2022-2-10

☐ 成功案例: [214号卷王] 助力2年经验卷王, 成功拿到17K月薪

① 卷王1号 超级版主 2022-2-10

☐ 成功案例: [92号卷王] 课程实操助力社群小伙伴喜收 喜马拉雅Offer

① 卷王1号 超级版主 2022-2-10

☐ 成功案例: 社群卷王小伙伴成功过了滴滴三面 获滴滴Offer

① 卷王1号 超级版主 2022-2-10

☐ [612号卷王]滴滴小伙伴, 蹲点考察半年, 觉得靠谱后加入 疯狂创客圈

① 卷王1号 超级版主 2022-2-10

☐ 成功案例: [732号卷王] 尼恩助力3年经验卷王收获 京东offer, 年薪35W

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [558号卷王] 2年经验卷王, 喜收 网易和阿里子公司两个优质offer

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [569号卷王] 双非应届生卷王, 喜收字节跳动实习offer

① 卷王1号 超级版主 2022-2-25

☐ 成功案例: [420号卷王] 狠卷1年, 卷王涨薪80%, 涨薪12000元!

① 卷王1号 超级版主 2022-2-25

☐ 成功案例: [76号卷王] 通过尼恩1年半的指导, 专科学历小伙伴从0.8K涨到22K

① 卷王1号 超级版主 2022-2-10

简历优化后的成功涨薪案例（VIP 含免费简历优化）

9年小伙伴拿到年薪90W offer

9月11日改简历 11月29日晒offer

秘诀: 简历指导+ 狠狠卷

薪资高 稳

这个是你微调的

省略号的地方, 需要你再补充一点

9月11日 下午17:38

上面你留着这个就行

易所 关于数字货币的

易所 也有打盹的时候, 该裁员, 照样一个不少

年包比 易所多19w

这么多

易所估计有70万

那你没有90个W?

易所给我68

就是吗

小伙8年经验 年薪60W

7月12日改简历 8月10日晒offer

秘诀: 改简历+ 狠狠卷

涨幅多少呀?

或者, 幅度多少呀?

之前36*15, 现在这个39*15

今年行情不太好, 还有一些 offer 基本都是持平, 没降薪的。

OK

这个马上来

刚在搞简历

哈哈

恭喜呀

这次找工作, 您的简历写得超厉害了。

我这次练习基本看的都是咱们课程的 简历。

6年小伙伴 年薪40W

9月6日改简历 9月21日晒offer

秘诀: 简历指导+ 狠狠卷

张这多 易所

今年这行情, 也算可以了

总包多少呀, 让我也围观一下

大概40w吧

谢谢恩师的指导和鼓励

是在深圳

深圳的行情尤其难

能有面试电话就不错了

是的

太牛啦

哈哈哈哈哈, 恩师的鼓励指导也很重要

5年小伙喜提3个offer 年薪35个W

5月22日改简历 11月29日晒offer

秘诀: 简历指导+ 狠狠卷

恩恩老师晚上好, 汇报下那说的 offer 情况, 最近面试收到了三个 offer, 两个是平薪, 一个是跨境电商公司的 offer, 涨幅暂时不到20%, 通过这三次面试也让我更加清楚自己距离高级开发还有一点点距离, 还要再多卷才能突破。

最后结合自己的情况, 先选择去跨境电商的公司再往下

昨天22:35

之前是20k*13.5, 跨境电商这个是23k* (14-15)

恭喜恭喜

9.9.9

就业寒冬, 能拿到3个 offer, 已经很不错了, 已经是高手了

很多小伙伴, 面试电话一个都没接到, 简历海投7000份, 只收到3次面试机会, 没有一个机会拿到最终 offer

您这个年薪, 算下来也有35W了吧

年终奖部分还要确认下, 大部分人听说只有两个月

这个时间点, 拿到这个水平, 挺不错的啦

持续加油卷哈

嘻嘻! 看看明年自己有没有能力冲击再开

把你视频都吃透

辛苦老师再帮忙指导下哈

1.5年小伙搞定15K offer

就业寒冬涨100%

5月7日改简历 11月21日晒offer

秘诀:
简历指导+狠狠卷

他那个不止再修修嘛，我才是两位，原先7k，现在15k

那也很牛

都翻倍了，都是牛人

正常就30%

好的

晚上指导你改

推送中台需要加不

我还没做完，准备八脱文新时没时间来搞这些，入职了会开始搞

OK

还有别的项目吗

一个项目，太少啦

那种练习项目也行

其实我工作一年，

下一步可以瞄准25k

慢下来

一年经验，搞定15k offer，舍你其谁

那你就更牛逼啦

卷王逆袭成功案例

6年小伙从很少面试机会到搞定35K*14薪

3月5日改简历 4月11日拿offer

一个月拿到了理想的offer

面试邀请少 小伙很苦恼

面试法宝 rocketmq四部曲

理想很丰满 目标35K

6年 经验小伙伴

喜收25K offer

3月12日改简历 12月1日晒offer

秘诀:
简历指导+狠狠卷

咱们以这个项目为模板改哈

项目有多少人，你带领多少人？

你工作几年了？

6年了

改简历那晚，20涨到25k，高级开发，P7带的后端团队

任务重，道路远，并不是没有方向

7年经验卷王

薪酬涨30%

7月11日改简历 9月1日晒offer

秘诀:
改简历+狠狠卷

只要本事好，拿offer比较容易的

入职了，最终并不输于面试官，还差劲的大佬，只涨了30%

小伙5月份"被毕业", 改简历后
斩获顶级央企Offer
涨薪7000+

5月29日改简历 7月5日晒offer

秘决:
简历指导+ 狠卷3高

快速看书, 就能不挂科, 把记录和阅读量大概一下, 然后看笔记的地方, 用刷的地方, 再走两步

尼恩 我被"毕业"了

这周末或下周我给你改一下简历

毕业了没有关系

ok, 发我吧

IT行业, 赶紧跑去, 大鹅索呀

嗯, 其实有点心理准备

5月29日 上午10:49

壹鼎VIP1002 简历指导-JAV...
35.0 KKB

5月29日 上午10:52

不太会写简历

尼恩 我拿到年前的 offer 了

涨 20%, 没数那么多。结果人家都不还价的

看起来不差吧

超过了 8000 没

平均算下来

7000 多

好的

有啥面试的心得吗

可以分享给其他小伙伴呀

1 面试用多问面试官问题, 2 面试官问的问题都准备好, 面试官问啥就回答啥

卷王逆袭成功案例

6年小伙喜提4个Offer 最高涨9k，年薪35W

4月14日改简历

**涨薪法宝：
改简历 + 狠狠卷**

5月17日晒offer

卷王逆袭成功案例

5年经验小伙收2个offer
最高涨薪8k，年薪42W

5月9日改简历 5月30日晒offer

秘诀：
简历指导+ 狠卷3高

以此为例
大家狠狼卷
打造最卷IT社群



卷王逆袭成功案例

非全日制 6年经验卷王
喜提3个Offer，年包30W

5月9日改简历 5月18日晒offer

面试法宝：
改简历+ 狠狼卷

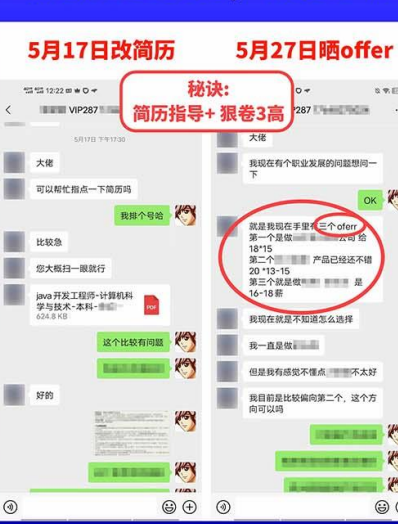


卷王逆袭成功案例

寒五冻六之际卷王大逆袭
收3大offer，涨30%

5月17日改简历 5月27日晒offer

秘诀：
简历指导+ 狠卷3高



卷王逆袭成功案例

4年卷王入职微软，涨50%

3月7日改简历 5月12日晒offer

涨薪法宝：
改简历+ 狠狼卷



4年小伙喜收百度、Boss直聘
等N个顶级Offer
最高涨幅100%

6月27日改简历

9月19日晒offer

秘诀:
改简历+狠狠卷

boss直聘和小满之间, boss那
边给的offer, 工资比小满高, 但是小满
那边给的offer, 工资比boss那边高, 我
不知道该怎么选, 如果你应该怎么选?
总体上是想着boss和小满之间
选一个
了
boss比小满高包多7w以上,
我这涨幅都快接近百分之百了

长那个那个思路结合上次指导的
内容改了几个点, 发现确实高大了
很多, 加上一套指标的说明确实会
让人眼前一亮的感觉。
明天睡醒时候有空了再和我看下哈,
一是有两个点我实在不知道该怎么
改了, 二是在排版表达这方面确实
不行, 还得让你帮忙把把关。

卷王逆袭成功案例

4年卷王入收2个offer, 涨50%

3月23日改简历

5月12日晒offer

offer决策
offer决策: n+1 电商erp, 部门成
熟, 人数多, 加班少
n+2 做业务中心 内部系
统, 新部门, 人数多, 加班多

涨薪法宝:
改简历+狠狠卷

涨了多少呀
地点在哪里
涨50%的样子
忘记你工作几年啦, 大概工作几年
呢

小伙大三暑期很焦虑
跟着尼恩卷一年
校招斩获顶级央企Offer

去年5月19日加入VIP群

今年7月5日晒offer

秘诀:
狠狠卷书+视频

尼恩老师
我校招去华润电力控股有限公司了
跟着你卷了一年 大学顺便拿了几个
国奖
不错不错, 这是央企

尼恩老师
总是很焦虑 自己算法这一
块卡住了
网盘里边有算法视频
去听一下吧
秋招来得及
谢谢大佬 不过经过几个月练习 看
你写的书比之前轻松多了
趁着还是学生这段时间 慢慢把知识
吃透
嗯嗯, 我的书, 比较深
期待大佬的下一本书, 已经迫不及待
去学习了

小伙高中学历
薪酬涨120%

5月6日改简历

7月22日晒offer

你这块估计要涨你668
\$668.00
哈哈, 不用了大佬, 你帮我改了
就行 今天晚上辅导就值几千了
老板很感谢
还有很多其他的模块
面试官要问
就是其他人做的
嗯嗯, 好

之前你的工资是多少的
翻了一翻
我涨你多少薪资来的
不知道, 原价给老板就行了
\$668.00
涨了一毛呢
咱们得说话算话呀
老板拿着您的那个简历开发改简历
点, 所向披靡。
ok
从头到尾给面试官讲的明明白白
后面继续狠狠卷哈

5年卷王喜收2大Offer
最高涨5K

5月19日改简历 9月13日晒offer

秘诀:
改简历 + 狠狠卷

发我吧

辛苦了

OK, 简历还再好改改哈

找主要介绍理由, 方法

嗯嗯, 我先消化一下。有问题在请教您

OK

目前薪资是18K, 目标想冲击到24K

嗯呀, 我跳槽成功了, 目前手上有两家相对心仪的公司, 想呀, 有空我参考下呗

恭喜恭喜

一家是外企, 主做IT云相关的, 给了22K, 一家是自研, 做AI推理的, 给22K

自研的是啥公司说

嗯呀, 今天有空看一看我的简历嘛? 帮我看优化优化

后面跟着简历在认真一遍

感谢

卷王逆袭成功案例
双非二本小伙春招大翻身
喜提9大offer

2月22日改简历 **4月13日晒offer**

15:53 周日 0天
h VIP 85 面试官
2月22日 周二 12:00
您好，能不能和我看看您的应届生校招简历呀
java 后端，熟练 redis 中间件，熟悉微服务，了解 docker
OK，稍等一下哈
2月23日 周三 12:07
好的谢谢哥哥
2月23日 周三 14:16
请问哥哥有时间了解，我也帮您了解一下~
2月23日 周三 15:16
没有的
稍微等一下哈，下午一点改下
2月23日 周三 15:48
好！
2月23日 周三 17:48

15:54 周日 0天
h VIP 85 面试官
2月23日 周三 12:00
欢迎技术，钻研技术的同学
好！面试官哥哥点
我研究一下
面试法宝：
改简历 + IM 求职
新鲜，您能给我下载校招的校招
还好是哥哥帮帮我修改了一下简历的
还有哥哥推荐的分布式 mvn 和 ops 框架
感谢哥哥呀
7:49

9大offer 最高年薪30万

公司	部门	岗位	薪资待遇	总包
1. 公司	能源数字化产品部	java后端开发	18.5w+1.5k*5k+200/月+500股权激励 <30w	
2. 公司	交易研发部	16w-14	22.4w	
3. 兄弟	待定	java游戏开发	15w+1k*5k+餐补+100/月	>22w
4. 公司	技术	全栈	13w-13	14.3w
5. 公司	技术	14k	>16.8w	
6. 公司	技术	9k		
7. 公司	技术	9k+包租房+饭钱津贴		
8. 公司	技术	12k+餐补+租房		
9. 公司	技术	17w		

修改简历找尼恩（资深简历优化专家）

- 如果面试表达不好，尼恩会提供 简历优化指导
- 如果项目没有亮点，尼恩会提供 项目亮点指导
- 如果面试表达不好，尼恩会提供 面试表达指导

作为 40 岁老架构师，尼恩长期承担技术面试官的角色：

- 从业以来，“阅历”无数，对简历有着点石成金、改头换面、脱胎换骨的指导能力。
- 尼恩指导过刚刚就业的小白，也指导过 P8 级的老专家，都指导他们上岸。

如何联系尼恩。尼恩微信，请参考下面的地址：

语雀：<https://www.yuque.com/crazymakercircle/gkkw8s/khigna>

码云：<https://gitee.com/crazymaker/SimpleCrayIM/blob/master/疯狂创客圈总目录.md>